Algorithms To Know For Interviews

Two Pointers

The **Two Pointers** algorithm is one of the most effective tools in a developer's kit for optimizing array and string problems. It transforms what would normally be a slow, exhaustive search into a sleek, linear process.

---

1. Problem it Solves and Why We Need It

Most beginners solve array problems using **Nested Loops** (a loop inside a loop). While this works, it is often incredibly slow.

If you have an array of 10,000 items, a nested loop performs 100,000,000 operations. This is known as **O(n^2)** complexity. The Two Pointers algorithm allows us to process the data in a single pass (10,000 operations), which is **O(n)** complexity.

We need it because:

- It reduces time complexity significantly.
- It uses almost no extra memory (Space Complexity stays low).
- It is the "gold standard" for sorted data or linked lists.

---

2. What Is It?

Simply put, Two Pointers is a strategy where you use two indices (variables usually named `left` and `right` or `i` and `j`) to iterate through a data structure at the same time. These pointers move independently based on certain conditions until they meet or reach the ends.

---

3. How Does It Work?

There are two main "scenarios" or patterns for Two Pointers:

Scenario A: Opposite Ends (The Squeeze)

You start one pointer at the beginning and one at the end. You move them toward each other. This is perfect for searching for pairs in a **sorted** array.

Walkthrough: Finding a pair that sums to 10 Array: `[1, 2, 4, 6, 8, 9]` | Target: 10

Step 1: `L` is at index 0 (value 1), `R` is at index 5 (value 9). Sum = 1 + 9 = 10. **Found it!** (Usually, if the sum was too small, you'd move `L` right; if too large, move `R` left).

Scenario B: Fast and Slow (The Tortoise and Hare)

Both pointers start at the same spot, but one moves faster than the other. This is the classic way to find the middle of a linked list or detect a "cycle" (a loop) in data.

Walkthrough: Finding a Cycle in a Linked List Imagine a list: `1 -> 2 -> 3 -> 4 -> 2 (back to 2)`

1. **Start:** `Slow` and `Fast` are at 1.
2. **Move 1:** `Slow` moves to 2. `Fast` jumps to 3.
3. **Move 2:** `Slow` moves to 3. `Fast` jumps from 3 to 2 (the loop).
4. **Move 3:** `Slow` moves to 4. `Fast` jumps from 2 to 4.
5. **Collision:** Since `Slow == Fast`, we know there is a loop!

---

4. Code Sample (JavaScript)

Here is a function that checks if a string is a palindrome (reads the same forward and backward) using two pointers.

```javascript
function isPalindrome(str) {
    // 1. Initialize pointers at the start and end
    let left = 0;
    let right = str.length - 1;

    while (left < right) {
        // 2. Compare the characters at both pointers
        if (str[left] !== str[right]) {
            // Characters don't match, not a palindrome
            return false;
        }

        // 3. Move pointers toward the center
        left++;
        right--;
    }

    // 4. If pointers meet without returning false, it is a palindrome
    return true;
}

// Example usage:
```

```
console.log(isPalindrome("racecar")); // true
console.log(isPalindrome("hello"));   // false
```

---

5. Leetcode Applications

- **Two Sum II (Sorted):** Start pointers at ends. If `sum < target`, move `left++`. If `sum > target`, move `right--`.
- **3Sum:** Sort the array. Fix one number, then use Two Pointers on the rest of the array to find the other two numbers.
- **Container With Most Water:** Pointers at ends. Calculate area. Move the pointer that points to the *shorter* line inward to try and find a taller one.
- **Remove Duplicates from Sorted Array:** Use a `slow` pointer to track the position of unique elements and a `fast` pointer to scan through all elements.

---

6. Time and Space Complexity

Time Complexity: O(n) Since each pointer travels at most `n` steps (the length of the array), and they never backtrack, the total operations are proportional to the size of the input.

```
Input Size (n) ---->
Pointer L: [------------>]
Pointer R: [<------------]
Total work = n operations = O(n)
```

Space Complexity: O(1) We only create two integer variables (`left` and `right`). We don't create new arrays or data structures, regardless of how big the input is.

```
Memory Used:
[left_var] + [right_var]
(Constant size, no matter if n is 10 or 10,000,000)
= O(1)
```

Variations and Tweaks

Variation 1: The "Sliding Window"

Sometimes the pointers move in the same direction to define a "range" or "window."

- **Tweak:** When a condition is met (like a sum being too high), you "shrink" the window from the left.

```
// Logic change:
while (currentSum > target) {
    currentSum -= arr[left];
    left++; // Shrink window
}
```

Variation 2: Moving at different speeds

Used for finding the middle or cycles.

- **Tweak:** The `fast` pointer moves twice as far as the `slow` one.

```
// Logic change:
slow = slow.next;
fast = fast.next.next;
```

Variation 3: Multiple Arrays

Comparing two different sorted arrays (like in Merge Sort).

- **Tweak:** Each pointer stays on its own array.

```
// Logic change:
if (arr1[p1] < arr2[p2]) {
    p1++;
} else {
    p2++;
}
```

Two Pointers with a Hashmap

Combining a **Hashmap** (or a plain Object in JavaScript) with **Two Pointers** is the secret sauce for solving problems involving **substrings** or **subarrays** where you need to track the frequency or presence of specific elements.

While Two Pointers alone help you navigate a range, the Hashmap acts as a "memory" for what is inside that range at any given moment.

When to use this combo?

You need this specific pairing when the problem asks for:

1. **Unique Elements:** "Find the longest substring with no repeating characters."

2. **Frequency Requirements:** "Find the shortest subarray that contains all characters of a target string."
3. **K-Distinct Items:** "Find a window containing exactly k different numbers."

How it works: The "Sliding Window" Pattern

In this scenario, the two pointers move in the same direction. The `right` pointer expands the window to "explore," and the `left` pointer shrinks the window to "rectify" a broken rule.

Walkthrough: Longest Substring Without Repeating Characters

Input: `pwwkew`

1. Expansion Phase:

- Move `right` pointer.
- Add character to Hashmap: `{ p: 1 }`.
- Move `right` pointer again: `{ p: 1, w: 1 }`.

2. The Conflict:

- Move `right` pointer to the second `w`.
- Hashmap now sees: `{ p: 1, w: 2 }`.
- **Rule Broken!** We have a duplicate `w`.

3. The Contraction (Shrinking) Phase:

- Move the `left` pointer forward to remove the old characters until the duplicate `w` is gone.
- Remove `p` from Hashmap: `{ w: 2 }`.
- Remove the first `w` from Hashmap: `{ w: 1 }`.
- Now the window is "valid" again, and we continue moving `right`.

The Code Tweak (JavaScript)

The key logic change is updating the map as the pointers move.

```javascript
let map = new Map();
let left = 0;
let maxLength = 0;

for (let right = 0; right < s.length; right++) {
    let char = s[right];

    // If char exists in map, move 'left' pointer to skip the duplicate
```

```
    if (map.has(char)) {
        // Move left pointer to the right of the previous seen index
        left = Math.max(left, map.get(char) + 1);
    }

    // Store/Update the last seen index of the character
    map.set(char, right);

    // Calculate current window size
    maxLength = Math.max(maxLength, right - left + 1);
}
```

Comparison: Why not just Two Pointers?

Feature	Two Pointers Only	Two Pointers + Hashmap
Best For	Sorted data, Sums, Reversing	Unsorted data, Character counts, Substrings
Memory	O(1) - Constant	O(k) - Size of the alphabet/unique items
Logic	Comparison of two values	Tracking state of many values in a range

---

Complexity

- **Time Complexity:** O(n). Even though there is a loop and the `left` pointer moves, each element is only visited twice (once by `right`, once by `left`).
- **Space Complexity:** O(k), where k is the number of unique elements stored in the Hashmap (e.g., O(26) for lowercase English letters).

Kadane's algorithm

Kadane's Algorithm Explained

1. The Problem It Solves and Why We Need It

The Problem: Kadane's Algorithm solves the "Maximum Subarray Problem." Given an array of integers (which can include both positive and negative numbers), the goal is to find a contiguous block of numbers (a subarray) within that array that adds up to the largest possible sum.

Why We Need It: If you try to solve this using a brute-force method (checking every possible subarray combination), you would need nested loops.

- Checking every subarray takes O(N^2) or even O(N^3) time complexity.
- For an array of 100,000 elements, brute force would take billions of operations, causing performance issues or timeouts.

Kadane's Algorithm drastically reduces this workload. It allows you to find the maximum subarray by passing through the array exactly once, reducing the time complexity to **O(N)**.

---

2. What Is It?

Kadane's Algorithm is an elegant application of Dynamic Programming (or a greedy approach, depending on how you look at it).

The core philosophy is simple: **A maximum subarray ending at a specific position is either the current element itself, or the current element combined with the maximum subarray ending at the previous position.**

At every single step as you walk through the array, you ask yourself: *"Is it better to add the current number to my running total, or is my running total dragging me down so much that I should just start a brand new subarray beginning with the current number?"*

---

3. How Does It Work?

You only need to keep track of two variables as you loop through the array:

1. **current_max:** The maximum sum of the subarray that ends at the current element you are looking at.
2. **global_max:** The highest subarray sum you have seen overall across the entire array.

At each element `x`, you update `current_max` by taking the maximum of two things: `x` itself, or `current_max + x`. Then, you update `global_max` if your new `current_max` is higher than the `global_max` on record.

Scenario A: Mixed Positive and Negative Numbers Input Array: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

Let's walk through it. We initialize our variables with the first element (`-2`).

- `current_max` = -2
- `global_max` = -2

```
================================================================================
Step | Element | Calculation for current_max     | New current_max | global_max
================================================================================
  1  |    1    | max(1, -2 + 1) -> max(1, -1)    |        1        | max(-2, 1) = 1
--------------------------------------------------------------------------------
  2  |   -3    | max(-3, 1 + -3) -> max(-3, -2)  |       -2        | max(1, -2) = 1
--------------------------------------------------------------------------------
  3  |    4    | max(4, -2 + 4) -> max(4, 2)     |        4        | max(1, 4) = 4  <-- Rese
--------------------------------------------------------------------------------
  4  |   -1    | max(-1, 4 + -1) -> max(-1, 3)   |        3        | max(4, 3) = 4
--------------------------------------------------------------------------------
  5  |    2    | max(2, 3 + 2) -> max(2, 5)      |        5        | max(4, 5) = 5
--------------------------------------------------------------------------------
  6  |    1    | max(1, 5 + 1) -> max(1, 6)      |        6        | max(5, 6) = 6  <-- High
--------------------------------------------------------------------------------
  7  |   -5    | max(-5, 6 + -5) -> max(-5, 1)   |        1        | max(6, 1) = 6
--------------------------------------------------------------------------------
  8  |    4    | max(4, 1 + 4) -> max(4, 5)      |        5        | max(6, 5) = 6
================================================================================
Result: global_max is 6. The subarray is [4, -1, 2, 1].
```

What happened at Step 3? The running total was `-2`. When we reached the number `4`, the algorithm realized: "Why add `-2` to `4` and get `2`, when I can just start fresh at `4`?" It dropped the baggage of the past and started a new subarray.

Scenario B: All Negative Numbers Input Array: `[-5, -2, -9]`

If an array contains only negative numbers, the largest sum is simply the smallest negative number (the one closest to zero). Kadane's handles this gracefully.

Initialize with first element (`-5`):

- `current_max` = -5
- `global_max` = -5

```
================================================================================
Step | Element | Calculation for current_max     | New current_max | global_max
================================================================================
  1  |   -2    | max(-2, -5 + -2) -> max(-2, -7) |       -2        | max(-5, -2) = -2
--------------------------------------------------------------------------------
  2  |   -9    | max(-9, -2 + -9) -> max(-9, -11)|       -9        | max(-2, -9) = -2
================================================================================
Result: global_max is -2. The subarray is [-2].
```

---

4. Code Sample (JavaScript)

```javascript
/**
 * Finds the maximum contiguous subarray sum using Kadane's Algorithm.
 * @param {number[]} nums - An array of integers.
 * @return {number} - The sum of the maximum subarray.
 */
function maxSubArray(nums) {
    // Edge case: If the array is empty, return 0 or throw an error based on requirements.
    if (nums.length === 0) return 0;

    // Initialize both tracking variables to the first element of the array.
    // We do not initialize to 0, because if the array is all negative numbers,
    // 0 would incorrectly be returned as the maximum sum.
    let currentMax = nums[0];
    let globalMax = nums[0];

    // Start the loop from the second element (index 1)
    for (let i = 1; i < nums.length; i++) {
        // Core Kadane's logic:
        // Is the current element bigger than the current element + previous sum?
        // If it is, that means the previous sum was negative and pulling us down,
        // so we start a new subarray right here.
        currentMax = Math.max(nums[i], currentMax + nums[i]);

        // If our current running subarray sum is higher than the best we've seen,
        // update the global maximum.
        if (currentMax > globalMax) {
            globalMax = currentMax;
        }
    }

    return globalMax;
}

// Example Usage:
console.log(maxSubArray([-2, 1, -3, 4, -1, 2, 1, -5, 4])); // Output: 6
```

---

5. LeetCode Questions It Applies To

Here are the most common LeetCode problems where Kadane's is the star player:

1. **LeetCode 53: Maximum Subarray**

- **Summary:** This is the exact implementation of standard Kadane's Algo-

rithm. No tweaks needed.

2. **LeetCode 152: Maximum Product Subarray**

- **Summary:** Uses a variation of Kadane's. Because multiplying two negative numbers creates a positive number, you must track *both* the `current_max` and the `current_min` at each step.

3. **LeetCode 918: Maximum Sum Circular Subarray**

- **Summary:** The array loops around. You solve this by finding the max subarray (normal Kadane's) and the min subarray (inverted Kadane's). The answer is either the normal max, or the total array sum minus the min subarray.

4. **LeetCode 121: Best Time to Buy and Sell Stock**

- **Summary:** While often solved with a simple two-pointer approach, it can be framed as Kadane's by finding the differences between consecutive days and finding the maximum subarray sum of those differences.

---

6. Time and Space Complexity

Time Complexity: O(N) We loop through the array exactly one time.

```
Derivation:
Array Size = N elements

Start ---> [ Loop i from 1 to N ] ---> Return Result
                  |
                  v
          Operations per loop:
          1. Addition (currentMax + nums[i])      --> O(1)
          2. Math.max() comparison                --> O(1)
          3. globalMax > currentMax comparison    --> O(1)
          4. Variable reassignment                --> O(1)

Total Time = (N-1) iterations * O(1) operations
Total Time = O(N)
```

Space Complexity: O(1) We are only allocating memory for two variables (`currentMax` and `globalMax`), regardless of how massive the input array gets.

```
Derivation:
Memory Used:
- currentMax (Integer) -> Constant Space
- globalMax (Integer)  -> Constant Space
- i (Iterator)         -> Constant Space
```

```
Since no arrays, objects, or matrices are created that scale with N:
Total Space = O(1)
```

---

7. Variations and Common Tweaks

Variation 1: Return the Subarray Indices Sometimes the interviewer doesn't just want the maximum sum; they want to know *where* the subarray starts and ends.

Tweak Required: You need to introduce pointers to track the start, end, and a temporary start index. Every time `currentMax` decides to start fresh at the current element, you update `tempStart`. Every time `globalMax` is updated, you lock in the `start` and `end` indices.

```
// Snippet for tracking indices
let start = 0, end = 0, tempStart = 0;

for (let i = 1; i < nums.length; i++) {
    if (nums[i] > currentMax + nums[i]) {
        currentMax = nums[i];
        tempStart = i; // Reset the starting point
    } else {
        currentMax = currentMax + nums[i];
    }

    if (currentMax > globalMax) {
        globalMax = currentMax;
        start = tempStart; // Lock in the start
        end = i;           // Lock in the end
    }
}
// Now return array.slice(start, end + 1)
```

Variation 2: Maximum Product Subarray Instead of adding, we multiply. The trick here is that a huge negative number can instantly become a huge positive number if the next element is negative.

Tweak Required: Track the minimum product alongside the maximum product. Swap them if the current number is negative.

```
// Snippet for Maximum Product Subarray
let currentMax = nums[0], currentMin = nums[0], globalMax = nums[0];

for (let i = 1; i < nums.length; i++) {
    let num = nums[i];
```

```
    // If we hit a negative number, the max becomes the min, and min becomes max
    if (num < 0) {
        let temp = currentMax;
        currentMax = currentMin;
        currentMin = temp;
    }

    // Now proceed with Kadane's logic for both max and min
    currentMax = Math.max(num, currentMax * num);
    currentMin = Math.min(num, currentMin * num);

    globalMax = Math.max(globalMax, currentMax);
}
```

Sliding Window

The **Sliding Window** algorithm is a powerful technique used to transform nested loops (which are slow) into a single loop (which is fast). It is a staple for coding interviews because it tests your ability to manage pointers and optimize performance.

---

1. The Problem it Solves and Why We Need It

Imagine you have a long array of numbers and you need to find the maximum sum of **3 consecutive numbers**.

- **The "Slow" Way (Brute Force):** You look at the first 3, add them. Then you move to the next position and add the next 3. In this process, you are re-calculating the middle numbers over and over again.
- **The "Window" Way:** You calculate the sum of the first 3. To get the next sum, you just **subtract** the number that is leaving the group and **add** the new number entering the group.

We need it because it reduces the time complexity from **O(N * K)** to **O(N)**, making your code run significantly faster on large datasets.

---

2. What is it?

A "window" is a sub-section of a larger data structure (like an array or string).

- The window can be **Fixed** (e.g., always 5 elements wide).
- The window can be **Dynamic** (it grows or shrinks based on a condition, like "find the shortest sub-array that adds up to 10").

---

3. How Does It Work?

Scenario A: Fixed Window (Max Sum of K elements)

Input: `[2, 1, 5, 1, 3, 2]`, `K = 3`

Walkthrough:

1. **Initial Window:** Sum the first 3 elements `[2, 1, 5]`. Sum = 8.
2. **Slide:** Move to the next element (`1`).

- Subtract the element leaving (`2`).
- Add the element entering (`1`).
- New Sum: `8 - 2 + 1 = 7`.

3. **Slide:** Move to the next element (`3`).

- Subtract `1`, add `3`.
- New Sum: `7 - 1 + 3 = 9`.

4. **Result:** The maximum sum found was 9.

Scenario B: Dynamic Window (Shortest Sub-array with Sum >= S)

Input: `[2, 3, 1, 2, 4, 3]`, `Target = 7`

Detailed Walkthrough:

- **Step 1:** Start both `Left` and `Right` pointers at index 0.
- Window: `[2]`, Sum: 2 (Less than 7, keep expanding).
- **Step 2:** Move `Right` to index 2.
- Window: `[2, 3, 1, 2]`, Sum: 8.
- **Condition Met!** Sum is 8 (>= 7). Length is 4.
- **Step 3:** Try to shrink from the `Left` to find a *shorter* window.
- Remove `2`. Window: `[3, 1, 2]`, Sum: 6.
- Sum is now < 7. Stop shrinking.
- **Step 4:** Move `Right` again.
- Window: `[3, 1, 2, 4]`, Sum: 10.
- **Condition Met!** Shrink from `Left`.
- Remove `3`. Window: `[1, 2, 4]`, Sum: 7. (Length: 3).
- **Step 5:** Final check.
- Window `[4, 3]` also equals 7. (Length: 2).

- **Result:** Shortest length is 2.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Fixed Window Example: Max Sum of Subarray of size K
 */
function maxSubarraySum(arr, k) {
  let maxSum = 0;
  let windowSum = 0;

  // 1. Create the initial window
  for (let i = 0; i < k; i++) {
    windowSum += arr[i];
  }
  maxSum = windowSum;

  // 2. Slide the window across the rest of the array
  for (let i = k; i < arr.length; i++) {
    // Add the next element, subtract the first element of the previous window
    windowSum += arr[i] - arr[i - k];
    maxSum = Math.max(maxSum, windowSum);
  }

  return maxSum;
}

const result = maxSubarraySum([2, 1, 5, 1, 3, 2], 3);
console.log(result); // Output: 9
```

---

5. LeetCode Applications

1. **Maximum Sum Subarray of Size K:** (Fixed Window) As shown above, maintain a running sum.
2. **Longest Substring Without Repeating Characters:** (Dynamic Window) Use a Map to store characters. If you see a duplicate, shrink the window from the left until the duplicate is gone.
3. **Minimum Size Subarray Sum:** (Dynamic Window) Expand `Right` until sum >= target, then shrink `Left` as much as possible while maintaining the sum.

---

6. Complexity Analysis

Time Complexity: O(n)

Even though there are two pointers (or a nested-looking while loop for shrinking), each element is visited at most **twice**: once by the `Right` pointer (entering) and once by the `Left` pointer (leaving).

```
n = array length
Each element enters window: 1 * n
Each element leaves window: 1 * n
Total operations: 2n
Since we drop constants: O(n)
```

Space Complexity: O(1) or O(k)

- **O(1):** If we only track sums or counts in variables.
- **O(k) / O(n):** If we use a Hash Map or Frequency Array to store the counts of elements inside the window.

Variations & Common Tweaks

Variation 1: The "At Most K Distinct" Tweak

Sometimes you need a window with at most K unique items. You use a Hash Map to track frequencies. **The Tweak:**

```
// When adding Right:
map.set(char, (map.get(char) || 0) + 1);

// When shrinking Left:
map.set(leftChar, map.get(leftChar) - 1);
if (map.get(leftChar) === 0) map.delete(leftChar);
```

Variation 2: The "String Anagram" Tweak

When comparing two strings, the window size is usually fixed to the length of the pattern. **The Tweak:** Instead of a sum, you compare two frequency maps or arrays of size 26.

```
// THE TWEAK: While the window contains ALL required characters
while (have === required) {

  // 1. Capture the 'Minimum' length found so far
  if ((right - left + 1) < minLen) {
    minLen = right - left + 1;
    result = [left, right];
  }
```

```
  // 2. Shrink from the left to find an even smaller valid window
  let leftChar = s[left];
  windowCounts[leftChar]--;

  // 3. If removing this char makes the window 'invalid', decrement 'have'
  if (needCounts[leftChar] && windowCounts[leftChar] < needCounts[leftChar]) {
    have--;
  }

  left++; // Move the left boundary forward
}
```

- Expansion: The right pointer moves until the window is "satisfied."
- Contraction: The left pointer moves (the while loop) to find the absolute smallest version of that satisfied window.
- Complexity: Even with the nested while, each pointer only travels from 0 to n once, keeping it O(n).

Sliding Window with a Hashmap

We need to combine a **Sliding Window** with a **Hashmap** when the problem requires us to track the **frequency**, **existence**, or **last seen position** of specific elements (usually characters or numbers) within the current window.

While a basic sliding window tracks a simple metric like a `sum` or `count`, the Hashmap allows us to handle more complex "state" inside that window.

---

1. When to Use This Combination

A. Tracking Unique Elements (Distinct Items)

If the problem asks for a window containing "at most K distinct elements" or "exactly K unique characters," a simple sum won't work. You need a Hashmap to store:

- **Key:** The element (e.g., character 'a').
- **Value:** How many times 'a' appears in the current window.

B. Finding the Longest Substring Without Repeating Characters

To know if a character is a "duplicate," you must remember what you've already seen. A Hashmap (or a Set) acts as the window's "memory."

C. Pattern Matching (Anagrams)

If you are looking for an anagram of a string `p` within a string `s`, you need to know if the window in `s` has the exact same character counts as `p`. You compare the Hashmap of your window against the Hashmap of the pattern.

---

2. How the Logic Changes

In a basic window, you just add/subtract a number. With a Hashmap, you have to manage the "life cycle" of a key.

The "Expansion" Logic (Right Pointer)

When the `right` pointer moves, you increment the count in the map.

```
let char = s[right];
map.set(char, (map.get(char) || 0) + 1);
```

The "Shrink" Logic (Left Pointer)

When the window becomes invalid (e.g., too many unique characters), you decrement the count. **Crucially**, if the count hits zero, you must delete the key so the `map.size` remains accurate.

```
let leftChar = s[left];
map.set(leftChar, map.get(leftChar) - 1);

if (map.get(leftChar) === 0) {
    map.delete(leftChar); // This is the most important step!
}
left++;
```

---

3. Example Walkthrough

Problem: Longest substring with at most 2 distinct characters. **Input:** `eceba`

```
Window: [e]       Map: {e:1}           Distinct: 1 (OK)
Window: [e, c]    Map: {e:1, c:1}      Distinct: 2 (OK)
Window: [e, c, e] Map: {e:2, c:1}      Distinct: 2 (OK)
Window: [e, c, e, b] -> Map: {e:2, c:1, b:1} Distinct: 3 (Too many!)

SHRINK TIME:
Remove 'e': Map {e:1, c:1, b:1} (Still 3)
Remove 'c': Map {e:1, b:1}      (Distinct: 2! OK)
Final Window: [e, b]
```

---

4. Common Tweaks for Map Variations

The "Last Index" Map

Instead of counting frequencies, you store the **last index** where a character was seen. This allows the `left` pointer to "jump" instead of sliding one by one.

- **Usage:** Longest Substring Without Repeating Characters.
- **Snippet:** `left = Math.max(left, map.get(char) + 1);`

The "Required Count" Tweak

Sometimes you use a variable `counter` alongside the map to track how many characters have met a specific frequency requirement. This prevents you from having to iterate over the whole map to check if the window is valid.

- **Usage:** Smallest window containing all characters of another string (Minimum Window Substring).

Sliding Window with Monotonic Queue

Here is a complete guide to understanding and implementing the Sliding Window with a Monotonic Queue algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are looking at a stock chart through a small "window" that only shows you 3 days at a time. As time moves forward, the window slides one day to the right. Your boss asks you: "What was the highest stock price inside the window at every single step?"

Why the "Naive" Approach fails: The obvious solution is to stop at every step, loop through the 3 days in your window, and find the maximum. If your array has N elements and your window size is K, this takes O(N * K) time. If you have 100,000 data points and a window size of 10,000, you are doing 1,000,000,000 operations. It's incredibly slow.

Why we need the Monotonic Queue: This algorithm acts as a highly efficient filter that instantly knows the maximum (or minimum) of the current window without having to scan through all K elements every time. It reduces the time complexity from O(N * K) down to a beautiful **O(N)**.

---

2. What is it?

A **Queue** is a standard data structure where the first item in is the first item out (like a line at a grocery store). **Monotonic** is a math word that simply means "always going in one direction" (either always increasing, or always decreasing).

Therefore, a **Monotonic Queue** is a special queue that forces its elements to stay sorted.

- To find **Maximums**, we use a **Monotonically Decreasing Queue** (Largest at the front, smallest at the back).
- To find **Minimums**, we use a **Monotonically Increasing Queue** (Smallest at the front, largest at the back).

The Secret Weapon (Storing Indices): Instead of storing the actual numbers in the queue, we store their **Indices** (their positions in the array). This is crucial because it allows us to know exactly *when* an element falls out of the sliding window and needs to be deleted.

---

3. How does it work? (Walkthrough)

The entire algorithm relies on one logical realization: **"The Younger and Taller Rule"**. Imagine a basketball team where players enter the team and leave the team based on their age (their index in the array). If a new player joins who is younger (entered later) AND taller (bigger number) than an existing player, the older/shorter player will *never* be the tallest person on the team again. We can fire them immediately to save space!

Let's walk through finding the **Sliding Window Maximum** with Window Size `K = 3`. **Input Array:** `[1, 3, -1, -3, 5]`

At every single step (index `i`), we do 3 things in order:

1. **Clean up the Front (Expire):** Is the guy at the front of the queue so old that he's no longer inside our window? If yes, kick him out!
2. **Clean up the Back (The Younger/Taller Rule):** Are the guys at the back of the queue smaller than our current number? If yes, kick them out! They are useless now.
3. **Add to the Back:** Put our current index at the back of the queue.

```
Input: [1, 3, -1, -3, 5] | Window Size K = 3

| Step | i | Value | Queue (Stores Indices) | Queue Values  | Window Max |
|------|---|-------|------------------------|---------------|------------|
|  1   | 0 |   1   | [0]                    | [1]           | (Not full) |
```

Walkthrough Step 1: Window isn't full yet (we need 3 items). Just add index 0.

```
|  2   | 1 |   3   | [1]                    | [3]           | (Not full) |
```

Walkthrough Step 2: * Clean Front: Index 0 is still valid.

- Clean Back: The value at index 0 is `1`. Our new value is `3`. 3 is bigger than 1! Kick index 0 out.
- Add: Add index 1.

```
|  3   | 2 |  -1   | [1, 2]                  | [3, -1]       | 3           |
```

Walkthrough Step 3:

- Clean Front: Index 1 is still inside window [0, 1, 2].
- Clean Back: Value `-1` is NOT bigger than `3`. Do nothing.
- Add: Add index 2.
- *Window is now size 3!* The answer for this window is always the front of the queue: `arr[Queue[0]] -> arr[1] -> 3`.

```
|  4   | 3 |  -3   | [1, 2, 3]               | [3, -1, -3]   | 3           |
```

Walkthrough Step 4:

- Clean Front: Index 1 is still inside window [1, 2, 3].
- Clean Back: Value `-3` is NOT bigger than `-1`. Do nothing.
- Add: Add index 3.
- *Record Answer:* Front of queue is index 1. Value is `3`.

```
|  5   | 4 |   5   | [4]                     | [5]           | 5           |
```

Walkthrough Step 5:

- Clean Front: Index 1 is OUTSIDE window [2, 3, 4]! (4 - 1 >= 3). Kick index 1 out! Queue is now `[2, 3]`.
- Clean Back: New value `5`. Is 5 bigger than queue back (index 3, value -3)? YES! Kick it out. Is 5 bigger than new queue back (index 2, value -1)? YES! Kick it out.
- Add: Add index 4.
- *Record Answer:* Front of queue is index 4. Value is `5`.

Final Output Array of Maximums: `[3, 3, 5]`

---

4. Code Sample (JavaScript)

```javascript
/**
 * Finds the maximum element in every sliding window of size K
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
function maxSlidingWindow(nums, k) {
    const result = [];
    const deque = []; // We will store INDICES here, not values
```

```
    for (let i = 0; i < nums.length; i++) {
        // Step 1: Clean up the Front (Remove expired indices)
        // If the index at the front of the queue is outside our window of size K, drop it.
        // The condition "i - deque[0] >= k" means the distance between our current
        // step and the oldest step is too large.
        if (deque.length > 0 && i - deque[0] >= k) {
            deque.shift();
        }

        // Step 2: Clean up the Back (The Younger and Taller rule)
        // While the queue has items AND the current number is GREATER THAN OR EQUAL TO
        // the number at the back of the queue, remove the back of the queue.
        while (deque.length > 0 && nums[i] >= nums[deque[deque.length - 1]]) {
            deque.pop();
        }

        // Step 3: Add the current index to the back of the queue
        deque.push(i);

        // Step 4: Record the answer
        // We only start recording once our window has reached size K.
        // i goes 0, 1, 2... so when i reaches K - 1, we have a full window.
        if (i >= k - 1) {
            // The maximum element for this window is ALWAYS at the front of the queue
            result.push(nums[deque[0]]);
        }
    }

    return result;
}

// --- Usage ---
const stockPrices = [1, 3, -1, -3, 5, 3, 6, 7];
const windowSize = 3;
console.log(maxSlidingWindow(stockPrices, windowSize));
// Output: [ 3, 3, 5, 5, 6, 7 ]
```

(Note: In pure JavaScript, `shift()` takes O(N) time. For true O(1) removals in highly strict environments, you would use a custom Doubly Linked List instead of an array, or just use a `leftIndex` pointer to simulate shifts. But the logic remains exactly the same!)

---

5. LeetCode questions it applies to

- **Sliding Window Maximum (LeetCode 239):** This is the exact algorithm explained above. You maintain a monotonically decreasing queue of indices and record the front of the queue once the window is full.
- **Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit (LeetCode 1438):** A brilliant variation. You don't have a fixed window size `K`. Your window shrinks and grows. You maintain TWO monotonic queues simultaneously: one decreasing (to track the window's max) and one increasing (to track the window's min). If `Max - Min > limit`, you shrink the left side of your window and clean the fronts of both queues.
- **Shortest Subarray with Sum at Least K (LeetCode 862):** A brutal "Hard" problem. It combines Prefix Sums with a Monotonic Increasing Queue. You store prefix sums in the queue. You check if `currentPrefixSum - queueFront >= K`. The queue helps you instantly find the closest valid starting point for your subarray without having to check everything backwards.

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Iterate through array  | N steps              | O(N)
2. Add index to deque     | 1 push per element   | O(1)
3. Remove from deque      | Elements popped      | O(1) amortized
-------------------------------------------------------
Total Time Complexity: O(N)
*Why is popping O(1)? Even though there is a 'while' loop
inside the 'for' loop, every single index is pushed into
the queue exactly ONCE, and popped exactly ONCE. N pushes
and N pops total means constant time per element!

Space Complexity Derivation:
-------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Deque storage          | Max K elements       | O(K)
-------------------------------------------------------
Total Space Complexity: O(K) (The queue never grows larger
than the window size).
```

---

7. Variations and Common Tweaks

Variation A: Finding the Minimums (Monotonic Increasing Queue) **The Problem:** The question asks for the minimum element in the window, not the maximum. **The Tweak:** You simply flip the comparison sign in Step 2. Instead of kicking out elements that are *smaller* than the current number, you kick out elements that are *larger* than the current number.

Snippet for Minimums:

```
// Change >= to <= to create an Increasing Queue
while (deque.length > 0 && nums[i] <= nums[deque[deque.length - 1]]) {
    deque.pop();
}
```

Variation B: The Dynamic Window (Two Queues) **The Problem:** The window size is not fixed. You must find the longest subarray where the difference between the maximum and minimum elements is `<= Limit`. **The Tweak:** Use a standard sliding window with `left` and `right` pointers. Inside the window, use a `maxQueue` and a `minQueue`. If the difference between the front of `maxQueue` and the front of `minQueue` breaks the limit, you increment the `left` pointer to shrink the window, dropping expired elements from the fronts of both queues along the way.

Snippet for Dynamic Two-Queue Window:

```
let left = 0;
let maxLength = 0;
// ... (inside the loop where 'right' moves forward and updates both queues)

// If the limit is broken, shrink the window from the left
while (nums[maxQueue[0]] - nums[minQueue[0]] > limit) {
    // If the element falling out of the window is our max/min, remove it from queue
    if (maxQueue[0] === left) maxQueue.shift();
    if (minQueue[0] === left) minQueue.shift();
    left++; // Shrink window
}

// Update max length
maxLength = Math.max(maxLength, right - left + 1);
```

Sweep Line

Here is a complete breakdown of the Sweep Line algorithm, designed to be simple to understand but detailed enough to help you master it.

---

1. Problem it Solves and Why We Need It

Imagine you are looking at a map with 10,000 overlapping rectangular buildings, and you need to find out how many buildings intersect with each other.

If you use a **Brute Force** approach, you have to take building #1 and check it against the other 9,999 buildings. Then take building #2 and check it against the remaining 9,998 buildings, and so on. This takes roughly N * N (or N^2) operations. For millions of objects, this becomes incredibly slow and computationally expensive.

Why we need Sweep Line: The Sweep Line algorithm solves this by organizing the chaos. It realizes that if building A is in New York and building B is in Los Angeles, we shouldn't even waste time checking if they overlap. It reduces the time complexity drastically by only comparing objects that are actively existing at the same point in space or time.

2. What is it?

The Sweep Line is a conceptual approach. Imagine a vertical line (like a laser scanner or a windshield wiper) moving slowly across a 2D plane from left to right (or top to bottom).

As this line moves, it stops only at specific points of interest called **"Events"**.

- An **Event** is usually the start or the end of an object (like the left edge of a rectangle, or the start time of a meeting).
- The algorithm maintains an **"Active Set"** (or State). This is a record of all the objects the sweep line is currently touching.
- When the line hits a "Start" event, we add that object to the active set. When it hits an "End" event, we remove it.

---

3. How does it work? (Walkthroughs)

Let's look at two scenarios, from simple to complex.

Scenario A (Simple): 1D Meeting Rooms

Input: Three meetings happening at time intervals: [1, 4], [2, 5], [7, 9]. We want to know the maximum number of overlapping meetings.

Step 1: Create Events Break every meeting into a "Start" (+1 room needed) and "End" (-1 room needed). Events: (1, Start), (4, End), (2, Start), (5, End), (7, Start), (9, End)

Step 2: Sort Events by Time

1. (1, Start)
2. (2, Start)

3. (4, End)
4. (5, End)
5. (7, Start)
6. (9, End)

Step 3: Sweep Line Execution

```
Timeline:  0--1--2--3--4--5--6--7--8--9--10
Meeting 1:    [========]
Meeting 2:       [========]
Meeting 3:                      [====]

Sweep Line sweeping left to right:
Time 1: Start M1. Active Rooms = 1. (Max so far = 1)
Time 2: Start M2. Active Rooms = 2. (Max so far = 2)  <-- Sweep line intersects M1 & M2
Time 4: End M1.   Active Rooms = 1. (Max so far = 2)
Time 5: End M2.   Active Rooms = 0. (Max so far = 2)
Time 7: Start M3. Active Rooms = 1. (Max so far = 2)
Time 9: End M3.   Active Rooms = 0. (Max so far = 2)

Result: 2 rooms needed.
```

Scenario B (Complex): The Skyline Problem

Input: 2D Buildings defined by [left_x, right_x, height]. Buildings: A = [1, 5, 10], B = [3, 7, 15] We want to draw the outer contour (skyline) of these overlapping buildings.

Step 1: Create Events We need to track the height. Start events bring a height INTO the active set. End events remove a height. Events: (1, Start A, h:10), (5, End A, h:10), (3, Start B, h:15), (7, End B, h:15)

Step 2: Sort Events by X-Coordinate

1. (1, Start A, h:10)
2. (3, Start B, h:15)
3. (5, End A, h:10)
4. (7, End B, h:15)

Step 3: Sweep Line Execution *Note: Our "Active Set" must now be a data structure that can always tell us the CURRENT MAXIMUM height. A Max-Heap or ordered map is perfect for this.*

```
Y (Height)
15 |      [========B========]
10 |  [========A========]
 5 |  |                 |
 0 +--1---2---3---4---5---6---7---> X
```

```
Event 1 (X=1, Start A):
- Add height 10 to Active Set.
- Max height is now 10.
- Skyline point: (1, 10)

Event 2 (X=3, Start B):
- Add height 15 to Active Set. Active = {10, 15}.
- Max height changes to 15!
- Skyline point: (3, 15)

Event 3 (X=5, End A):
- Remove height 10 from Active Set. Active = {15}.
- Max height is still 15. (No change to skyline contour here).

Event 4 (X=7, End B):
- Remove height 15 from Active Set. Active = {0} (ground).
- Max height changes to 0!
- Skyline point: (7, 0)

Final Contour Output: [ [1,10], [3,15], [7,0] ]
```

---

4. Code Sample (JavaScript)

Here is a clean implementation of the simpler **Meeting Rooms II** problem to show the core skeleton of Sweep Line.

```javascript
/**
 * Finds the minimum number of meeting rooms required.
 * @param {number[][]} intervals - Array of [start, end] times
 * @return {number}
 */
function minMeetingRooms(intervals) {
    if (!intervals || intervals.length === 0) return 0;

    const events = [];

    // 1. Generate Events
    for (let i = 0; i < intervals.length; i++) {
        // We use +1 to denote a room is taken (start)
        // We use -1 to denote a room is freed (end)
        events.push([intervals[i][0], 1]);
        events.push([intervals[i][1], -1]);
    }

    // 2. Sort Events
```

```
    events.sort((a, b) => {
        // If times are the same, process the 'End' (-1) event BEFORE the 'Start' (1) event
        // This ensures back-to-back meetings don't artificially increase room count.
        if (a[0] === b[0]) {
            return a[1] - b[1];
        }
        // Otherwise, sort by time (x-coordinate) ascending
        return a[0] - b[0];
    });

    let maxRooms = 0;
    let currentActiveRooms = 0;

    // 3. Sweep the Line
    for (let i = 0; i < events.length; i++) {
        const eventType = events[i][1];
        currentActiveRooms += eventType; // Add 1 or subtract 1

        // Record the peak of the active set
        if (currentActiveRooms > maxRooms) {
            maxRooms = currentActiveRooms;
        }
    }

    return maxRooms;
}
```

---

5. LeetCode Questions it Applies To

1. **Merge Intervals (LeetCode #56):**

- *Summary:* Sort by start times. As you sweep, keep a running end-time boundary. If the next interval's start is before your current running end-time boundary, stretch the boundary. If it's after, you've found a clean break; push the merged interval to your result array and start a new one.

2. **Meeting Rooms II (LeetCode #253):**

- *Summary:* Deconstruct into start (+1) and end (-1) events. Sort them. Keep a running tally as you sweep left to right. The maximum value that tally ever hits is your answer.

3. **The Skyline Problem (LeetCode #218):**

- *Summary:* Events are the left and right walls of buildings. As you sweep, use a Max-Heap or sorted tree to keep track of all active building heights.

Whenever the maximum height in your heap changes, record the current X coordinate and the new max height as a skyline key-point.

4. **Rectangle Area II (LeetCode #850):**

- *Summary:* You are given 2D rectangles. Sweep a vertical line from left to right. At each X-coordinate event, look at the active Y-intervals (heights). Calculate the active height and multiply it by the horizontal distance moved since the last X-event to get the area of that specific vertical "slice".

---

6. Time and Space Complexity

Time Complexity: O(N log N)

The dominant operation is always sorting the events. Processing them via the sweep is linear.

```
Derivation:
-------------------------------------------------------
[Operation]              [Cost per item]   [Total Time]
1. Creating Events:      O(1) * 2N items = O(N)
2. Sorting Events:       Sorting 2N items= O(N log N)  <-- Dominant
3. Sweeping/Processing:  O(1) * 2N items = O(N)
-------------------------------------------------------
Overall Time Complexity: O(N log N)
```

(Note: For complex variations like Skyline, inserting/removing into a Max-Heap during step 3 takes log N time, making step 3 take O(N log N) as well. The overall complexity remains O(N log N)).

Space Complexity: O(N)

```
Derivation:
-------------------------------------------------------
[Data Structure]         [Size]            [Total Space]
1. Events Array:         Stores 2N events= O(N)
2. Active Set/State:     Stores up to N  = O(N)
-------------------------------------------------------
Overall Space Complexity: O(N)
```

---

7. Variations and Common Tweaks

Algorithms change slightly based on the strictness of overlaps and the dimensions of the problem.

Tweak 1: Handling Tie-Breakers (Same X-coordinate)

If an object ends exactly when another begins (e.g., [1, 5] and [5, 10]), does that count as an overlap?

- **If it IS NOT an overlap (like Meeting Rooms):** Process End events *before* Start events.

```
// Snippet for sorting End before Start on tie
events.sort((a, b) => {
    if (a.time === b.time) return a.type - b.type; // Assuming -1 is End, 1 is Start
    return a.time - b.time;
});
```

- **If it IS an overlap (like maximum points overlapping a line):** Process Start events *before* End events.

```
// Snippet for sorting Start before End on tie
events.sort((a, b) => {
    if (a.time === b.time) return b.type - a.type; // 1 (Start) comes before -1 (End)
    return a.time - b.time;
});
```

Tweak 2: Upgrading the "Active Set" Data Structure

- **Simple Count (1D):** If you only need to know *how many* objects overlap, a simple integer variable `activeCount` is enough (like the code sample).
- **Set/Map (1D):** If you need to know *which* specific objects overlap, use a Hash Set to add/remove IDs.
- **Max-Heap/Priority Queue (2D):** If you need to know the *maximum or minimum* attribute of the active overlapping items (like the tallest building in the Skyline problem), you must use a Heap.

```
// Snippet for using a Max-Heap as the Active Set (Conceptual)
for (let event of events) {
    if (event.isStart) {
        maxHeap.insert(event.height);
    } else {
        // For End events, we mark the height as "deleted" lazily,
        // or remove it directly if using a specialized tree structure.
        maxHeap.remove(event.height);
    }

    let currentHighest = maxHeap.peek();
    // Do logic with currentHighest...
}
```

Binary Search

Binary Search Algorithm Explained

1. The Problem It Solves and Why We Need It

The Problem: Binary Search solves the problem of finding a specific value (the "target") efficiently within a **sorted** collection of data (like an array).

Why We Need It: If you have an unsorted list, your only choice is a "Linear Search"—checking every single item from start to finish until you find what you need.

- If you have an array of 4 billion users and you need to find one specific ID, a Linear Search could take up to 4 billion checks. That takes O(N) time.
- We need a way to skip data we *know* doesn't contain our target. Binary Search allows us to find that same ID in a maximum of just 32 checks, drastically reducing the time complexity to **O(log N)**.

---

2. What Is It?

Binary Search is a "divide and conquer" algorithm.

Think of how you look up a word like "Mango" in a physical dictionary. You don't read page 1, then page 2, then page 3. You open the book exactly in the middle. If you land on the letter "P", you know "Mango" comes *before* "P" alphabetically. You instantly ignore the entire right half of the dictionary and repeat the process on the left half.

Binary search does exactly this with an array. It looks at the middle element, checks if the target is smaller or larger, and discards the half of the array where the target cannot possibly exist.

---

3. How Does It Work?

You need three pointers to make this work:

1. **Left (L):** Represents the start of your current search space.
2. **Right (R):** Represents the end of your current search space.
3. **Mid (M):** The middle index between Left and Right.

At each step, you calculate `Mid = Left + floor((Right - Left) / 2)`.

- If `array[Mid]` is your target, you are done!
- If `array[Mid]` is smaller than your target, the target *must* be to the right. You move your `Left` pointer to `Mid + 1`.
- If `array[Mid]` is larger than your target, the target *must* be to the left. You move your `Right` pointer to `Mid - 1`.

Scenario A: Target Exists in the Array **Input Array:** `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` **Target:** `23`

```
================================================================================
STEP 1: Initial State
Target: 23
Index:    0   1   2   3   4   5   6   7   8   9
Array:  [ 2,  5,  8, 12, 16, 23, 38, 56, 72, 91 ]
          ^               ^                   ^
          L               M                   R

Calculations:
L = 0, R = 9
M = 0 + floor((9 - 0) / 2) = 4
array[4] is 16.
Comparison: 16 is less than 23. Target must be in the right half.
Action: Move L to M + 1 (which is 5).
================================================================================
STEP 2: Second Pass
Index:    0   1   2   3   4   5   6   7   8   9
Array:  [ 2,  5,  8, 12, 16, 23, 38, 56, 72, 91 ]
                             ^       ^       ^
                             L       M       R

Calculations:
L = 5, R = 9
M = 5 + floor((9 - 5) / 2) = 7
array[7] is 56.
Comparison: 56 is greater than 23. Target must be in the left half.
Action: Move R to M - 1 (which is 6).
================================================================================
STEP 3: Finding the Target
Index:    0   1   2   3   4   5   6   7   8   9
Array:  [ 2,  5,  8, 12, 16, 23, 38, 56, 72, 91 ]
                             ^
                            L,M
                             ^
                             R  (R is at 6)

Calculations:
L = 5, R = 6
M = 5 + floor((6 - 5) / 2) = 5
array[5] is 23.
Comparison: 23 equals 23! Target found.
Action: Return index 5.
================================================================================
```

Scenario B: Target Does Not Exist **Input Array:** `[2, 5, 8, 12]` **Target:** `6`

```
STEP 1: L=0, R=3, M=1. arr[1] is 5. 5 < 6. Move L to M+1 (2).
Array:  [ 2,  5,  8, 12 ]
                   ^   ^
                 L,M  R

STEP 2: L=2, R=3, M=2. arr[2] is 8. 8 > 6. Move R to M-1 (1).

STEP 3: L is now 2. R is now 1.
Because L is now strictly greater than R (L > R), the search space has collapsed completely.
The loop breaks. The target is not in the array. Return -1.
```

---

4. Code Sample (JavaScript)

```javascript
/**
 * Performs binary search on a sorted array to find a target value.
 * @param {number[]} nums - A sorted array of integers.
 * @param {number} target - The integer to search for.
 * @return {number} - The index of the target, or -1 if not found.
 */
function search(nums, target) {
    // Initialize pointers to the very edges of the array
    let left = 0;
    let right = nums.length - 1;

    // We keep searching as long as our search space is valid
    // (meaning the left pointer hasn't crossed the right pointer)
    while (left <= right) {
        // Calculate the middle index safely to prevent integer overflow bugs
        // (Though less of an issue in modern JS, it's a best practice)
        let mid = left + Math.floor((right - left) / 2);

        // Check if we hit the bullseye
        if (nums[mid] === target) {
            return mid;
        }

        // If the number at mid is too small, target is to the right.
        // Discard the left half by moving the 'left' pointer.
        if (nums[mid] < target) {
            left = mid + 1;
        }
        // If the number at mid is too big, target is to the left.
```

```
        // Discard the right half by moving the 'right' pointer.
        else {
            right = mid - 1;
        }
    }

    // If the loop finishes and we haven't returned, the target isn't there.
    return -1;
}

// Example Usage:
console.log(search([2, 5, 8, 12, 16, 23, 38], 23)); // Output: 5
```

---

5. LeetCode Questions It Applies To

1. **LeetCode 704: Binary Search**

- **Summary:** This is the vanilla, exact implementation shown in the code above. No tweaks needed.

2. **LeetCode 35: Search Insert Position**

- **Summary:** If the target isn't found, you need to return the index where it *would* be inserted. Tweak: Instead of returning `-1` at the end, simply return the `left` pointer, which naturally lands exactly where the missing target should go.

3. **LeetCode 74: Search a 2D Matrix**

- **Summary:** You are given a sorted grid instead of a flat list. Tweak: Treat the total number of cells as a 1D array from `0` to `(rows * cols) - 1`. Calculate `mid` normally, then translate `mid` back into 2D coordinates using division and modulo operations: `row = Math.floor(mid / cols)`, `col = mid % cols`.

4. **LeetCode 875: Koko Eating Bananas**

- **Summary:** This applies Binary Search on the *answer space*, not an array index. You know the minimum speed Koko can eat is `1`, and the max is the `maximum bananas in a single pile`. You binary search between `1` and `max` to find the optimal eating speed, using a helper function to verify if `mid` speed allows her to finish in time.

---

6. Time and Space Complexity

Time Complexity: O(log N) (Logarithmic time) Because we are cutting the search space in half every single step, the maximum number of steps is the

number of times we can divide N by 2 before reaching 1.

```
Derivation:
Start:            [ ------------- N elements ------------- ]  (Size: N)
Step 1:           [ ------ N/2 ------ ] XXXXXXXXXXXXXXXXXXX  (Size: N/2)
Step 2:           [ - N/4 - ] XXXXXXXXX XXXXXXXXXXXXXXXXXXX  (Size: N/4)
Step 3:           [ N/8 ] XXX XXXXXXXXX XXXXXXXXXXXXXXXXXXX  (Size: N/8)
...
Step k:           [ 1 ]                                      (Size: N / 2^k)

At the final step, the size is 1.
So, 1 = N / (2^k)
Multiply both sides by 2^k:  2^k = N
Solve for k (the number of steps): k = log2(N)

Therefore, Time Complexity = O(log N)
```

Space Complexity: O(1) (Constant space) We only use a few integer variables (`left`, `right`, `mid`), no matter how massive the input array is.

```
Derivation:
Memory Used:
- left pointer  (Integer) -> Constant Space
- right pointer (Integer) -> Constant Space
- mid pointer   (Integer) -> Constant Space

Since no arrays or recursive call stacks scale with N:
Total Space = O(1)
```

---

7. Variations and Common Tweaks

Variation 1: Finding the First or Last Occurrence (Lower/Upper Bound) If an array has duplicates (e.g., `[2, 4, 4, 4, 6]`) and you search for `4`, standard binary search will just return the first one it lands on (likely the middle `4`). But what if the question asks for the *first* occurrence?

Tweak Required: When you find the target (`nums[mid] === target`), don't return immediately. Instead, record `mid` as a potential answer in a variable, and continue searching in the left half to see if there's an even earlier occurrence.

```javascript
// Snippet for finding the First Occurrence
let result = -1;
while (left <= right) {
    let mid = left + Math.floor((right - left) / 2);

    if (nums[mid] === target) {
        result = mid;      // Record the answer
```

```
        right = mid - 1;   // Force the search to keep going LEFT
    } else if (nums[mid] < target) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
return result;
```

Variation 2: Binary Search on Answer Space Sometimes, you aren't searching through an array at all. You are searching for an *optimal value* within a known range of numbers (e.g., finding the minimum capacity of a ship to deliver packages within `D` days).

Tweak Required: Your `left` and `right` bounds represent the minimum and maximum possible *answers*. You use a custom helper function `isValid(mid)` to check if the current middle value satisfies the problem's conditions.

```
// Snippet for Binary Search on Answer Space
let left = minPossibleAnswer;
let right = maxPossibleAnswer;
let bestAnswer = right;

while (left <= right) {
    let mid = left + Math.floor((right - left) / 2);

    // isValid is a custom function you write based on the specific problem rules
    if (isValid(mid, otherData)) {
        bestAnswer = mid; // This capacity works, record it!
        right = mid - 1;  // Now try to find a SMALLER, more optimal capacity
    } else {
        left = mid + 1;   // Capacity too small, we must increase it
    }
}
return bestAnswer;
```

Single Pointer with Lookahead

Single Pointer with Lookahead Explained

1. The Problem It Solves and Why We Need It

The Problem: In many array or string problems, the value or meaning of the *current* item depends entirely on what comes *immediately after it*. For example, in the Roman numeral "IV", the "I" only means "subtract 1" because a "V" is right next to it.

Why We Need It: If you just look at elements one by one in isolation, you miss the context.

- A brute-force way to get context is to use multiple nested loops to scan ahead, which ruins performance (pushing time complexity to O(N * M)).
- Another way is to create brand new arrays to group items together, which wastes memory (O(N) space).

The "Lookahead" technique solves this by peeking at the next element (`i + 1`) during a single pass. It keeps your algorithm fast (O(N) time) and memory-efficient (O(1) space), while avoiding dreaded "Index Out Of Bounds" errors.

---

2. What Is It?

The "Single Pointer with Lookahead" is a simple but powerful iteration pattern. You use a standard loop with a single pointer (usually `i`) moving through the data. However, inside the loop, you "peek" at `array[i + 1]` (or sometimes even further ahead).

Based on what you see in the lookahead, you might:

1. Change how you process the current element.
2. Group the current element with the lookahead element.
3. Skip the pointer forward manually so you don't process the lookahead element twice.

---

3. How Does It Work?

Let's look at two scenarios to see how looking ahead changes our decisions.

Scenario A: The "Variable Jump" (String Compression) Input Array: `["a", "a", "b", "c", "c", "c"]` **Goal:** Compress this into `a2b1c3`.

Instead of just moving `i` by 1 every time, we look ahead to see how many matching characters there are, and then *jump* `i` to the end of that group.

```
================================================================================
STEP 1: Start at 'a'
Index:    0    1    2    3    4    5
Array:  [ a,   a,   b,   c,   c,   c ]
          ^    ^
          i    lookahead (i+1)

Logic: array[i] is 'a'. The lookahead array[i+1] is also 'a'.
We keep looking ahead (using a temporary count) until the character changes.
We find two 'a's.
```

```
Action: Add "a2" to our result. Jump 'i' forward by 2 to land on 'b'.
================================================================================
STEP 2: Processing 'b'
Index:    0    1    2    3    4    5
Array:  [ a,   a,   b,   c,   c,   c ]
                     ^    ^
                     i    lookahead

Logic: array[i] is 'b'. Lookahead is 'c'. They don't match.
Count is just 1.
Action: Add "b1" to result. Move 'i' forward by 1 to land on 'c'.
================================================================================
STEP 3: Processing 'c'
Index:    0    1    2    3    4    5
Array:  [ a,   a,   b,   c,   c,   c ]
                          ^    ^
                          i    lookahead

Logic: array[i] is 'c'. Lookaheads at index 4 and 5 match.
We find three 'c's.
Action: Add "c3" to result. The array ends.
================================================================================
Result: "a2b1c3"
```

Scenario B: The "Context Change" (Roman to Integer) Input String: `M C M` (1900) **Goal:** Convert to a number.

- M = 1000
- C = 100

```
================================================================================
STEP 1: First 'M'
String:   M    C    M
          ^    ^
          i    lookahead

Value of current (M) = 1000. Value of lookahead (C) = 100.
Rule: If current is GREATER THAN OR EQUAL to lookahead, we ADD current.
Action: Total = 1000. Move 'i' by 1.
================================================================================
STEP 2: The 'C'
String:   M    C    M
               ^    ^
               i    lookahead

Value of current (C) = 100. Value of lookahead (M) = 1000.
```

```
Rule: If current is LESS THAN lookahead, it means subtraction!
We must SUBTRACT current from the total.
Action: Total = 1000 - 100 = 900. Move 'i' by 1.
================================================================================
STEP 3: Final 'M'
String:   M    C    M
                    ^    ^
                    i    (out of bounds, no lookahead)

No lookahead exists. We just ADD the current value.
Action: Total = 900 + 1000 = 1900.
================================================================================
Result: 1900
```

---

4. Code Sample (JavaScript)

Here is a clean implementation of the String Compression scenario (Scenario A).

```javascript
/**
 * Compresses an array of characters using a single pointer with lookahead.
 * @param {character[]} chars - Array of characters
 * @return {string} - The compressed string
 */
function compressString(chars) {
    if (chars.length === 0) return "";

    let result = "";
    let i = 0; // Our single pointer

    // We use a while loop instead of a for loop because we want to
    // control exactly how far 'i' jumps based on our lookahead.
    while (i < chars.length) {
        let currentChar = chars[i];
        let count = 0;

        // THE LOOKAHEAD:
        // While we are within bounds, AND the next element matches our current...
        while (i < chars.length && chars[i] === currentChar) {
            count++;
            i++; // Move the pointer forward to consume the duplicate
        }

        // Append the character and its count to the result
        result += currentChar + count;
```

```
        // Notice we don't need an extra i++ here at the end of the outer loop,
        // because the inner lookahead loop already advanced 'i' to the next
        // completely new character!
    }

    return result;
}

// Example Usage:
console.log(compressString(["a","a","b","c","c","c"])); // Output: "a2b1c3"
```

---

5. LeetCode Questions It Applies To

1. **LeetCode 13: Roman to Integer**

- **Summary:** As walked through above, look at `s[i]`. If `s[i+1]` is a larger numeral, you subtract `s[i]` from your total instead of adding it.

2. **LeetCode 443: String Compression**

- **Summary:** Count consecutive repeating characters by looking ahead. The tweak here is you must modify the array *in-place* instead of creating a new string.

3. **LeetCode 605: Can Place Flowers**

- **Summary:** You are checking if you can plant a flower at `i`. You must verify that `i` is empty, AND look behind to `i-1` to ensure it's empty, AND look ahead to `i+1` to ensure it's empty.

---

6. Time and Space Complexity

Time Complexity: O(N) (Linear Time) Even though there is a `while` loop *inside* another `while` loop in the compression example, we only ever visit each element in the array exactly once. The inner loop advances the main pointer `i`.

```
Derivation:
Array Size = N elements

Start ---> [ Element 0 ]
             |-- Lookahead checks Element 1
             |-- Lookahead checks Element 2 (skips pointer to index 3)
           [ Element 3 ]
             |-- Lookahead checks Element 4 (skips pointer to index 5)
           [ Element 5 ]
```

```
Total array reads: N reads.
Because we never go backwards, Time = N * O(1) operations = O(N).
```

Space Complexity: O(1) (Constant Space) We are only using a few pointers and variables (like `i`, `count`, `currentChar`), which take up the same tiny amount of memory whether the array has 10 items or 10 million items. *(Note: If you return a new string, the output string takes O(N) space, but the algorithmic overhead is O(1)).*

```
Derivation:
Memory Used:
- Pointer 'i'      (Integer) -> Constant Space
- Counter 'count'  (Integer) -> Constant Space

Since we don't create intermediate arrays that scale with N:
Total Algorithmic Space = O(1)
```

---

7. Variations and Common Tweaks

Variation 1: Safe Lookahead (Preventing Out of Bounds) When you look at `array[i + 1]`, what happens when you are at the very last element of the array? `i + 1` doesn't exist!

Tweak Required: You must always add a boundary check *before* you evaluate the lookahead value.

```
// Snippet for Safe Lookahead
for (let i = 0; i < array.length; i++) {
    // TWEAK: Check if i + 1 is within the array length first!
    if (i + 1 < array.length && array[i] < array[i + 1]) {
        // Do something based on lookahead
    } else {
        // Handle the last element safely, or when the condition fails
    }
}
```

Variation 2: Lookbehind + Lookahead Sometimes you need to know the entire neighborhood (both left and right) to make a decision, like in the "Can Place Flowers" problem.

Tweak Required: Treat out-of-bounds edges (the start and end of the array) as valid "empty" spaces.

```
// Snippet for Lookbehind + Lookahead
for (let i = 0; i < flowerbed.length; i++) {
    if (flowerbed[i] === 0) {
```

```
        // TWEAK: If i === 0, there is no left neighbor, so treat it as 0 (empty)
        let emptyLeft = (i === 0) || (flowerbed[i - 1] === 0);

        // TWEAK: If i is the last element, there is no right neighbor, treat as 0
        let emptyRight = (i === flowerbed.length - 1) || (flowerbed[i + 1] === 0);

        if (emptyLeft && emptyRight) {
            flowerbed[i] = 1; // Plant the flower
            // Optional: i++ here to skip the next spot since we just planted!
        }
    }
}
```

Single Pointer with Lookahead and Hashmap

Single Pointer with Hashmap (The "Omniscient Lookahead") Explained

1. The Problem It Solves and Why We Need It

The Problem: The standard "Single Pointer with Lookahead" (checking `array[i + 1]`) is great when the data you need is directly next door. But what if the matching piece of data is 10,000 steps away, or behind you? If you are looking for pairs of numbers (like finding two numbers that add up to a target), checking every single combination takes a massive amount of time.

Why We Need It: * A brute-force search for pairs requires a loop inside a loop, checking element 1 against elements 2, 3, 4, etc. This takes **O(N^2)** time.

- By introducing a **Hashmap** (a Map, Dictionary, or Object), you give your single pointer a superpower: a perfect memory and an instant lookup mechanism.
- Instead of scanning the array repeatedly to look ahead for a matching value, you just ask the Hashmap: *"Does this value exist?"* This reduces the time complexity to a lightning-fast **O(N)**.

---

2. What Is It?

This algorithm combines a standard single-pointer loop with a Hashmap.

As your pointer `i` moves through the array, you do two things at every step:

1. **The Lookahead/Lookup:** You calculate what your current element "needs" to form a pair or sequence, and you instantly ask the Hashmap if it has seen that needed value.

2. **The Memory Bank:** If the Hashmap doesn't have the answer yet, you store your *current* element in the Hashmap so that future elements can find it when *they* do their lookahead.

It turns a spatial search (scanning an array) into a constant-time memory lookup.

---

3. How Does It Work?

Let's walk through the most famous scenario: finding two numbers that add up to a specific target.

Scenario A: The "Target Pair" (Two Sum) **Input Array:** `[3, 2, 4, 8]` **Target:** `6`

We will use a Hashmap to store the numbers we've seen and their indices. Format: `{ Number: Index }`.

```
================================================================================
STEP 1: Pointer at index 0
Array:    [ 3,   2,   4,   8 ]
            ^
            i

Calculation: Target (6) - Current (3) = 3 (This is the complement we need)
Hashmap Lookup: Does '3' exist in our Hashmap?
Current Hashmap: {}  -> No, it's empty.
Action: We can't form a pair yet. Save current number to Hashmap.
New Hashmap: { 3: 0 }
================================================================================
STEP 2: Pointer at index 1
Array:    [ 3,   2,   4,   8 ]
                 ^
                 i

Calculation: Target (6) - Current (2) = 4
Hashmap Lookup: Does '4' exist in our Hashmap?
Current Hashmap: { 3: 0 } -> No, we only have 3.
Action: Save current number to Hashmap.
New Hashmap: { 3: 0, 2: 1 }
================================================================================
STEP 3: Pointer at index 2
Array:    [ 3,   2,   4,   8 ]
                       ^
                       i
```

```
Calculation: Target (6) - Current (4) = 2
Hashmap Lookup: Does '2' exist in our Hashmap?
Current Hashmap: { 3: 0, 2: 1 } -> YES! The value '2' is at index 1.
Action: We found our pair! We pair the current index (2) with the
        stored index of the complement (1).
================================================================================
Result: [1, 2]
```

Scenario B: The "Consecutive Sequence Builder" **Input Array:** `[100, 4, 200, 1, 3, 2]` **Goal:** Find the length of the longest consecutive sequence of numbers (which would be 1, 2, 3, 4 -> length 4).

Here, we don't look for a math complement; we look for neighbors. We dump everything into a Hashmap (or Set) first, then iterate.

```
STEP 1: Build the Hashmap (Set) for instant lookups.
Map = { 100, 4, 200, 1, 3, 2 }

STEP 2: Iterate and use "Lookbehind" and "Lookahead".
Pointer at 100:
- Lookbehind: Does 99 exist in Map? No. (So 100 is the START of a sequence).
- Lookahead: Does 101 exist? No. Sequence length = 1.

Pointer at 4:
- Lookbehind: Does 3 exist? YES.
- Action: SKIP! 4 is not the start of a sequence. We will count it later
  when we process the actual start of its sequence.

Pointer at 1:
- Lookbehind: Does 0 exist? No. (1 is a START).
- Lookahead: Does 2 exist? Yes. Does 3 exist? Yes. Does 4 exist? Yes. Does 5 exist? No.
- Action: We counted 4 steps. Max Length is now 4.
```

---

4. Code Sample (JavaScript)

Here is the implementation of Scenario A (Two Sum).

```javascript
/**
 * Finds two numbers in an array that add up to a target using a Hashmap.
 * @param {number[]} nums - Array of integers.
 * @param {number} target - The sum we are looking for.
 * @return {number[]} - The indices of the two numbers.
 */
function twoSum(nums, target) {
    // Create our Hashmap. In JS, a Map object or a plain {} object works.
    // We will store key-value pairs as { numberSeen : indexOfNumber }
```

```javascript
    const seenNumbers = new Map();

    // Single pointer 'i' moves through the array once
    for (let i = 0; i < nums.length; i++) {
        let currentNum = nums[i];

        // Calculate the exact piece we need to finish the puzzle
        let complement = target - currentNum;

        // THE LOOKUP:
        // Does the puzzle piece we need already exist in our Hashmap?
        if (seenNumbers.has(complement)) {
            // If yes, we are done! Return the index of the complement,
            // and our current index 'i'.
            return [seenNumbers.get(complement), i];
        }

        // If no, we add the current number to the Hashmap so that
        // future numbers can look back and find it.
        seenNumbers.set(currentNum, i);
    }

    // If no pairs are found (based on problem constraints, this might not happen)
    return [];
}

// Example Usage:
console.log(twoSum([3, 2, 4, 8], 6)); // Output: [1, 2]
```

---

5. LeetCode Questions It Applies To

1. **LeetCode 1: Two Sum**

- **Summary:** The exact code provided above. You use a Hashmap to store `num: index` and check if `target - num` exists as you iterate.

2. **LeetCode 128: Longest Consecutive Sequence**

- **Summary:** Put all numbers in a HashSet. Iterate with a pointer. If `num - 1` is not in the set, it's the start of a sequence. Use a `while` loop to look ahead for `num + 1`, `num + 2`, counting how far you can go.

3. **LeetCode 560: Subarray Sum Equals K**

- **Summary:** You keep a running total (prefix sum). At each step, you ask the Hashmap: "Have I ever seen a running total equal to `current_total`

- K in the past?" If yes, a subarray matching K exists between that past point and now.

---

6. Time and Space Complexity

Time Complexity: O(N) (Linear Time) We pass through the array exactly one time. Looking up a value in a Hashmap takes O(1) time on average.

```
Derivation:
Array Size = N elements

Start ---> [ Loop i from 0 to N-1 ]
                  |
                  v
          Operations per loop:
          1. Subtraction (target - num)   --> O(1)
          2. Hashmap Lookup (map.has)     --> O(1) average
          3. Hashmap Insertion (map.set)  --> O(1) average

Total Time = N iterations * O(1) operations
Total Time = O(N)
```

Space Complexity: O(N) (Linear Space) This is the trade-off. To get that blazing fast O(N) time, we must spend memory. In the worst-case scenario (where the pair is at the very end of the array, or doesn't exist), we end up storing every single element of the array into the Hashmap.

```
Derivation:
Memory Used:
- Pointer 'i'              -> Constant Space
- Variable 'complement'    -> Constant Space
- Hashmap 'seenNumbers'    -> Grows up to size N

Since the Hashmap grows in direct proportion to the input array size:
Total Algorithmic Space = O(N)
```

---

7. Variations and Common Tweaks

Variation 1: The Frequency Map (Counting Occurrences) Sometimes you don't care *where* a number is, you care *how many times* it appears. This happens a lot in anagram problems or when trying to find duplicates.

Tweak Required: Instead of storing indices, your Hashmap values will be frequency counters. If the number exists, increment the counter by 1.

```javascript
// Snippet for Frequency Map
const freqMap = new Map();

for (let i = 0; i < nums.length; i++) {
    let num = nums[i];

    // If it exists, get the current count and add 1.
    // If it doesn't exist, default to 0 and add 1.
    freqMap.set(num, (freqMap.get(num) || 0) + 1);

    // Now you can look ahead/check: "Do I have enough of this number?"
    if (freqMap.get(num) > 3) {
        // Handle logic if a number appears more than 3 times
    }
}
```

Variation 2: Prefix Sum with Hashmap (The "Running Total" Lookup) In subarray problems, checking every subarray takes O(N^2). Instead, you keep a "running total" and store that total in a Hashmap. If your `current_total - target` exists in the map, it means the subarray *between* that past total and your current total perfectly equals your target.

Tweak Required: You store the running total as the key, and the frequency (how many times you've seen that exact total) as the value. You must initialize the map with `{ 0: 1 }` to account for subarrays that start at the very beginning.

```javascript
// Snippet for Prefix Sum with Hashmap
const prefixMap = new Map();
prefixMap.set(0, 1); // Base case: A sum of 0 has occurred 1 time (empty array)

let runningSum = 0;
let countOfSubarrays = 0;

for (let i = 0; i < nums.length; i++) {
    runningSum += nums[i];

    // Look back: Have we seen a sum that, if chopped off, leaves us with 'K'?
    let neededSum = runningSum - K;

    if (prefixMap.has(neededSum)) {
        countOfSubarrays += prefixMap.get(neededSum);
    }

    // Store the current running sum for future elements to find
    prefixMap.set(runningSum, (prefixMap.get(runningSum) || 0) + 1);
```

```
}
```

Prefix and suffix sum

Here is a complete guide to understanding and using the Prefix and Suffix Sum algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you have a long list of numbers (like daily sales figures for a year), and you need to frequently answer questions like: "What was the total sales between day 45 and day 112?"

If you just loop through the array from day 45 to day 112 and add the numbers up, it takes time. If you have to answer thousands of these "range sum" queries, looping every single time becomes incredibly slow.

Why we need it: Without this algorithm, calculating the sum of a range of size N takes O(N) time. If you have Q queries, your total time is O(Q * N).

The Prefix Sum algorithm allows us to answer *any* range sum query instantly in O(1) time. We do a one-time pre-computation step that takes O(N) time, making our total time for Q queries just O(N + Q). This is a massive performance boost.

---

2. What is it?

Prefix Sum is a pre-computation technique. It involves creating a new array of the same length as your input array. In this new array, the value at index `i` stores the cumulative sum of all the elements from the very beginning of the original array up to index `i`.

Suffix Sum is the exact same concept, but in reverse. The value at index `i` stores the cumulative sum of all elements from the very end of the array down to index `i`.

---

3. How does it work? (Walkthrough)

Let's break down the mechanics using a simple array of numbers.

Scenario A: 1D Prefix Sum **Input Array:** `[2, 4, 1, 5, 3]`

Step 1: Building the Prefix Array We create a `prefix` array. The formula is: `prefix[i] = prefix[i-1] + arr[i]`

- Index 0: The sum up to index 0 is just the first element. `prefix[0] = 2`
- Index 1: `prefix[0]` + `arr[1]` -> 2 + 4 = 6
- Index 2: `prefix[1]` + `arr[2]` -> 6 + 1 = 7
- Index 3: `prefix[2]` + `arr[3]` -> 7 + 5 = 12
- Index 4: `prefix[3]` + `arr[4]` -> 12 + 3 = 15

```
Original: |  2  |  4  |  1  |  5  |  3  |
Index:    |  0  |  1  |  2  |  3  |  4  |
          -------------------------------
Prefix:   |  2  |  6  |  7  | 12  | 15  |
```

Step 2: Querying a Range We want the sum from Index 1 to Index 3 (values: 4 + 1 + 5 = 10). Instead of adding them, we look at our `prefix` array. The sum from index 0 to 3 is `prefix[3]` (which is 12). But this includes index 0, which we don't want. So, we subtract the sum up to index 0, which is `prefix[0]` (which is 2). Formula: `Sum(L to R) = prefix[R] - prefix[L-1]` Calculation: 12 - 2 = 10. We got the answer instantly!

Scenario B: 1D Suffix Sum Input Array: `[2, 4, 1, 5, 3]`

Building the Suffix Array We build from right to left. The formula is: `suffix[i] = suffix[i+1] + arr[i]`

- Index 4: Just the last element. `suffix[4] = 3`
- Index 3: `suffix[4]` + `arr[3]` -> 3 + 5 = 8
- Index 2: `suffix[3]` + `arr[2]` -> 8 + 1 = 9
- Index 1: `suffix[2]` + `arr[1]` -> 9 + 4 = 13
- Index 0: `suffix[1]` + `arr[0]` -> 13 + 2 = 15

```
Original: |  2  |  4  |  1  |  5  |  3  |
Index:    |  0  |  1  |  2  |  3  |  4  |
          -------------------------------
Suffix:   | 15  | 13  |  9  |  8  |  3  |
```

Suffix sums are incredibly useful when you need to compare the left side of an array with the right side of an array simultaneously.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Class to handle Prefix and Suffix Sums
 */
class ArraySummations {
    constructor(arr) {
        this.arr = arr;
        this.n = arr.length;
```

```
        // Initialize arrays with zeros
        this.prefix = new Array(this.n).fill(0);
        this.suffix = new Array(this.n).fill(0);

        this.buildPrefix();
        this.buildSuffix();
    }

    buildPrefix() {
        if (this.n === 0) return;

        // Base case: first element
        this.prefix[0] = this.arr[0];

        // Build the rest from left to right
        for (let i = 1; i < this.n; i++) {
            this.prefix[i] = this.prefix[i - 1] + this.arr[i];
        }
    }

    buildSuffix() {
        if (this.n === 0) return;

        // Base case: last element
        this.suffix[this.n - 1] = this.arr[this.n - 1];

        // Build the rest from right to left
        for (let i = this.n - 2; i >= 0; i--) {
            this.suffix[i] = this.suffix[i + 1] + this.arr[i];
        }
    }

    /**
     * Get the sum of elements between index L and R (inclusive)
     */
    getRangeSum(L, R) {
        // If L is 0, the answer is just the prefix sum at R
        if (L === 0) {
            return this.prefix[R];
        }
        // Otherwise, subtract the unwanted left portion
        return this.prefix[R] - this.prefix[L - 1];
    }
}

// --- Usage Example ---
```

```
const myArray = [2, 4, 1, 5, 3];
const summations = new ArraySummations(myArray);

console.log(summations.prefix); // Output: [ 2, 6, 7, 12, 15 ]
console.log(summations.suffix); // Output: [ 15, 13, 9, 8, 3 ]
console.log(summations.getRangeSum(1, 3)); // Output: 10
```

---

5. LeetCode Questions it applies to

Here is how you apply this pattern to common interview questions:

- **Range Sum Query - Immutable (LeetCode 303):** This is the exact algorithm. You calculate the prefix array in the class constructor, and the `sumRange(left, right)` method just returns `prefix[right] - prefix[left-1]`.
- **Find Pivot Index (LeetCode 724):** You need to find an index where the sum of numbers to the left equals the sum to the right. You can solve this by keeping a running `leftSum`. The `rightSum` is simply `totalSum - leftSum - arr[i]`. If they match, you found the pivot.
- **Product of Array Except Self (LeetCode 238):** Instead of sums, you use Prefix and Suffix *Products*. The answer for index `i` is the product of everything to its left (prefixProduct[i-1]) multiplied by everything to its right (suffixProduct[i+1]).
- **Subarray Sum Equals K (LeetCode 560):** You calculate a running prefix sum. At each step, you check if `currentPrefixSum - K` exists in a Hash Map that tracks previous prefix sums. If it does, you've found a valid subarray.

---

6. Time and Space Complexity

Time Complexity: O(N) for pre-computation, O(1) per query. **Space Complexity:** O(N) to store the new arrays.

```
Time Complexity Derivation:
-------------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Create empty array     | N memory slots       | O(N)
2. Loop over input arr    | N iterations         | O(N)
3. Add arr[i] + prev sum  | 1 addition per step  | O(1)
-------------------------------------------------------------
Total Building Time: O(N) + O(N) * O(1) = O(N)

Querying:
```

```
-------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Array lookup at R      | 1 memory access      | O(1)
2. Array lookup at L-1    | 1 memory access      | O(1)
3. Subtraction            | 1 arithmetic op      | O(1)
-------------------------------------------------------
Total Query Time: O(1) + O(1) + O(1) = O(1)
```

---

7. Variations and Common Tweaks

The core idea of "accumulating state as you move through an array" can be tweaked for different operations.

Variation A: Prefix Product Instead of range sums, you need range products. **Tweak:** Change the addition to multiplication. *Note: If there are zeros in the array, range queries using division (`prefix[R] / prefix[L-1]`) will break due to division by zero. You usually have to track zeros separately.*

```
// Snippet for Prefix Product
prefixProd[0] = arr[0];
for (let i = 1; i < n; i++) {
    prefixProd[i] = prefixProd[i-1] * arr[i];
}
```

Variation B: Prefix XOR Useful for queries asking for the bitwise XOR sum of a range. XOR has a neat property where `X ^ X = 0`, so subtraction is replaced by another XOR! **Tweak:** Use the `^` operator. `RangeXOR(L, R) = prefixXOR[R] ^ prefixXOR[L-1]`.

```
// Snippet for Prefix XOR
prefixXor[0] = arr[0];
for (let i = 1; i < n; i++) {
    prefixXor[i] = prefixXor[i-1] ^ arr[i];
}
```

Variation C: 2D Prefix Sum (Image / Matrix) When dealing with a grid (like an image pixel matrix), you might need the sum of a rectangular sub-grid. A 1D array isn't enough; you need a 2D prefix matrix where `prefix[r][c]` is the sum of all elements in the rectangle from `(0,0)` to `(r,c)`.

Tweak: The math involves the Inclusion-Exclusion principle. You add the rectangle from above, the rectangle from the left, subtract the overlapping top-left corner, and add the current cell.

```
// Snippet for building 2D Prefix Sum
prefix[r][c] = arr[r][c]
             + prefix[r-1][c]      // Area above
             + prefix[r][c-1]      // Area to the left
             - prefix[r-1][c-1];   // Subtract overlapping top-left area
```

Prefix and suffix sum with a Hashmap

You need to combine a **Prefix Sum with a Hashmap** when you transition from asking *"What is the sum of this specific range?"* to asking *"How many ranges have a specific sum?"* or *"What is the longest range with a specific sum?"*

In a standard prefix sum, you already know the start and end points (L and R), and you just want the total. But in many interview questions, **you are given a Target Sum, and you need to find the boundaries (L and R) that create that sum.**

If you try to check every possible starting and ending point to find your target, it takes O(N^2) time. Using a Hashmap alongside your running prefix sum acts as a time machine, bringing the time complexity down to a beautiful **O(N)**.

---

1. The Core Logic: The "Time Machine" Equation

Let's look at the standard prefix sum formula we used earlier: `Sum(L to R) = Prefix[R] - Prefix[L-1]`

In these new problems, we already know the `Sum(L to R)` we want—let's call it our `Target`. We are currently looping through the array, so we are at index `R`. That means we also know our current running sum, `Prefix[R]`.

What we *don't* know is if there was a previous starting point (`Prefix[L-1]`) that makes the math work perfectly. Let's rearrange the algebra:

`Target = CurrentSum - PreviousSum` which becomes: `PreviousSum = CurrentSum - Target`

This is the golden rule. At every step in our array, we calculate our `CurrentSum`. Then we subtract our `Target`. The result is the exact `PreviousSum` we need to have seen earlier in the array for a valid subarray to exist ending right here.

We use a Hashmap to remember every `PreviousSum` we've ever seen.

---

2. How it Works: The Walkthrough

The Problem: Find how many subarrays add up to a Target of `7`. **Input Array:** `[3, 4, 1, 6, -3]` **Target:** `7`

The Setup: We need a `currentSum` variable starting at `0`. We need a Hashmap. **Crucial Step:** We must initialize the Hashmap with `{ 0: 1 }`. This means "we have seen a prefix sum of 0 exactly 1 time". This handles the edge case where a valid subarray starts at the very first element of the array (Index 0).

Let's walk through the array. The hashmap stores: `[Prefix Sum Value : How many times we've seen it]`.

```
Initial State:
Target = 7
currentSum = 0
Hashmap = { 0: 1 }
Subarrays Found = 0
```

Index 0: Value is 3

- `currentSum` becomes 0 + 3 = 3.
- We need: `currentSum - Target` -> 3 - 7 = -4.
- Check Hashmap: Have we seen -4? No.
- Update Hashmap: Add our `currentSum` of 3 to the map.
- `Hashmap state: { 0:1, 3:1 }`

Index 1: Value is 4

- `currentSum` becomes 3 + 4 = 7.
- We need: `currentSum - Target` -> 7 - 7 = 0.
- Check Hashmap: Have we seen 0? **YES!** We've seen it 1 time.
- Action: We found 1 valid subarray! (The subarray is `[3, 4]`).
- Update Hashmap: Add our `currentSum` of 7 to the map.
- `Hashmap state: { 0:1, 3:1, 7:1 }`

Index 2: Value is 1

- `currentSum` becomes 7 + 1 = 8.
- We need: `currentSum - Target` -> 8 - 7 = 1.
- Check Hashmap: Have we seen 1? No.
- Update Hashmap: Add our `currentSum` of 8 to the map.
- `Hashmap state: { 0:1, 3:1, 7:1, 8:1 }`

Index 3: Value is 6

- `currentSum` becomes 8 + 6 = 14.
- We need: `currentSum - Target` -> 14 - 7 = 7.
- Check Hashmap: Have we seen 7? **YES!** We saw it at Index 1.
- Action: We found another valid subarray! (The subarray is `[1, 6]`).
- Update Hashmap: Add our `currentSum` of 14 to the map.

- `Hashmap state: { 0:1, 3:1, 7:1, 8:1, 14:1 }`

By the end of the loop, we successfully found both subarrays without ever having to loop backward!

---

3. The Three Major Variations

Depending on what the question asks, you change *what* you store as the value in your Hashmap.

Variation A: "How many subarrays?" (Count)

- **What you store in Hashmap:** `[PrefixSum : Frequency]` (How many times this sum has occurred).
- **Why:** If an array has negative numbers, a prefix sum can go up and down and hit the same value multiple times. If you are looking for a `PreviousSum` of 5, and the map says you've seen a sum of 5 three times earlier, that means there are *three different starting points* that work. You add 3 to your total count.
- **LeetCode Example:** 560. Subarray Sum Equals K

Variation B: "What is the longest subarray?" (Length)

- **What you store in Hashmap:** `[PrefixSum : Earliest Index]` (The very first index where this sum occurred).
- **Why:** To get the longest possible length, you want the starting boundary (`L-1`) to be as far back in the past as possible. If you calculate `currentSum - Target` and find it in the map, you calculate the length as: `Current Index - Map.get(PreviousSum)`. You only add a new prefix sum to the map if it's not already there.
- **LeetCode Example:** 325. Maximum Size Subarray Sum Equals k

Variation C: "Equal numbers of X and Y" (The Conversion Trick)

- **The Tweak:** Questions will ask "Find the longest subarray with an equal number of 0s and 1s". There is no explicit "Target Sum" here. The trick is to convert all `0`s to `-1`s in your mind as you loop.
- **Why:** If 0s are -1s and 1s are 1s, then a subarray with an equal number of both will perfectly cancel out to a **Sum of 0**. Now it's just Variation B with a Target of 0!
- **LeetCode Example:** 525. Contiguous Array

---

4. Time and Space Complexity

```
Algorithm: Prefix Sum + Hashmap
-------------------------------------------------------
Time Complexity: O(N)
We iterate through the array exactly one time.
Checking a value in a Hashmap and inserting into a Hashmap
both take O(1) constant time on average.
N iterations * O(1) work = O(N) Total Time.

Space Complexity: O(N)
In the worst-case scenario, every single prefix sum is a
unique number. Therefore, our Hashmap will need to store
N different key-value pairs, taking up O(N) memory space.
-------------------------------------------------------
```

Prefix sum with Monotonic Queue

Here is a complete guide to understanding and using the highly advanced **Prefix Sum with Monotonic Queue** algorithm.

---

1. Problem it solves and Why we need it

The Problem: You are asked to find the **shortest subarray** whose sum is greater than or equal to a target number **K**. Crucially, the array contains **negative numbers**.

Why the "Naive" Approaches fail:

1. **Brute Force:** You could check every single subarray combination. This takes O(N^2) time. For an array of 100,000 numbers, this is far too slow.
2. **Standard Sliding Window (Two Pointers):** Normally, for a "shortest subarray sum" problem, you use a sliding window. You expand the right side to increase your sum, and shrink the left side to make it shorter.

- *The Fatal Flaw:* This ONLY works if the array has only positive numbers. If you have negative numbers, expanding your window might actually *decrease* your sum, and shrinking it might *increase* your sum. The standard sliding window completely breaks down because it loses its "direction".

Why we need this algorithm: By combining a **Prefix Sum** array (which gives us instant subarray sums) with a **Monotonic Queue** (which intelligently filters out useless starting points), we can solve this problem in a blazing fast **O(N)** time, even with negative numbers!

---

2. What is it?

This algorithm is a marriage of two concepts we've explored:

1. **Prefix Sum:** An array where each index stores the running total of all numbers before it.

- *Rule:* The sum of a subarray from index `i` to index `j` is simply `Prefix[j] - Prefix[i]`.

2. **Monotonic Increasing Queue:** A double-ended queue (deque) that stores the **indices** of the Prefix Sum array. It strictly maintains the prefix sums in increasing order (smallest at the front, largest at the back).

The queue acts as a "smart memory". As we iterate through the array, the queue remembers the best possible starting points (`i`) for our subarrays and ruthlessly throws away any starting points that are mathematically proven to be useless.

---

3. How does it work? (Walkthrough)

This is a notoriously tricky algorithm. Let's break it down using the two golden rules of the queue.

The Setup: We want the shortest subarray with a sum >= `3`. **Input Array:** `[2, -1, 2]` **Prefix Array:** `[0, 2, 1, 3]` *(Note: We always add a `0` at the very beginning of a prefix array to represent the sum before any elements are added. This makes the math work perfectly).*

We will loop through our Prefix Array. At every step `j`, we check the queue for valid starting points `i`.

Rule 1: The "Shortest Subarray" Rule (Clean the Front) If the current Prefix sum minus the Prefix sum at the front of the queue is >= K, we found a valid subarray! We record its length. *Then, we kick that starting point out of the front of the queue.* **Why?** Because we just found a valid subarray using that starting point. If we keep it in the queue and use it with a future, later ending point, the resulting subarray would just be *longer*. We want the shortest! It is useless to us now.

Rule 2: The "Superior Starting Point" Rule (Clean the Back) Before adding our current index to the back of the queue, we look at the guys already at the back. If our current Prefix sum is **less than or equal to** the Prefix sum at the back, we kick the guy at the back out! **Why?** Because our current index is *later* in the array (which means it will create a shorter subarray) AND its prefix sum is *smaller* (which means when we subtract it, we get a larger, better sum). The older, larger prefix sum is entirely obsolete.

The Walkthrough Target `K = 3`. Prefix Array `P = [0, 2, 1, 3]`.

```
Step 0: j = 0, P[0] = 0
Queue is empty.
Action: Add index 0 to back.
Queue (stores indices): [0]
Queue (represents values): [0]

Step 1: j = 1, P[1] = 2
Rule 1 (Front): P[1] - P[Queue[0]] -> 2 - 0 = 2. (2 is not >= 3). Do nothing.
Rule 2 (Back): Is P[1] <= P[Queue back]? -> Is 2 <= 0? No. Do nothing.
Action: Add index 1 to back.
Queue: [0, 1]
Values: [0, 2]

Step 2: j = 2, P[2] = 1
Rule 1 (Front): P[2] - P[Queue[0]] -> 1 - 0 = 1. (1 is not >= 3). Do nothing.
Rule 2 (Back): Is P[2] <= P[Queue back]? -> Is 1 <= 2? YES!
* Action: Kick index 1 out! Index 2 is a superior starting point.
Action: Add index 2 to back.
Queue: [0, 2]
Values: [0, 1]

Step 3: j = 3, P[3] = 3
Rule 1 (Front): P[3] - P[Queue[0]] -> 3 - 0 = 3. (3 IS >= 3!)
* Action: Valid subarray found! Length is (j - Queue[0]) -> (3 - 0) = 3.
* Action: Kick index 0 out of the front! (Rule 1)
Rule 1 (Front) loop repeats: Queue[0] is now index 2.
P[3] - P[Queue[0]] -> 3 - 1 = 2. (2 is not >= 3). Stop front cleanup.
Rule 2 (Back): Is P[3] <= P[Queue back]? -> Is 3 <= 1? No.
Action: Add index 3 to back.
Queue: [2, 3]
Values: [1, 3]
```

Result: The shortest valid subarray we recorded had a length of 3 (the whole array `[2, -1, 2]`).

---

4. Code Sample (JavaScript)

```javascript
/**
 * Finds the length of the shortest subarray with sum >= K
 * @param {number[]} nums
 * @param {number} k
 * @return {number} - Returns -1 if no such subarray exists
 */
function shortestSubarray(nums, k) {
    const n = nums.length;
```

```
// 1. Build the Prefix Sum array (size N + 1)
// We use BigInt or Number depending on constraints. Using standard Number here.
const prefix = new Array(n + 1).fill(0);
for (let i = 0; i < n; i++) {
    prefix[i + 1] = prefix[i] + nums[i];
}

// This will store our answer. Initialize with infinity.
let shortestLength = Infinity;

// 2. Our Monotonic Queue (Stores indices of the prefix array)
const deque = [];

// 3. Iterate through the prefix array
for (let j = 0; j <= n; j++) {

    // RULE 1: Clean the Front (We found a valid subarray!)
    // If current prefix minus prefix at the front of queue >= k
    while (deque.length > 0 && prefix[j] - prefix[deque[0]] >= k) {
        // Calculate length
        const length = j - deque[0];
        if (length < shortestLength) {
            shortestLength = length;
        }
        // Remove the front index because we want the shortest length.
        // Any future 'j' paired with this front index would just be longer.
        deque.shift();
    }

    // RULE 2: Clean the Back (Maintain Monotonically Increasing Order)
    // If current prefix is smaller than or equal to the prefix at the back,
    // the back index is useless. The current 'j' is a better starting point.
    while (deque.length > 0 && prefix[j] <= prefix[deque[deque.length - 1]]) {
        deque.pop();
    }

    // Add current index to the queue
    deque.push(j);
}

// If shortestLength is still Infinity, we never found a valid subarray
if (shortestLength === Infinity) {
    return -1;
}
return shortestLength;
```

```
}

// --- Usage ---
// Target K = 3
console.log(shortestSubarray([2, -1, 2], 3)); // Output: 3
```

(Note on JS Arrays: In a strict interview setting, remind the interviewer that `shift()` *on a JS array is O(N), so in a highly optimized environment, you would implement a true double-ended queue using a Linked List or a Left/Right pointer system to guarantee pure O(1) shifts. The logic remains identical).*

---

5. LeetCode questions it applies to

- **Shortest Subarray with Sum at Least K (LeetCode 862):** This is the flagship question for this algorithm. The code provided above is the exact, optimal solution for this "Hard" level problem.
- **Max Value of Equation (LeetCode 1499):** While not exactly a prefix sum, it uses the exact same Monotonic Queue logic. You are given an equation `yi + yj + |xi - xj|`. By rewriting it as `(yi - xi) + (yj + xj)`, you treat `(yi - xi)` like the "prefix" that you store in the monotonic queue to find the maximum possible pairing.
- **Constrained Subsequence Sum (LeetCode 1425):** You use dynamic programming combined with a Monotonic Queue. The queue keeps track of the maximum DP states within a jumping window `k` to optimize the transition time.

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Build Prefix Array     | Loop N times         | O(N)
2. Iterate Prefix Array   | Loop N+1 times       | O(N)
3. Push to Deque          | Every index exactly 1| O(1) amortized
4. Pop/Shift from Deque   | Every index exactly 1| O(1) amortized
-----------------------------------------------------------
Total Time Complexity: O(N)
Even though there are 'while' loops inside the main 'for' loop,
an index can only be pushed into the queue once, and removed
once. Therefore, the internal loops do at most N total operations
across the entire run of the algorithm.

Space Complexity Derivation:
```

```
-------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Prefix Array           | Array of size N + 1  | O(N)
2. Monotonic Deque        | Max size N + 1       | O(N)
-------------------------------------------------------
Total Space Complexity: O(N)
```

---

7. Variations and Common Tweaks

Variation A: Finding the Maximum Subarray Sum with a Length Limit **The Problem:** You want to find the *largest* possible subarray sum, but the subarray cannot be longer than `M` elements. **The Tweak:** The logic shifts.

1. Instead of looking for a specific target `K`, we just want to maximize `Prefix[j] - Prefix[i]`. To make that as big as possible, we need `Prefix[i]` to be as *small* as possible. So, we still keep a Monotonic Increasing queue.
2. **Rule 1 (Front Clean) changes:** We don't pop when we find a target. We ONLY pop from the front if the index has expired (i.e., `j - deque[0] > M`). This ensures our window doesn't exceed the length limit.
3. **The Answer Step:** At every step, the front of the queue is guaranteed to be the *minimum* prefix sum available within our window limit. So, `Prefix[j] - Prefix[deque[0]]` will give us the maximum possible subarray sum ending at `j`!

Snippet for Max Sum with Length Limit M:

```javascript
let maxSum = -Infinity;

for (let j = 0; j <= n; j++) {

    // Tweak 1: Expire old starting points if they violate the length limit M
    while (deque.length > 0 && j - deque[0] > M) {
        deque.shift();
    }

    // Tweak 2: Record the best sum using the front of the queue
    // (We do this BEFORE adding 'j' to the queue so we don't subtract j from itself)
    if (deque.length > 0) {
        let currentSum = prefix[j] - prefix[deque[0]];
        if (currentSum > maxSum) {
            maxSum = currentSum;
        }
    }
```

```
    // Rule 2 remains the same: Keep queue Monotonically Increasing
    while (deque.length > 0 && prefix[j] <= prefix[deque[deque.length - 1]]) {
        deque.pop();
    }

    deque.push(j);
}
```

Prefix and suffix product

Here is a complete guide to understanding and applying the Prefix and Suffix Product algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you have an array of numbers, and for every position in the array, you need to find the product of *all the other numbers* except the one at your current position.

If you just use nested loops to multiply everything else for every single index, it takes O(N^2) time. If the array is huge, this is far too slow.

The "Obvious" (But Flawed) Solution: You might think: *"Why not just multiply all the numbers together once to get a Total Product, and then divide that Total Product by the number at the current index?"* **Why we need Prefix/Suffix Product instead:** The division method is fast (O(N)), but it completely breaks down if your array contains a **zero**.

- If there is one zero in the array, the Total Product becomes 0. When you try to find the answer for the zero's index, you do `0 / 0`, which is a mathematical error (or `NaN` in code).
- If there are multiple zeros, the logic breaks even further.

The Prefix and Suffix Product algorithm solves this cleanly in O(N) time **without ever using division**, making it perfectly safe no matter what numbers (including zeros) are in the array.

---

2. What is it?

Just like Prefix Sum, this algorithm relies on pre-computation. However, to solve the classic "Product Except Self" problem, we slightly tweak the definition to make the math perfectly align:

- **Prefix Product (Strictly Left):** An array where the value at index `i` is the product of all elements strictly to the *left* of `i`.
- **Suffix Product (Strictly Right):** An array where the value at index `i` is the product of all elements strictly to the *right* of `i`.

If we want the product of everything *except* the element at index `i`, we simply multiply its Left Prefix by its Right Suffix!

---

3. How does it work? (Walkthrough)

Let's break this down with a standard scenario and a tricky scenario.

Scenario A: Standard Array (No Zeros) **Input Array:** `[1, 2, 3, 4]`

Step 1: Building the Prefix (Left) Array We start from the left. Since there is nothing to the left of the very first element, its left product is inherently `1` (the multiplicative identity).

- Index 0: Nothing to the left -> `1`
- Index 1: Only arr[0] to the left -> `1`
- Index 2: arr[0] * arr[1] (which is prefix[1] * arr[1]) -> 1 * 2 = `2`
- Index 3: arr[0] * arr[1] * arr[2] (which is prefix[2] * arr[2]) -> 2 * 3 = `6`

```
Input:  |  1  |  2  |  3  |  4  |
Index:  |  0  |  1  |  2  |  3  |
        -------------------------
Prefix: |  1  |  1  |  2  |  6  |
```

Step 2: Building the Suffix (Right) Array We build from right to left. Since there is nothing to the right of the last element, its right product is inherently `1`.

- Index 3: Nothing to the right -> `1`
- Index 2: Only arr[3] to the right -> `4`
- Index 1: suffix[2] * arr[2] -> 4 * 3 = `12`
- Index 0: suffix[1] * arr[1] -> 12 * 2 = `24`

```
Input:  |  1  |  2  |  3  |  4  |
Index:  |  0  |  1  |  2  |  3  |
        -------------------------
Suffix: | 24  | 12  |  4  |  1  |
```

Step 3: Calculating the Final Result For any index `i`, the answer is `Prefix[i] * Suffix[i]`.

- Index 0: 1 * 24 = 24
- Index 1: 1 * 12 = 12
- Index 2: 2 * 4 = 8
- Index 3: 6 * 1 = 6 **Final Answer:** `[24, 12, 8, 6]`

Scenario B: The "Zero" Problem Let's see why this perfectly handles zeros without crashing. **Input Array:** `[1, 0, 3, 4]`

```
Input:  |  1  |  0  |  3  |  4  |
        -------------------------
Prefix: |  1  |  1  |  0  |  0  |  <-- Notice how everything after 0 becomes 0
Suffix: |  0  | 12  |  4  |  1  |  <-- Notice how everything before 0 becomes 0
        -------------------------
Result: |  0  | 12  |  0  |  0  |  (Prefix * Suffix)
```

Look at Index 1 (where the zero was). The Prefix is `1`, the Suffix is `12`. We multiply them to get `12`. We successfully found the product of the rest of the array (1 * 3 * 4 = 12) without ever trying to divide by zero!

---

4. Code Sample (JavaScript)

```javascript
/**
 * Calculates the product of an array except for self using Prefix/Suffix
 * @param {number[]} nums
 * @return {number[]}
 */
function productExceptSelf(nums) {
    const n = nums.length;

    // Initialize our arrays with 1s (multiplicative identity)
    const prefix = new Array(n).fill(1);
    const suffix = new Array(n).fill(1);
    const result = new Array(n);

    // 1. Build the Prefix (Left) Array
    // Start at index 1 because index 0 has nothing to its left
    for (let i = 1; i < n; i++) {
        // The left product for 'i' is the left product of 'i-1'
        // multiplied by the actual number at 'i-1'
        prefix[i] = prefix[i - 1] * nums[i - 1];
    }

    // 2. Build the Suffix (Right) Array
    // Start at second-to-last item because the last item has nothing to its right
    for (let i = n - 2; i >= 0; i--) {
        // The right product for 'i' is the right product of 'i+1'
        // multiplied by the actual number at 'i+1'
        suffix[i] = suffix[i + 1] * nums[i + 1];
    }
```

```
    // 3. Multiply them together for the final result
    for (let i = 0; i < n; i++) {
        result[i] = prefix[i] * suffix[i];
    }

    return result;
}

// --- Usage ---
console.log(productExceptSelf([1, 2, 3, 4])); // Output: [24, 12, 8, 6]
```

---

5. LeetCode Questions it applies to

- **Product of Array Except Self (LeetCode 238):** This is the exact algorithm described above. You calculate the strictly-left prefix product, the strictly-right suffix product, and multiply them.
- **Maximum Product Subarray (LeetCode 152):** A brilliant variation. Because multiplying two negative numbers creates a positive number, a subarray's product can suddenly become huge if you hit a second negative number. You can solve this by keeping a running Prefix Product (left to right) and a Suffix Product (right to left). If you hit a 0, you reset your running product to 1. The maximum value you ever see in either direction is your answer.
- **Construct the Array (various platforms):** Any problem asking you to evaluate the "state of everything else" relative to a single point without using inverse operations (like division or subtraction) will use this pattern.

---

6. Time and Space Complexity

Time Complexity: O(N) **Space Complexity:** O(N) (to store the prefix, suffix, and result arrays)

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Build Prefix Array     | N iterations         | O(N)
2. Build Suffix Array     | N iterations         | O(N)
3. Multiply Both Arrays   | N iterations         | O(N)
-----------------------------------------------------------
Total Time: O(N) + O(N) + O(N) = O(3N). We drop the constant -> O(N)

Space Complexity Derivation:
-----------------------------------------------------------
```

```
Task                      | Array Size           | Big O
--------------------------|----------------------|--------
1. Prefix Array           | N memory slots       | O(N)
2. Suffix Array           | N memory slots       | O(N)
-------------------------------------------------------------
Total Space: O(N) + O(N) = O(2N). We drop the constant -> O(N)
```

---

7. Variations and Common Tweaks

Variation A: The O(1) Space Optimization In coding interviews, once you write the O(N) space solution, the interviewer will often ask: *"Can you do this without using extra space for the prefix and suffix arrays?"* (The final `result` array doesn't count against your space complexity).

The Tweak: Instead of making a `prefix` array, write the left products directly into your `result` array. Then, instead of making a `suffix` array, you just use a single variable (e.g., `rightRunningProduct`) to keep track of the right side as you loop backward, multiplying it directly into the `result` array on the fly!

Snippet for Optimization:

```javascript
function optimizedProductExceptSelf(nums) {
    const n = nums.length;
    const result = new Array(n).fill(1);

    // Build prefix directly into result
    for (let i = 1; i < n; i++) {
        result[i] = result[i - 1] * nums[i - 1];
    }

    // Suffix computation on the fly!
    let rightRunningProduct = 1;
    for (let i = n - 1; i >= 0; i--) {
        // Multiply the already-stored prefix by the running suffix
        result[i] = result[i] * rightRunningProduct;

        // Update the running suffix for the next iteration leftward
        rightRunningProduct = rightRunningProduct * nums[i];
    }

    return result;
}
```

Variation B: Maximum Product Subarray (Resetting at Zero) As mentioned in the LeetCode section, sometimes you want the maximum contiguous product. The tweak here is that you compute the standard inclusive

prefix/suffix product, but you **reset your tracker to 1 whenever you encounter a 0**, because a 0 destroys a contiguous product chain.

Snippet for Resetting:

```
let maxProduct = -Infinity;
let currentPrefix = 1;

for (let i = 0; i < n; i++) {
    // Reset to 1 if we hit a 0 in the previous step
    if (currentPrefix === 0) currentPrefix = 1;

    currentPrefix = currentPrefix * nums[i];
    maxProduct = Math.max(maxProduct, currentPrefix);
}
// (You would then repeat this going backward for the Suffix)
```

Merge and Mergesort

Here is a complete guide to understanding and implementing the Merge Sort algorithm and its core component, the Merge function.

---

1. Problem it solves and Why we need it

The Problem: You are given an unsorted array of numbers, and you need to arrange them in ascending (or descending) order.

Why we need it: Simple sorting algorithms like Bubble Sort, Insertion Sort, or Selection Sort compare almost every element with every other element. This takes O(N^2) time. If you have 100,000 items, that is 10,000,000,000 operations. It's far too slow for real-world applications.

Merge Sort solves this by using a "Divide and Conquer" strategy, bringing the time down to **O(N * log2(N))**. For 100,000 items, this is roughly 1,700,000 operations—a massive, game-changing speedup. It is also a **stable sort**, meaning duplicate numbers maintain their original relative order.

---

2. What is it?

Merge Sort is actually two different algorithms working together:

1. **MergeSort (The Divider):** A recursive function that keeps slicing the array exactly in half until you are left with tiny arrays containing only 1 element. *(An array of 1 element is, by definition, already sorted!)*

2. **Merge (The Conqueror):** A helper function that takes two *already sorted* arrays and zips them together into a single, larger sorted array.

The magic happens because it is mathematically much faster to zip two already-sorted arrays together than to sort one giant messy array from scratch.

---

3. How does it work? (Walkthrough)

Let's break this down into the two phases using the input array: `[38, 27, 43, 3]`

Phase A: The Divide (MergeSort) We simply cut the array in half repeatedly.

```
Level 0:          [38, 27, 43,  3]
                 /                \
Level 1:     [38, 27]          [43,  3]
            /        \        /        \
Level 2:  [38]      [27]    [43]      [3]

(Stop! Every array is now size 1. They are "sorted".)
```

Phase B: The Conquer (Merge) Now we work our way back up. How do we merge two sorted arrays like `[27, 38]` and `[3, 43]`?

The Merge Walkthrough: We use a **Two-Pointer** technique.

- Pointer `i` looks at the left array.
- Pointer `j` looks at the right array. We compare the numbers at `i` and `j`. The smaller number wins, gets placed in our `result` array, and its pointer moves forward.

Let's merge Left: `[27, 38]` and Right: `[3, 43]`

```
Start:
Left:  [27, 38]     Right: [ 3, 43]     Result: []
        ^                   ^
        i=0                 j=0
```

Step 1: Compare Left[0] (27) and Right[0] (3). `3` is smaller! Add `3` to result. Move pointer `j` forward.

```
Left:  [27, 38]     Right: [ 3, 43]     Result: [3]
        ^                         ^
        i=0                       j=1
```

Step 2: Compare Left[0] (27) and Right[1] (43). `27` is smaller! Add `27` to result. Move pointer `i` forward.

```
Left:  [27, 38]     Right: [ 3, 43]     Result: [3, 27]
            ^                   ^
            i=1                 j=1
```

Step 3: Compare Left[1] (38) and Right[1] (43). `38` is smaller! Add `38` to result. Move pointer `i` forward.

```
Left:  [27, 38]     Right: [ 3, 43]     Result: [3, 27, 38]
                ^                   ^
          (i is out!)               j=1
```

Step 4: Left array is empty! We just take whatever is left in the Right array and dump it into the result. Add `43` to result. **Final Result:** `[3, 27, 38, 43]`

---

4. Code Sample (JavaScript)

```javascript
/**
 * Main MergeSort function (The Divider)
 * @param {number[]} arr
 * @return {number[]} sorted array
 */
function mergeSort(arr) {
    // Base case: arrays with 0 or 1 element are already sorted
    if (arr.length <= 1) {
        return arr;
    }

    // Find the middle index
    const mid = Math.floor(arr.length / 2);

    // Split the array into left and right halves
    const leftHalf = arr.slice(0, mid);
    const rightHalf = arr.slice(mid);

    // Recursively sort both halves
    const sortedLeft = mergeSort(leftHalf);
    const sortedRight = mergeSort(rightHalf);

    // Merge the sorted halves back together
    return merge(sortedLeft, sortedRight);
}

/**
 * Helper function to merge two sorted arrays (The Conqueror)
 * @param {number[]} left
```

```
 * @param {number[]} right
 * @return {number[]}
 */
function merge(left, right) {
    let result = [];
    let i = 0; // Pointer for left array
    let j = 0; // Pointer for right array

    // Compare elements while both arrays have items left
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) {
            result.push(left[i]);
            i++;
        } else {
            result.push(right[j]);
            j++;
        }
    }

    // If there are leftovers in the left array, add them
    while (i < left.length) {
        result.push(left[i]);
        i++;
    }

    // If there are leftovers in the right array, add them
    while (j < right.length) {
        result.push(right[j]);
        j++;
    }

    return result;
}

// --- Usage ---
const unsortedArray = [38, 27, 43, 3, 9, 82, 10];
console.log(mergeSort(unsortedArray));
// Output: [ 3, 9, 10, 27, 38, 43, 82 ]
```

---

5. LeetCode questions it applies to

- **Sort an Array (LeetCode 912):** This is the direct application of the algorithm. You are asked to sort an array in O(N log N) time without using built-in functions. The code above is the exact answer.

- **Merge Sorted Array (LeetCode 88):** You are given two already sorted arrays, `nums1` and `nums2`, and must merge them. The twist? You must do it *in-place* inside `nums1`. You solve this by running the two-pointer `merge` logic backwards starting from the end of the arrays!
- **Merge k Sorted Lists (LeetCode 23):** Instead of two arrays, you have `K` linked lists. You can use a Divide and Conquer approach, pairing up lists and merging them two at a time using the standard merge logic until only one list remains.
- **Reverse Pairs / Count of Smaller Numbers After Self (LeetCode 493 / 315):** These are hard problems solved beautifully by slightly tweaking the `merge` function (see "Counting Inversions" in the Variations section below).

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Divide Phase           | Split array in half  | O(log2(N)) levels
2. Merge Phase            | Compare N elements   | O(N) work per level
-----------------------------------------------------------
Total Time: log2(N) levels * N work per level = O(N * log2(N))
*Note: This time is guaranteed for Best, Worst, and Average cases!

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Array Size           | Big O
--------------------------|----------------------|--------
1. Slicing Arrays         | Temporary arrays     | O(N) memory total
2. Recursion Call Stack   | log2(N) deep         | O(log2(N))
-----------------------------------------------------------
Total Space: O(N) + O(log2(N)) -> Dominant term is O(N) Space.
```

---

7. Variations and Common Tweaks

Variation A: Counting Inversions (The "Jumping" Element Trick)
The Problem: An "inversion" is when a larger number appears *before* a smaller number in an array. Interviewers will ask: "How many pairs `(i, j)` exist where `i < j` but `arr[i] > arr[j]`?" **The Tweak:** Merge Sort is the perfect tool for this. When we are merging `left` and `right`, both are sorted. If `left[i]` is strictly greater than `right[j]`, it means `left[i]` and *every number after it in the left array* is greater than `right[j]`. We can count all these inversions

instantly without nested loops!

Snippet for Counting Inversions during Merge:

```
let inversionCount = 0;

// Inside the while loop of your Merge function:
if (left[i] <= right[j]) {
    result.push(left[i]);
    i++;
} else {
    // TWEAK: A number on the right is smaller than a number on the left!
    // Since 'left' is sorted, everything from 'i' to the end of 'left'
    // is also strictly greater than right[j]. Count them all!
    inversionCount = inversionCount + (left.length - i);

    result.push(right[j]);
    j++;
}
```

Variation B: Merging Linked Lists (O(1) Space Optimization) **The Problem:** Merge Sort takes O(N) space because we have to create new arrays to hold the merged results. Can we do it without creating new arrays? **The Tweak:** If the data structure is a Linked List instead of an Array, you don't need to create new memory space. You just change the `next` pointers to point to the smaller node. This drops the space complexity of the Merge phase to `O(1)`.

Snippet for Merging Linked Lists:

```
function mergeLinkedLists(l1, l2) {
    let dummyHead = new ListNode(0);
    let current = dummyHead;

    while (l1 !== null && l2 !== null) {
        if (l1.val <= l2.val) {
            current.next = l1; // Point to the existing node
            l1 = l1.next;
        } else {
            current.next = l2; // Point to the existing node
            l2 = l2.next;
        }
        current = current.next;
    }

    // Attach leftovers
    current.next = l1 !== null ? l1 : l2;
```

```
    return dummyHead.next;
}
```

Quicksort

Here is a complete guide to understanding and implementing the Quicksort algorithm.

---

1. Problem it solves and Why we need it

The Problem: Just like Merge Sort, we need to efficiently sort an unsorted array of numbers.

Why we need it (when we already have Merge Sort): Merge Sort is fantastic because it guarantees an O(N log2(N)) sorting time. However, Merge Sort's fatal flaw is its **Space Complexity**. It requires O(N) extra memory to create temporary arrays during the merge phase. If you are sorting a massive database of 10 million records, copying them into brand new arrays will eat up your RAM and slow down your system due to memory allocation overhead.

Quicksort solves this by sorting "In-Place". It achieves the same blazing fast O(N log2(N)) average time complexity, but it shuffles the elements around directly inside the original array. It requires virtually zero extra memory (just a tiny amount for the recursion call stack), making it highly cache-efficient and, in the real world, often faster than Merge Sort.

---

2. What is it?

Quicksort is a "Divide and Conquer" algorithm driven by a single, powerful concept: **The Pivot and Partitioning.**

Instead of blindly cutting the array in half, Quicksort picks one number to act as the "Pivot". It then rearranges the array so that:

1. Everything smaller than the Pivot moves to the left.
2. Everything larger than the Pivot moves to the right.

Once this is done, **the Pivot is exactly in its final, permanently sorted position.** We never have to look at it again! We then repeat this exact same process on the unsorted left side and the unsorted right side.

---

3. How does it work? (Walkthrough)

There are a few ways to partition an array. We will use the **Lomuto Partition Scheme** because it is the most intuitive and easiest to code. We always choose the *very last element* as our pivot.

We use two pointers:

- `i`: The boundary of the "Smaller than Pivot" club.
- `j`: The explorer that looks at each number.

Scenario A: The Standard Partition **Input Array:** `[8, 3, 5, 9, 2, 6]` **Pivot:** `6` (the last element)

```
Start:
Array: |  8  |  3  |  5  |  9  |  2  |  6  |
          ^                          pivot
         i,j
```

Step 1 (j=0, value 8): Is 8 smaller than 6? No. Do nothing. Explorer `j` moves forward. **Step 2 (j=1, value 3):** Is 3 smaller than 6? YES!

- Action: Swap `arr[i]` and `arr[j]`. (Swap 8 and 3).
- Move boundary `i` forward. Move `j` forward.

```
Array: |  3  |  8  |  5  |  9  |  2  |  6  |
               ^     ^               pivot
               i     j
```

Step 3 (j=2, value 5): Is 5 smaller than 6? YES!

- Action: Swap `arr[i]` and `arr[j]`. (Swap 8 and 5).
- Move `i` forward. Move `j` forward.

```
Array: |  3  |  5  |  8  |  9  |  2  |  6  |
                     ^     ^         pivot
                     i     j
```

Step 4 (j=3, value 9): Is 9 smaller than 6? No. Do nothing. `j` moves forward.
Step 5 (j=4, value 2): Is 2 smaller than 6? YES!

- Action: Swap `arr[i]` and `arr[j]`. (Swap 8 and 2).
- Move `i` forward. Move `j` forward.

```
Array: |  3  |  5  |  2  |  9  |  8  |  6  |
                           ^           ^
                           i         j/pivot
```

Step 6: The Final Swap! Explorer `j` has reached the pivot. The partitioning is done. Now, we just swap the Pivot itself with the element at boundary `i`. (Swap 9 and 6).

```
Final Partitioned Array:
|  3  |  5  |  2  | [6] |  8  |  9  |
```

Look at the magic! `6` is in its absolute final sorted position. Everything to the left `[3, 5, 2]` is smaller. Everything to the right `[8, 9]` is larger. We now recursively call Quicksort on `[3, 5, 2]` and `[8, 9]`.

Scenario B: The Worst Case (Already Sorted Array) What happens if the array is already sorted? **Input:** `[2, 3, 5, 6, 8, 9]` **Pivot:** `9`

Because *every* number is smaller than 9, pointer `i` and `j` will just move together step-by-step. At the end, 9 stays where it is. The right array size is 0. The left array size is N-1. Instead of chopping the problem in half, we only reduced the problem size by 1! This degrades Quicksort to O(N^2) time. (See "Variations and Tweaks" on how to fix this).

---

4. Code Sample (JavaScript)

```javascript
/**
 * Main Quicksort function
 * @param {number[]} arr - The array to sort
 * @param {number} low - Starting index
 * @param {number} high - Ending index
 */
function quickSort(arr, low = 0, high = arr.length - 1) {
    if (low < high) {
        // Find the pivot index. After partitioning, the pivot is in its final spot.
        const pivotIndex = partition(arr, low, high);

        // Recursively sort the elements to the left of the pivot
        quickSort(arr, low, pivotIndex - 1);

        // Recursively sort the elements to the right of the pivot
        quickSort(arr, pivotIndex + 1, high);
    }
    return arr;
}

/**
 * Lomuto Partitioning Scheme
 */
function partition(arr, low, high) {
    // We choose the last element as our pivot
    const pivot = arr[high];
```

```
    // i is the boundary of the "smaller than pivot" section
    let i = low;

    // j explores the array from left to right
    for (let j = low; j < high; j++) {
        // If we find a smaller element, swap it into the "smaller" section
        if (arr[j] < pivot) {
            swap(arr, i, j);
            i++; // Expand the "smaller" boundary
        }
    }

    // Finally, swap the pivot into its rightful place at boundary i
    swap(arr, i, high);

    // Return the final index of the pivot
    return i;
}

/**
 * Simple helper function to swap two array elements in-place
 */
function swap(arr, index1, index2) {
    const temp = arr[index1];
    arr[index1] = arr[index2];
    arr[index2] = temp;
}

// --- Usage ---
const unsorted = [8, 3, 5, 9, 2, 6];
console.log(quickSort(unsorted));
// Output: [ 2, 3, 5, 6, 8, 9 ]
```

---

5. LeetCode questions it applies to

- **Sort an Array (LeetCode 912):** Directly asks you to sort an array. *Note:* Leetcode intentionally passes worst-case sorted arrays to fail standard Quicksort. You MUST use the "Randomized Pivot" tweak (explained below) to pass this question with Quicksort.
- **Kth Largest Element in an Array (LeetCode 215):** The ultimate Quicksort question. You don't need to sort the whole array. You use a variation called "Quickselect". Since partitioning puts the pivot in its final sorted index, you just partition once. If the pivot ends up at index K, you found your answer instantly!

- **Sort Colors / Dutch National Flag (LeetCode 75):** An array has only 0s, 1s, and 2s. You use a 3-way partitioning variation of Quicksort where you maintain three pointers to separate the array into three sections simultaneously in a single pass.

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                        | Operations           | Big O
---------------------------|----------------------|--------
1. Partition Array          | Iterate N elements   | O(N) work per level
2. Divide Phase (Avg)       | Split roughly in 1/2 | O(log2(N)) levels
3. Divide Phase (Worst)     | Reduce size by 1     | O(N) levels
-----------------------------------------------------------
Average Time: log2(N) levels * N work = O(N * log2(N))
Worst Time: N levels * N work = O(N^2)

Space Complexity Derivation:
-----------------------------------------------------------
Task                        | Memory Used          | Big O
---------------------------|----------------------|--------
1. In-place Swapping        | No extra arrays      | O(1)
2. Call Stack (Avg)         | Recursion depth      | O(log2(N))
3. Call Stack (Worst)       | Recursion depth      | O(N)
-----------------------------------------------------------
Total Space (Average): O(log2(N))
```

---

7. Variations and Common Tweaks

Variation A: Quickselect (Finding the Kth element without fully sorting) **The Problem:** Find the 3rd smallest element in an array of 1,000,000 numbers. Sorting the whole thing takes O(N log N). We want O(N) time. **The Tweak:** After one `partition()`, the pivot is exactly where it belongs. Let's say we want the 3rd smallest (index 2). If our partition puts the pivot at index 5, we *know* the answer is to the left. We throw away the entire right half of the array and only recursively call `quickSelect` on the left half!

Snippet for Quickselect:

```
function quickSelect(arr, low, high, k) {
    if (low === high) return arr[low];

    const pivotIndex = partition(arr, low, high);
```

```
    if (k === pivotIndex) {
        return arr[k]; // We found it instantly!
    } else if (k < pivotIndex) {
        // It must be in the left half. Ignore the right!
        return quickSelect(arr, low, pivotIndex - 1, k);
    } else {
        // It must be in the right half. Ignore the left!
        return quickSelect(arr, pivotIndex + 1, high, k);
    }
}
```

Variation B: The Randomized Pivot (Beating the Worst Case) **The Problem:** If an array is already sorted, or reverse sorted, picking the last element as the pivot causes O(N^2) time. Hackers can exploit this to cause Denial of Service (DoS) attacks on web servers by sending specifically crafted malicious arrays to be sorted. **The Tweak:** Before doing the standard Lomuto partition, pick a completely random index between `low` and `high`. Swap that random element with the last element. Then, run standard partitioning! This makes it mathematically nearly impossible to hit the O(N^2) worst case.

Snippet for Randomized Partition:

```
function randomizedPartition(arr, low, high) {
    // Pick a random index between low and high (inclusive)
    const randomIndex = Math.floor(Math.random() * (high - low + 1)) + low;

    // Swap it to the end so our standard logic still works
    swap(arr, randomIndex, high);

    // Now just call your normal partition function
    return partition(arr, low, high);
}
```

Bucket sort

Here is a complete guide to understanding and implementing the Bucket Sort algorithm.

---

1. Problem it solves and Why we need it

The Problem: Algorithms like Merge Sort and Quicksort are fantastic, but they are "Comparison Sorts" (they figure out the order by directly comparing two elements against each other). Mathematics proves that any comparison-based sort can never be faster than O(N log2(N)) time.

But what if you need to sort millions of items even faster?

Why we need it: If you know something about the *nature* of your data—specifically, if you know your data is fairly evenly distributed over a known range (like test scores between 0 and 100, or floating-point numbers between 0.0 and 1.0)—you can cheat the mathematical limits.

Bucket Sort is a "Non-Comparison Sort". By using basic math to toss items into ordered groups, it avoids having to compare every item against every other item. Under the right conditions, Bucket Sort can sort an array in blazing fast **O(N)** linear time.

---

2. What is it?

Bucket Sort is a "Scatter-Gather" algorithm.

Imagine you have a messy stack of unsorted mail. Instead of sorting the whole stack at once, you set up 26 physical mail bins (buckets) labeled A through Z.

1. **Scatter:** You toss every piece of mail into its corresponding bin.
2. **Sort:** You individually sort the tiny handful of letters inside the 'A' bin, then the 'B' bin, etc. (Since the piles are so small, this takes almost no time).
3. **Gather:** You pick up the 'A' pile, put the 'B' pile under it, then the 'C' pile, and suddenly, your entire stack of mail is perfectly sorted.

---

3. How does it work? (Walkthrough)

Let's walk through two scenarios: the classic floating-point scenario, and a more complex integer range scenario.

Scenario A: The Classic 0.0 to 1.0 (Uniform Distribution) **Input Array:** [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21]

Step 1: Create the Buckets We have 7 numbers. We will create 10 buckets, representing the tenths ranges (0.0 to 0.9).

Step 2: Scatter To find the right bucket for a number, we multiply it by the number of buckets (10) and chop off the decimal.

- 0.78 * 10 = 7.8 -> Bucket 7
- 0.17 * 10 = 1.7 -> Bucket 1
- 0.39 * 10 = 3.9 -> Bucket 3
- …and so on.

```
Scattering Phase:
Bucket 0: []
```

```
Bucket 1: [0.17]
Bucket 2: [0.26, 0.21]  <-- Collision!
Bucket 3: [0.39]
Bucket 4: []
Bucket 5: []
Bucket 6: []
Bucket 7: [0.78, 0.72]  <-- Collision!
Bucket 8: []
Bucket 9: [0.94]
```

Step 3: Sort Individual Buckets Notice that Bucket 2 has [0.26, 0.21]. We use a simple sorting algorithm (like Insertion Sort) to fix this tiny sub-array.

- Bucket 2 becomes: [0.21, 0.26]
- Bucket 7 becomes: [0.72, 0.78]

Step 4: Gather Read the buckets from 0 to 9 and concatenate them. **Result:** [0.17, 0.21, 0.26, 0.39, 0.72, 0.78, 0.94]

Scenario B: The Integer Range (More Complex) What if your numbers are completely random integers? **Input Array:** [45, 12, 89, 33, 67, 21]

We can't just multiply by 10. We need a mapping formula to figure out which bucket a number belongs to.

1. **Find Min and Max:** Min is 12. Max is 89.
2. **Determine Bucket Count:** Let's say we want 5 buckets.
3. **Calculate the Gap (Range per bucket):** Gap = (Max - Min) / BucketCount Gap = (89 - 12) / 5 = 15.4 (Let's round up to 16 for safety).
4. **The Index Formula:** BucketIndex = Math.floor((Value - Min) / Gap)

Let's map the number 67: Index = Math.floor((67 - 12) / 16) Index = Math.floor(55 / 16) Index = Math.floor(3.43) -> **Bucket 3**

You repeat this for every number, sort the small buckets, and gather them!

---

4. Code Sample (JavaScript)

```javascript
/**
 * Bucket Sort for integers over a wide range.
 * @param {number[]} arr - The unsorted array
 * @param {number} bucketSize - How many numbers each bucket should hold
 * @return {number[]} - The sorted array
 */
function bucketSort(arr, bucketSize = 5) {
    if (arr.length === 0) {
        return arr;
    }
```

```
    // 1. Find minimum and maximum values
    let minValue = arr[0];
    let maxValue = arr[0];
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] < minValue) minValue = arr[i];
        if (arr[i] > maxValue) maxValue = arr[i];
    }

    // 2. Initialize the buckets
    // Calculate how many buckets we need based on the range
    const bucketCount = Math.floor((maxValue - minValue) / bucketSize) + 1;
    const buckets = new Array(bucketCount);

    for (let i = 0; i < buckets.length; i++) {
        buckets[i] = [];
    }

    // 3. Scatter the array elements into the correct buckets
    for (let i = 0; i < arr.length; i++) {
        // The mapping formula
        const bucketIndex = Math.floor((arr[i] - minValue) / bucketSize);
        buckets[bucketIndex].push(arr[i]);
    }

    // 4. Sort each bucket and Gather (Flatten)
    const sortedArray = [];
    for (let i = 0; i < buckets.length; i++) {
        // If the bucket has items, sort it.
        // Note: For small arrays, JavaScript's built-in sort (which is highly optimized)
        // or a manual Insertion Sort is perfect here.
        if (buckets[i].length > 0) {
            // Sort numerically
            buckets[i].sort((a, b) => a - b);

            // Gather items into the final array
            for (let j = 0; j < buckets[i].length; j++) {
                sortedArray.push(buckets[i][j]);
            }
        }
    }

    return sortedArray;
}

// --- Usage ---
```

```
const scores = [88, 12, 45, 99, 32, 56, 15, 8, 77, 65];
// Using a bucket size of 10
console.log(bucketSort(scores, 10));
// Output: [ 8, 12, 15, 32, 45, 56, 65, 77, 88, 99 ]
```

---

5. LeetCode questions it applies to

- **Top K Frequent Elements (LeetCode 347):** This is the most famous Bucket Sort interview question. Instead of sorting the numbers by their value, you map them by their *Frequency*. You create an array of buckets where the `index` is the frequency count. If the number '7' appears 4 times, you put '7' into bucket index 4. Then you just read the buckets from right-to-left to get the most frequent items in O(N) time!
- **Sort Characters By Frequency (LeetCode 451):** The exact same concept as above. Count the frequencies, use the frequency as the bucket index, and gather the characters backwards.
- **Maximum Gap (LeetCode 164):** A hard problem that asks you to find the maximum difference between two successive elements in a sorted form, but you MUST do it in linear O(N) time. You use the math formula from Scenario B to map numbers to buckets. The maximum gap will *always* be the difference between the maximum value of one bucket and the minimum value of the next non-empty bucket!

---

6. Time and Space Complexity

Let N be the number of elements in the array. Let K be the number of buckets.

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Find Min/Max           | Iterate array once   | O(N)
2. Scatter into Buckets   | Iterate array once   | O(N)
3. Sort Buckets (Avg)     | Tiny constant arrays | O(N)
4. Gather (Flatten)       | Output N elements    | O(N)
-----------------------------------------------------------
Average Time Complexity: O(N + K)

Worst Case Trap: O(N^2)
If all your data is clustered together (e.g., [1, 2, 1, 1, 2, 1])
and your mapping formula puts EVERY single element into the
exact same bucket, you just fall back to whatever sorting
algorithm you used for that individual bucket. If you use
Insertion Sort, your time becomes O(N^2).
```

```
Space Complexity Derivation:
-------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Create Buckets array   | Array of K arrays    | O(K)
2. Store elements         | N elements inside    | O(N)
-------------------------------------------------------
Total Space Complexity: O(N + K)
```

---

7. Variations and Common Tweaks

Variation A: The Frequency Bucket Sort (Top K Elements) **The Problem:** You need the most frequently occurring items, and you cannot use a sorting algorithm because it must be done in strict O(N) time. **The Tweak:** You use a Hashmap to count frequencies. Then, you create an array of buckets where the `bucket array length is N + 1`. The *index* of the bucket represents the frequency.

Snippet for Frequency Buckets:

```
// Step 1: Count frequencies using a Map
// Example Map: { 'apple': 3, 'banana': 1 }

// Step 2: Create buckets where Index = Frequency
// Max possible frequency is the length of the array (N)
const buckets = new Array(nums.length + 1).fill().map(() => []);

// Step 3: Populate buckets
for (const [number, frequency] of frequencyMap.entries()) {
    buckets[frequency].push(number);
}
// Now, buckets[3] contains ['apple']!
// Read backwards from the end of the buckets array to get the top K.
```

Variation B: Radix Sort Integration **The Problem:** What if your numbers are huge (like 9-digit social security numbers) and you don't want to figure out a complex mapping formula? **The Tweak:** You use 10 buckets (labeled 0 through 9). You run a Bucket Sort multiple times. First, you bucket everything by the 1s digit. Then you gather them. Next, you bucket them by the 10s digit. Gather them. Then the 100s digit. This is called **Radix Sort**, and it uses Bucket Sort as its internal engine to achieve stable, linear-time sorting for massive integers.

Segment tree

Here is a complete guide to understanding and building the Segment Tree algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you have an array of numbers, and you need to repeatedly perform two operations:

1. **Update:** Change the value of a specific element at index `i`.
2. **Query:** Find the sum (or minimum, or maximum) of all elements in a range from index `L` to `R`.

Why Prefix Sums fail here: As we discussed previously, Prefix Sums are brilliant for finding range sums instantly in O(1) time. However, Prefix Sums only work well on *static* arrays (arrays that never change). If you change just one element at the beginning of your array, you have to recalculate almost the entire prefix sum array, which takes O(N) time. If you have thousands of updates and queries mixed together, your program will grind to a halt.

Why we need Segment Trees: A Segment Tree gives us the best of both worlds for *dynamic* arrays (arrays that change). It allows us to update an element in **O(log2(N))** time AND query a range in **O(log2(N))** time. It is a massive performance boost when dealing with a heavy mix of updates and queries.

---

2. What is it?

A Segment Tree is a binary tree data structure.

- **The Leaves (Bottom Nodes):** Represent the individual elements of our original array.
- **The Internal Nodes:** Represent the "merged" result of their left and right children. For a Sum Segment Tree, an internal node stores the sum of all the leaves under it.
- **The Root:** Represents the result for the entire array (e.g., the total sum).

Instead of creating complex objects with "left" and "right" pointers, we usually build the tree flat inside a simple array. If a parent node is at index `i` in our tree array:

- Its Left Child is at index `(2 * i) + 1`
- Its Right Child is at index `(2 * i) + 2`

---

3. How does it work? (Walkthrough)

Let's build a Sum Segment Tree for the input array: `[2, 5, 1, 4]`

Step 1: Building the Tree Every node has a "responsibility range" of the original array it covers.

```
Level 0:                 [12]
                   (covers index 0 to 3)
                  /                      \
Level 1:       [7]                       [5]
          (covers 0 to 1)            (covers 2 to 3)
          /            \              /            \
Level 2: [2]          [5]            [1]          [4]
      (index 0)    (index 1)      (index 2)    (index 3)
```

- We start at the leaves (Level 2).
- The parent of `[2]` and `[5]` is their sum: `[7]`.
- The parent of `[1]` and `[4]` is their sum: `[5]`.
- The root is the sum of its children: 7 + 5 = `[12]`.

Step 2: Querying a Range Let's find the sum from **Index 1 to Index 3** (The numbers are 5, 1, 4. Answer should be 10). We start at the root `[12]` (range 0 to 3) and compare its range to our query `[1, 3]`.

There are 3 rules for querying a node:

1. **Total Overlap:** The node's range is completely inside the query range. -> *Return the node's value immediately!*
2. **No Overlap:** The node's range is completely outside the query range. -> *Return 0 (it doesn't contribute).*
3. **Partial Overlap:** The node's range partially overlaps the query. -> *Split the work! Ask the left and right children, then add their answers together.*

Walkthrough for Query [1, 3]:

- **Root (0 to 3):** Partial overlap with [1, 3]. Split to children!
- **Left Child (0 to 1):** Partial overlap with [1, 3]. Split!
- **Left-Left Child (0 to 0):** No overlap with [1, 3]. Returns 0.
- **Left-Right Child (1 to 1):** Total overlap with [1, 3]! Returns its value: 5.
- *Left Child total = 0 + 5 = 5.*
- **Right Child (2 to 3):** Total overlap with [1, 3]! (The whole right half of the array is inside our query). Returns its value instantly without going deeper: 5.
- **Root Total = 5 + 5 = 10.** We got the answer!

Step 3: Updating a Value Let's update **Index 2** from `1` to `3`. We walk down the tree to find the leaf for Index 2, update it, and then fix the parents on the way back up.

```
Before Update:                  After Update:
      [12]                            [14]  (+2)
      /  \                            /  \
    [7]  [5]                        [7]  [7]  (+2)
         / \                             / \
       [1] [4]                         [3] [4]  (+2 to leaf)
     (idx 2)                         (idx 2)
```

4. Code Sample (Easy to understand)

A Segment Tree is the "Swiss Army Knife" of range queries. An L5 engineer chooses it when they need both **point updates** and **range queries** to be extremely fast (logarithmic time).

---

1. Problem Explanation Imagine you have an array and you frequently need to:

1. Find the **sum** of numbers between index `L` and `R`.
2. **Update** a single number at index `i`.

If you use a simple loop for sums, it takes **O(n)**. If you use a Segment Tree, it takes **O(log n)**.

---

2. Solution Code (JavaScript) This implementation uses a **1-indexed array** for the tree because the math for finding children is simpler:

- Left child: `index * 2`
- Right child: `index * 2 + 1`

```javascript
class SegmentTree {
    constructor(data) {
        this.n = data.length;
        // The tree size is 4 * n to safely handle all nodes
        this.tree = new Array(4 * this.n).fill(0);
        this.build(data, 1, 0, this.n - 1);
    }

    /**
     * Build the tree recursively
     * node: current index in this.tree
     * start/end: range in the original data array
```

```
 */
build(data, node, start, end) {
    if (start === end) {
        // Leaf node stores the actual element
        this.tree[node] = data[start];
        return;
    }
    let mid = Math.floor((start + end) / 2);
    this.build(data, 2 * node, start, mid);
    this.build(data, 2 * node + 1, mid + 1, end);
    // Internal node stores the sum of its children
    this.tree[node] = this.tree[2 * node] + this.tree[2 * node + 1];
}

/**
 * Update a value at a specific index
 */
update(idx, val, node = 1, start = 0, end = this.n - 1) {
    if (start === end) {
        this.tree[node] = val;
        return;
    }
    let mid = Math.floor((start + end) / 2);
    if (idx <= mid) {
        this.update(idx, val, 2 * node, start, mid);
    } else {
        this.update(idx, val, 2 * node + 1, mid + 1, end);
    }
    this.tree[node] = this.tree[2 * node] + this.tree[2 * node + 1];
}

/**
 * Query the sum in range [L, R]
 */
query(L, R, node = 1, start = 0, end = this.n - 1) {
    // Case 1: Range is completely outside
    if (R < start || end < L) return 0;

    // Case 2: Range is completely inside
    if (L <= start && end <= R) return this.tree[node];

    // Case 3: Range is partially inside
    let mid = Math.floor((start + end) / 2);
    return this.query(L, R, 2 * node, start, mid) +
           this.query(L, R, 2 * node + 1, mid + 1, end);
}
```

```
}

// Usage example:
const nums = [1, 3, 5, 7, 9, 11];
const st = new SegmentTree(nums);
console.log(st.query(1, 3)); // Sum of [3, 5, 7] = 15
st.update(1, 10);            // Change 3 to 10
console.log(st.query(1, 3)); // Sum of [10, 5, 7] = 22
```

---

3. Complexity Analysis

```
Operation         | Complexity | Reason
------------------|------------|----------------------------------------
Build Tree        | O(n)       | We visit each of the 2n-1 nodes once.
Point Update      | O(log n)   | We only traverse the height of the tree.
Range Query       | O(log n)   | We visit at most 4 nodes at each level.
Space Complexity  | O(n)       | Tree requires 4n space.
------------------|------------|----------------------------------------
```

Why 4 * n? Mathematically, a Segment Tree for `n` elements is a full binary tree with at most `2 * (next power of 2 of n) - 1` nodes. For simplicity and to avoid overflow, **4n** is the industry-standard safe buffer used in competitive programming and interviews.

5. Code Sample (JavaScript)

```
/**
 * Segment Tree for finding Range Sums
 */
class SegmentTree {
    constructor(arr) {
        this.n = arr.length;
        this.originalArr = arr;
        // The tree array size is generally safe at 4 * N
        this.tree = new Array(4 * this.n).fill(0);

        if (this.n > 0) {
            this.buildTree(0, 0, this.n - 1);
        }
    }

    /**
     * Recursively builds the tree
     * @param {number} treeIndex - Current index in the tree array
     * @param {number} left - Left boundary of the original array range
```

```
 * @param {number} right - Right boundary of the original array range
 */
buildTree(treeIndex, left, right) {
    // Base Case: We hit a leaf node (1 element range)
    if (left === right) {
        this.tree[treeIndex] = this.originalArr[left];
        return;
    }

    // Find the middle to split the array
    const mid = Math.floor((left + right) / 2);
    const leftChildIdx = 2 * treeIndex + 1;
    const rightChildIdx = 2 * treeIndex + 2;

    // Recursively build left and right subtrees
    this.buildTree(leftChildIdx, left, mid);
    this.buildTree(rightChildIdx, mid + 1, right);

    // Merge step: Parent node is the sum of its children
    this.tree[treeIndex] = this.tree[leftChildIdx] + this.tree[rightChildIdx];
}

/**
 * Updates an element at a specific index
 */
update(index, newValue) {
    this._updateHelper(0, 0, this.n - 1, index, newValue);
}

_updateHelper(treeIndex, left, right, targetIndex, newValue) {
    // Base Case: We found the exact leaf node
    if (left === right) {
        this.tree[treeIndex] = newValue;
        this.originalArr[targetIndex] = newValue;
        return;
    }

    const mid = Math.floor((left + right) / 2);
    const leftChildIdx = 2 * treeIndex + 1;
    const rightChildIdx = 2 * treeIndex + 2;

    // Decide which path to walk down
    if (targetIndex <= mid) {
        // Target is in the left half
        this._updateHelper(leftChildIdx, left, mid, targetIndex, newValue);
    } else {
```

```
            // Target is in the right half
            this._updateHelper(rightChildIdx, mid + 1, right, targetIndex, newValue);
        }

        // Merge step: Update the parent node on the way back up
        this.tree[treeIndex] = this.tree[leftChildIdx] + this.tree[rightChildIdx];
    }

    /**
     * Queries the sum in range [queryLeft, queryRight]
     */
    query(queryLeft, queryRight) {
        return this._queryHelper(0, 0, this.n - 1, queryLeft, queryRight);
    }

    _queryHelper(treeIndex, left, right, queryLeft, queryRight) {
        // Rule 1: Total Overlap
        if (queryLeft <= left && queryRight >= right) {
            return this.tree[treeIndex];
        }

        // Rule 2: No Overlap
        if (queryLeft > right || queryRight < left) {
            return 0; // 0 does not affect a sum
        }

        // Rule 3: Partial Overlap (Split and ask children)
        const mid = Math.floor((left + right) / 2);
        const leftChildIdx = 2 * treeIndex + 1;
        const rightChildIdx = 2 * treeIndex + 2;

        const leftSum = this._queryHelper(leftChildIdx, left, mid, queryLeft, queryRight);
        const rightSum = this._queryHelper(rightChildIdx, mid + 1, right, queryLeft, queryRi

        return leftSum + rightSum;
    }
}

// --- Usage ---
const arr = [2, 5, 1, 4];
const segTree = new SegmentTree(arr);

console.log(segTree.query(1, 3)); // Output: 10 (5 + 1 + 4)
segTree.update(2, 3);             // Array is now [2, 5, 3, 4]
console.log(segTree.query(1, 3)); // Output: 12 (5 + 3 + 4)
```

---

5. LeetCode questions it applies to

- **Range Sum Query - Mutable (LeetCode 307):** The absolute classic. The problem gives you an array and asks you to implement `update(index, val)` and `sumRange(left, right)`. The code provided in section 4 is the exact solution for this problem.
- **Count of Smaller Numbers After Self (LeetCode 315):** A hard problem that becomes manageable with a Segment Tree. You process the array from right to left. The segment tree's "leaves" represent the *frequencies* of numbers occurring. When you look at a number `X`, you query the tree for the sum of frequencies in the range `[min_possible_value, X - 1]`. Then you update the tree by adding 1 to the leaf representing `X`.
- **Falling Squares (LeetCode 699):** Deals with finding the maximum height of overlapping squares. This requires a variation called a "Maximum Segment Tree" with "Lazy Propagation" (explained below).

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-------------------------------------------------------
Task                     | Operations                 | Big O
-------------------------|----------------------------|-------------
1. Build Tree            | Visit every node once      | O(N)
                         | A tree with N leaves has   |
                         | roughly 2N total nodes.    |

2. Point Update          | Walk from root to 1 leaf   | O(log2(N))
                         | Height of tree is log2(N)  |

3. Range Query           | Worst case: visit 4 nodes  | O(log2(N))
                         | per level of the tree.     |
-------------------------------------------------------

Space Complexity Derivation:
-------------------------------------------------------
Task                     | Array Size                 | Big O
-------------------------|----------------------------|-------------
1. Store Tree Array      | Max size is 4 * N nodes    | O(N)
                         | to guarantee enough space  |
                         | for a perfect binary tree  |
-------------------------------------------------------
```

---

7. Variations and Common Tweaks

The beauty of a Segment Tree is that by changing just one or two lines of code, you solve completely different problems.

Variation A: Range Minimum/Maximum Query (RMQ) Instead of asking for the sum of a range, the question asks for the smallest or largest number in a range.

The Tweak: Change the "Merge Step" from addition to `Math.min()` or `Math.max()`. Also, change the "No Overlap" return value. If you return `0` for a minimum query, you'll ruin the answer. You must return `Infinity` for minimums, and `-Infinity` for maximums.

Snippets for Minimum Segment Tree:

```
// Change in buildTree and update (Merge Step)
this.tree[treeIndex] = Math.min(this.tree[leftChildIdx], this.tree[rightChildIdx]);

// Change in query (No Overlap)
if (queryLeft > right || queryRight < left) {
    return Infinity; // Infinity will lose to any real number in Math.min
}

// Change in query (Return Step)
return Math.min(leftAnswer, rightAnswer);
```

Variation B: Lazy Propagation (Range Updates) **The Problem:** What if an interviewer asks you to update an *entire range* of numbers at once? (e.g., "Add 5 to every element from index 2 to index 1000"). If you do a Point Update 999 times, it takes `O(N * log2(N))` which is too slow. **The Fix:** We use "Lazy Propagation". We add a second array called `lazy`. When we are told to update a large range, we find the high-level nodes that cover that range, update them, and leave a "lazy note" for their children. We don't update the children until a future query actually forces us to walk down there. This keeps range updates at a blazing fast `O(log2(N))`.

Snippet of how Lazy Propagation intercepts a query:

```
_queryHelper(treeIndex, left, right, queryLeft, queryRight) {
    // 1. Resolve any lazy notes before doing anything else!
    if (this.lazy[treeIndex] !== 0) {
        // Apply the pending update to this node
        this.tree[treeIndex] = this.tree[treeIndex] + (this.lazy[treeIndex] * (right - left

        // Pass the lazy note down to children (if not a leaf)
        if (left !== right) {
            this.lazy[2 * treeIndex + 1] += this.lazy[treeIndex];
```

```
            this.lazy[2 * treeIndex + 2] += this.lazy[treeIndex];
        }
        // Clear current node's lazy note
        this.lazy[treeIndex] = 0;
    }

    // ... proceed with normal query logic (Total Overlap, No Overlap, etc.)
}
```

Fenwick tree

Here is a complete guide to understanding and using the Fenwick Tree algorithm.

---

1. Problem it solves and Why we need it

The Problem: Just like the Segment Tree, we need to efficiently handle an array that frequently changes (Updates) while simultaneously answering range queries (like "What is the sum from index L to R?").

Why we need it (when we already have Segment Trees): While Segment Trees are incredibly powerful, they have a few downsides:

1. They take up a lot of memory (4 * N space).
2. They require writing somewhat lengthy recursive code.
3. Recursion adds a small delay (overhead) in practice.

The Fenwick Tree solves the exact same point-update/range-query problems as a Segment Tree, but it is **much more memory-efficient (exactly N space), incredibly short to write (a few lines of code), and extremely fast** because it uses bitwise operations instead of recursion.

The tradeoff? It is slightly harder to wrap your head around at first, and it only works for operations that can be "reversed" (like addition/subtraction). It is very difficult to use a Fenwick tree for Range Minimum/Maximum queries.

---

2. What is it?

A Fenwick Tree (also known as a Binary Indexed Tree or BIT) is an elegant data structure that looks like a standard flat array, but acts like a tree by exploiting the binary representation of its indices.

Every number can be represented as a sum of powers of 2. A Fenwick Tree uses this fact to break down any range sum into a combination of pre-calculated sub-ranges whose lengths are powers of 2.

The Golden Rule: A Fenwick tree strictly uses **1-based indexing**. The very first element must be at index 1. If you try to use index 0, the binary math will break completely.

---

3. How does it work? (Walkthrough)

The entire magic of a Fenwick Tree relies on one bitwise trick: isolating the lowest set bit. The lowest set bit (the rightmost `1` in binary) tells us exactly how many elements a specific index is "responsible" for.

The Magic Formula: `i & (-i)` In programming, negative numbers are stored using "Two's Complement". Doing a bitwise AND between a number and its negative perfectly isolates that rightmost 1-bit.

- Example: For index 6 (binary `0110`), `6 & -6` evaluates to `2`. This means index 6 stores the sum of 2 elements!

Scenario: The Responsibility Chart Let's look at an array of size 8 to see what each index in the Fenwick Tree stores:

```
Index | Binary | Lowest Set Bit | Responsible For (Range)   | Covers Length
-------------------------------------------------------------------------
  1   |  0001  |       1        | [1]                       | 1 element
  2   |  0010  |       2        | [1, 2]                    | 2 elements
  3   |  0011  |       1        | [3]                       | 1 element
  4   |  0100  |       4        | [1, 2, 3, 4]              | 4 elements
  5   |  0101  |       1        | [5]                       | 1 element
  6   |  0110  |       2        | [5, 6]                    | 2 elements
  7   |  0111  |       1        | [7]                       | 1 element
  8   |  1000  |       8        | [1, 2, 3, 4, 5, 6, 7, 8]  | 8 elements
```

Notice a pattern? Odd indices (1, 3, 5, 7) only cover themselves. Powers of two (2, 4, 8) cover everything from the beginning up to themselves!

Walkthrough A: Querying a Prefix Sum (Sum up to Index 7) We want the sum from index 1 to 7. We start at 7 and work our way backward by **subtracting** the lowest set bit.

1. **Start at index 7** (Binary `0111`).

- Add the value at `tree[7]` to our total. (This gives us the sum of `[7]`).
- Subtract the lowest set bit: 7 - (7 & -7) -> 7 - 1 = 6.

2. **Move to index 6** (Binary `0110`).

- Add `tree[6]` to our total. (This gives us the sum of `[5, 6]`).
- Subtract the lowest set bit: 6 - (6 & -6) -> 6 - 2 = 4.

3. **Move to index 4** (Binary `0100`).

- Add `tree[4]` to our total. (This gives us the sum of `[1, 2, 3, 4]`).
- Subtract the lowest set bit: 4 - (4 & -4) -> 4 - 4 = 0.

4. **Hit 0. Stop.** *Total pieces gathered:* `[7] + [5, 6] + [1, 2, 3, 4]`. We successfully got the sum of 1 through 7!

Walkthrough B: Updating a Value (Add +3 to Index 5) If we change the original array at index 5, we must update `tree[5]` and every parent node that includes index 5 in its responsibility range. We climb the tree by **adding** the lowest set bit.

1. **Start at index 5** (Binary `0101`).

- Add +3 to `tree[5]`.
- Add lowest set bit: 5 + (5 & -5) -> 5 + 1 = 6.

2. **Move to index 6** (Binary `0110`).

- Add +3 to `tree[6]` (Because index 6 covers [5, 6]).
- Add lowest set bit: 6 + (6 & -6) -> 6 + 2 = 8.

3. **Move to index 8** (Binary `1000`).

- Add +3 to `tree[8]` (Because index 8 covers [1…8]).
- Add lowest set bit: 8 + (8 & -8) -> 8 + 8 = 16.

4. **Index 16 is out of bounds. Stop.**

---

4. Code Sample (JavaScript)

```javascript
/**
 * Fenwick Tree (Binary Indexed Tree) for finding Range Sums
 */
class FenwickTree {
    constructor(size) {
        // We add +1 because Fenwick tree strictly uses 1-based indexing
        this.size = size;
        this.tree = new Array(size + 1).fill(0);
    }

    /**
     * Adds 'delta' to the element at the given original array index.
     * Note: Input index should be 0-based for user convenience,
     * but we shift it to 1-based internally.
     */
    update(index, delta) {
        // Shift to 1-based indexing
        let i = index + 1;
```

```
        // Climb the tree by ADDING the lowest set bit
        while (i <= this.size) {
            this.tree[i] = this.tree[i] + delta;

            // i & (-i) isolates the lowest set bit
            i = i + (i & -i);
        }
    }

    /**
     * Queries the prefix sum from index 0 up to 'index'
     */
    queryPrefix(index) {
        let sum = 0;
        // Shift to 1-based indexing
        let i = index + 1;

        // Walk down the tree by SUBTRACTING the lowest set bit
        while (i > 0) {
            sum = sum + this.tree[i];
            i = i - (i & -i);
        }
        return sum;
    }

    /**
     * Queries the sum in range [left, right]
     */
    queryRange(left, right) {
        // Just like Prefix Sum arrays: Sum(L, R) = Prefix(R) - Prefix(L-1)
        return this.queryPrefix(right) - this.queryPrefix(left - 1);
    }
}

// --- Usage ---
// Original array: [2, 5, 1, 4] (Size 4)
const bit = new FenwickTree(4);

// Build the tree by updating each element
const arr = [2, 5, 1, 4];
for (let i = 0; i < arr.length; i++) {
    bit.update(i, arr[i]);
}

console.log(bit.queryRange(1, 3)); // Output: 10 (5 + 1 + 4)
```

```
bit.update(2, 2);                  // Add 2 to index 2 (changes 1 to 3)
console.log(bit.queryRange(1, 3)); // Output: 12 (5 + 3 + 4)
```

---

5. LeetCode questions it applies to

- **Range Sum Query - Mutable (LeetCode 307):** The exact code above solves this problem flawlessly. You keep a copy of the original array so you can calculate `delta` (the difference between the new value and the old value) and pass that `delta` to the `update()` function.
- **Count of Smaller Numbers After Self (LeetCode 315):** A brilliant application. You iterate from the right side of the array to the left. The Fenwick tree doesn't store the numbers themselves; it stores *frequencies* of numbers you've seen so far. For each number, you query the tree to see the sum of frequencies of all numbers strictly smaller than it, then you update the tree to mark that you've seen the current number.
- **Reverse Pairs (LeetCode 493):** Similar to 315, but you query the Fenwick tree for the count of numbers that are strictly less than `nums[i] / 2.0` before adding the current number to the tree.

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations                  | Big O
--------------------------|-----------------------------|-------------
1. Point Update           | Walk through binary bits    | O(log2(N))
                          | Number of set bits is at    |
                          | most log2(N).               |

2. Range Query            | Two prefix sum queries      | O(log2(N))
                          | each taking log2(N) steps.  |

3. Build Tree (by loop)   | N elements * update time    | O(N * log2(N))
-----------------------------------------------------------

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Array Size                  | Big O
--------------------------|-----------------------------|-------------
1. Store Tree Array       | Size is N + 1               | O(N)
-----------------------------------------------------------
```

---

7. Variations and Common Tweaks

Variation A: Range Update, Point Query (The Difference Array Tweak) **The Problem:** The standard Fenwick handles (Point Update + Range Query). What if the problem asks you to add a value to an *entire range* [L, R], and then ask for the value of a *single point*? **The Tweak:** You use the Fenwick tree as a **Difference Array**. To add `value` to the range `[L, R]`:

1. You `update(L, value)`
2. You `update(R + 1, -value)` When you query the prefix sum up to index `X`, the prefix sum will automatically apply all the intersecting range updates and give you the exact point value at `X`.

Snippet for Range Update/Point Query:

```
// Add 'val' to everything from index L to R
function rangeUpdate(bit, L, R, val) {
    bit.update(L, val);
    bit.update(R + 1, -val);
}

// Find the exact value at index X
function pointQuery(bit, X) {
    return bit.queryPrefix(X);
}
```

Variation B: 2D Fenwick Tree (Image/Matrix Updates) **The Problem:** You have a 2D matrix, and you need to update a specific cell and query the sum of a rectangular area. **The Tweak:** A 2D array where you use a nested loop for updates and queries. The outer loop traverses the rows using the `i & -i` trick, and the inner loop traverses the columns using a `j & -j` trick.

Snippet for 2D Update:

```
function update2D(row, col, delta) {
    for (let r = row + 1; r <= rows; r += r & -r) {
        for (let c = col + 1; c <= cols; c += c & -c) {
            tree2D[r][c] += delta;
        }
    }
}
```

Aho-Corasick algorithm

Here is a complete guide to understanding and building the Aho-Corasick algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are building a profanity filter for a chat app, or a DNA sequence analyzer. You have a "Dictionary" of hundreds of target words, and a massive "Text" (the chat logs or DNA strand). You need to find all occurrences of all dictionary words inside the text.

Why we need it:

- **Naive Approach:** You could use a `.includes()` or loop through the text for *every single word* in your dictionary. If the text has length N and the total length of all dictionary words is M, this takes O(N * M) time. Too slow.
- **Standard Trie Approach:** You could put the dictionary into a Trie (Prefix Tree), and then for every single letter in the text, try to walk down the Trie. But if you fail halfway through a word, you have to back up and start all over again from the next letter in the text.
- **The Aho-Corasick Solution:** This algorithm searches for *all* words simultaneously in exactly **one single pass** through the text. It never backs up. It solves the multi-pattern string search problem in optimal O(N + M + Z) time (where Z is the number of matches found).

---

2. What is it?

The Aho-Corasick algorithm is a **Trie (Prefix Tree) on steroids**. It takes a standard Trie and adds two special types of "safety nets" (pointers) to it:

1. **Failure Links:** If you are walking down the Trie reading a word, and suddenly the next letter in the text doesn't match, you don't start over. The "Failure Link" acts as a teleportation portal. It instantly moves you to the longest possible valid prefix that matches the suffix of what you just read.
2. **Output Links (Dictionary Links):** Sometimes, a smaller dictionary word is completely hidden inside a larger one (like "he" inside "she"). Output links ensure that when you find "she", you also get credit for finding "he" without doing extra work.

Essentially, Aho-Corasick turns a dictionary of words into a Deterministic Finite Automaton (DFA) state machine.

---

3. How does it work? (Walkthrough)

Let's use a dictionary with four words: `["he", "she", "his", "hers"]` Our target text is: `ushers`

Step 1: Build the Standard Trie First, we insert all dictionary words into a standard Prefix Tree.

```
       (Root)
       /    \
     'h'    's'
     / \      \
   'e'* 'i'   'h'
   /      \     \
 'r'      's'* 'e'*
 /
's'*

(* means a word ends here)
```

Step 2: Build the Failure Links (The hardest sub-concept) We build failure links level-by-level using Breadth-First Search (BFS).

The Rule for Failure Links: To find the failure link for a current node, you look at its parent's failure link, and see if that "fallback" node has a child with the same letter.

Let's trace the interesting failure links:

- Level 1: 'h' and 's' fail to Root. (If you fail on the first letter, you start over).
- Let's look at the 'h' under 's' (representing the prefix "sh").
- Parent is 's'. Parent's failure link goes to Root.
- Does Root have an 'h' child? Yes!
- So, "sh" fails to "h". (Because "h" is the longest valid suffix of "sh").
- Let's look at the 'e' under 'sh' (representing "she").
- Parent is "sh". Parent's failure link goes to "h".
- Does "h" have an 'e' child? Yes!
- So, "she" fails to "he".

```
Failure Link Diagram (Simplified):
(sh) ----fails to----> (h)
(she) ---fails to----> (he)
(his) ---fails to----> (s)
```

Step 3: Build the Output Links While building failure links, if a node's failure link points to a node that is the end of a word, we create an "Output Link".

- "she" fails to "he". Since "he" is a complete word, we give "she" an output link to "he". This means: "If I ever successfully match 'she', I must also output 'he'!".

Step 4: Search the Text ("ushers") We start at the Root and read one letter at a time.

1. Read **'u'**: Root has no 'u'. Stay at Root.
2. Read **'s'**: Go to node 's'.
3. Read **'h'**: Go down to node 'sh'.
4. Read **'e'**: Go down to node 'she'.

- *Match Found!* Node 'she' is a word. Output: "she".
- *Output Link Check!* Node 'she' has an output link to "he". Output: "he".

5. Read **'r'**: We are at 'she'. It has no 'r' child!

- *Failure Link!* Teleport from 'she' to 'he'.
- Does 'he' have an 'r' child? Yes! Move to 'her'.

6. Read **'s'**: Move to 'hers'.

- *Match Found!* Node 'hers' is a word. Output: "hers".

Final Results: Found "she", "he", and "hers". Notice we never backed up in the text! We read `ushers` straight through exactly once.

---

4. Code Sample (JavaScript)

```
class AhoCorasickNode {
    constructor() {
        this.children = {};       // Map of character -> AhoCorasickNode
        this.isWord = false;      // True if a dictionary word ends here
        this.word = null;         // The actual word string (for easy output)
        this.fail = null;         // The Failure Link (fallback node)
        this.outputLink = null;   // The Output Link (subset word node)
    }
}

class AhoCorasick {
    constructor(dictionary) {
        this.root = new AhoCorasickNode();
        this.buildTrie(dictionary);
        this.buildFailureLinks();
    }

    // Step 1: Standard Trie Insertion
    buildTrie(dictionary) {
```

```
    for (let word of dictionary) {
        let curr = this.root;
        for (let char of word) {
            if (!curr.children[char]) {
                curr.children[char] = new AhoCorasickNode();
            }
            curr = curr.children[char];
        }
        curr.isWord = true;
        curr.word = word;
    }
}

// Step 2: Build Failure and Output Links via BFS
buildFailureLinks() {
    let queue = [];

    // Initialize level 1 nodes (children of root) to fail to root
    for (let char in this.root.children) {
        let child = this.root.children[char];
        child.fail = this.root;
        queue.push(child);
    }

    while (queue.length > 0) {
        let curr = queue.shift();

        // Check all children of the current node
        for (let char in curr.children) {
            let child = curr.children[char];

            // 1. Find the failure link for the child
            let fallback = curr.fail;

            // Keep falling back until we find a node that has 'char' as a child,
            // or until we hit the root.
            while (fallback !== null && !fallback.children[char]) {
                fallback = fallback.fail;
            }

            if (fallback === null) {
                child.fail = this.root;
            } else {
                child.fail = fallback.children[char];
            }
```

```
            // 2. Build the Output Link
            // If the failure link points to a completed word, point to it!
            // Otherwise, inherit the failure link's output link.
            if (child.fail.isWord) {
                child.outputLink = child.fail;
            } else {
                child.outputLink = child.fail.outputLink;
            }

            queue.push(child);
        }
    }
}

// Step 3: Search the text in O(N + matches) time
search(text) {
    let results = [];
    let curr = this.root;

    for (let i = 0; i < text.length; i++) {
        let char = text[i];

        // If no path forward, follow failure links!
        while (curr !== null && !curr.children[char]) {
            curr = curr.fail;
        }

        if (curr === null) {
            curr = this.root;
            continue;
        } else {
            curr = curr.children[char];
        }

        // Check if current node is a word
        if (curr.isWord) {
            results.push({ word: curr.word, endIndex: i });
        }

        // Check output links (hidden smaller words)
        let temp = curr.outputLink;
        while (temp !== null) {
            results.push({ word: temp.word, endIndex: i });
            temp = temp.outputLink;
        }
    }
```

```
        return results;
    }
}

// --- Usage ---
const dictionary = ["he", "she", "his", "hers"];
const ac = new AhoCorasick(dictionary);
const text = "ushers";

// Output:
// [ { word: 'she', endIndex: 2 },
//   { word: 'he', endIndex: 2 },
//   { word: 'hers', endIndex: 5 } ]
console.log(ac.search(text));
```

---

5. LeetCode questions it applies to

- **Stream of Characters (LeetCode 1032):** The absolute perfect problem for Aho-Corasick. You are given a dictionary of words, and then a continuous stream of characters arriving one by one. You must return true if any suffix of the stream matches a dictionary word. Since Aho-Corasick processes text character-by-character as a state machine, you just feed the stream into the `search` step one letter at a time. *(Note: This can also be solved by building a Trie of reversed words, but AC is the textbook forward-matching solution).*
- **String Matching in an Array (LeetCode 1408):** You need to find all strings in an array that are substrings of another string in the array. You can build an Aho-Corasick automaton using all the strings, and then feed each string into the automaton as text.
- **Multi-Search (Cracking the Coding Interview / Hard):** Given a big string and an array of smaller strings, find all occurrences of the smaller strings in the big string. This is the exact definition of the Aho-Corasick problem.

---

6. Time and Space Complexity

Let **N** = Length of the Target Text. Let **M** = Total length of all words in the Dictionary combined. Let **Z** = Total number of matches found in the text. Let **A** = Alphabet size (e.g., 26 for lowercase English).

```
Time Complexity Derivation:
-------------------------------------------------------
Task                      | Operations                 | Big O
```

```
-------------------------|----------------------------|-------------
1. Build Trie            | Insert M characters        | O(M)
2. Build Failure Links   | BFS traverses M nodes      | O(M)
3. Search Text           | Read N characters, plus    | O(N + Z)
                         | Z total output link jumps. |
-----------------------------------------------------------
Total Time Complexity: O(N + M + Z)

Space Complexity Derivation:
-----------------------------------------------------------
Task                     | Node Connections           | Big O
-------------------------|----------------------------|-------------
1. Store Trie Nodes      | M nodes, each with 'A'     | O(M * A)
                         | potential child pointers   |
-----------------------------------------------------------
Total Space Complexity: O(M * A)
```

---

7. Variations and Common Tweaks

Variation A: Infinite Stream Processing (Stateful Search) **The Problem:** In LeetCode 1032, you don't get the whole text at once. You get it character by character, and you must return `true/false` immediately. **The Tweak:** Instead of a `search(text)` function that loops over a string, you make the `curr` node a class variable. You write a `query(char)` method that updates `curr` and checks for matches on the fly.

Snippet for Stream Query:

```
// Inside AhoCorasick class
this.currState = this.root; // Keep track of where we are across calls

query(char) {
    while (this.currState !== null && !this.currState.children[char]) {
        this.currState = this.currState.fail;
    }

    if (this.currState === null) {
        this.currState = this.root;
        return false;
    }

    this.currState = this.currState.children[char];

    // Return true if we hit a word or an output link
    return this.currState.isWord || this.currState.outputLink !== null;
```

```
}
```

Variation B: Counting Frequencies Instead of Outputting **The Problem:** You just want to know *how many times* each dictionary word appeared, not their exact indices. **The Tweak:** You don't need to populate an array of objects. Instead, you add a `count` variable to your Trie nodes, or map the `curr.word` to an external frequency hash map whenever a match or output link is triggered.

KMP algorithm

Here is a complete guide to understanding and implementing the KMP (Knuth-Morris-Pratt) algorithm.

---

1. Problem it solves and Why we need it

The Problem: You are given a massive "Text" string (like an entire book) and a smaller "Pattern" string (like a specific word). You need to find if and where the Pattern exists inside the Text.

Why the "Naive" Approach fails: The obvious way is to put the Pattern at the very beginning of the Text and check letter by letter. If you hit a mismatch, you slide the Pattern over by exactly 1 spot, go all the way back to the beginning of the Pattern, and start comparing again. If your Text has length N and your Pattern has length M, this naive approach can take O(N * M) time. If the text is "AAAAAAAB" and the pattern is "AAAB", you do a massive amount of redundant checking.

Why we need KMP: The KMP algorithm guarantees an **O(N + M)** time complexity. It achieves this blazing speed with one golden rule: **Never move the Text pointer backwards.** Once you read a character in the Text, you never re-evaluate it. When a mismatch happens, KMP knows exactly how far to slide the Pattern forward without missing any potential matches.

---

2. What is it?

KMP is a string-matching algorithm that relies on pre-processing the Pattern. Before we even look at the Text, we analyze the Pattern to find internal repetitions.

We do this by creating an **LPS Array** (Longest Proper Prefix which is also Suffix).

- **Prefix:** Characters from the beginning of a string. (For "HELLO", prefixes are H, HE, HEL, HELL. *Proper* prefixes exclude the whole word).

- **Suffix:** Characters from the end of a string. (For "HELLO", suffixes are O, LO, LLO, ELLO).

The LPS array tells us: *If I fail at this exact letter in my pattern, how much of the beginning of my pattern did I just successfully match anyway?*

---

3. How does it work? (Walkthrough)

There are two major phases: Building the LPS array, and using it to Search.

Phase A: Building the LPS Array Let's build the LPS array for the Pattern: `A B A B C`

We use two pointers: `len` (length of previous longest prefix suffix) and `i` (current character). We start with `lps[0] = 0` (a single character has no proper prefix/suffix).

- **i = 1 ('B'):** Does 'B' match our prefix at `len` (0, which is 'A')? No. So `lps[1] = 0`.
- **i = 2 ('A'):** Does 'A' match prefix at `len` (0, which is 'A')? YES! Increment `len` to 1. `lps[2] = 1`.
- **i = 3 ('B'):** Does 'B' match prefix at `len` (1, which is 'B')? YES! Increment `len` to 2. `lps[3] = 2`.
- **i = 4 ('C'):** Does 'C' match prefix at `len` (2, which is 'A')? No.
- *The tricky part:* We don't just reset `len` to 0. We check the previous LPS value: `len = lps[len - 1] -> lps[1] -> 0`.
- Now does 'C' match prefix at `len` (0, 'A')? No. `lps[4] = 0`.

```
Pattern: |  A  |  B  |  A  |  B  |  C  |
Index:   |  0  |  1  |  2  |  3  |  4  |
         -------------------------------
LPS:     |  0  |  0  |  1  |  2  |  0  |
```

Read index 3: The `2` means "The substring 'ABAB' ends with 'AB', and it also starts with 'AB'."

Phase B: Searching the Text **Text:** `A B A B A B C A` **Pattern:** `A B A B C`

We use pointer `i` for Text, pointer `j` for Pattern.

Step 1: Match characters one by one.

```
Text:    A B A B A B C A
Pattern: A B A B C
         ^ ^ ^ ^ !
         (Mismatch at i=4, j=4)
```

At `i=4`, Text is 'A', but Pattern at `j=4` is 'C'. **Mismatch!**

Step 2: The KMP Magic! In a naive search, we would move `i` back to index 1. With KMP, `i` STAYS at 4. We change `j`. We look at the character *before* the mismatch in our Pattern (which is `j-1`, or index 3). We check our LPS array: `lps[3] = 2`. This tells us: "You just matched 'ABAB'. The last 2 characters ('AB') are also the first 2 characters of your pattern. Don't throw them away!" We set `j = 2`.

Step 3: Resume Matching

```
Text:    A B A B A B C A
Pattern:     A B A B C
                 ^
                 (We resume comparing Text[i=4] with Pattern[j=2])
```

Does Text `i=4` ('A') match Pattern `j=2` ('A')? YES!

- `i=5, j=3`: 'B' matches 'B'. YES!
- `i=6, j=4`: 'C' matches 'C'. YES!

We reached the end of our Pattern (`j == 5`). We found a match ending at index 6!

---

4. Code Sample (JavaScript)

```javascript
/**
 * Main KMP Search Function
 * @param {string} text
 * @param {string} pattern
 * @returns {number} - Index of first match, or -1 if not found
 */
function kmpSearch(text, pattern) {
    if (pattern.length === 0) return 0;

    // 1. Pre-process the pattern to build the LPS array
    const lps = buildLPS(pattern);

    let i = 0; // Pointer for text
    let j = 0; // Pointer for pattern

    // 2. Traverse the text exactly once
    while (i < text.length) {
        if (text[i] === pattern[j]) {
            // Characters match, move both pointers forward
            i++;
            j++;
```

```
        }

        if (j === pattern.length) {
            // We reached the end of the pattern! Match found.
            // Return the starting index in the text.
            return i - j;
        } else if (i < text.length && text[i] !== pattern[j]) {
            // Mismatch after some matches
            if (j !== 0) {
                // Don't reset i! Just push j back to the safe LPS prefix
                j = lps[j - 1];
            } else {
                // If j is 0, we can't fall back. We MUST move i forward.
                i++;
            }
        }
    }

    return -1; // No match found
}

/**
 * Helper to build the Longest Proper Prefix Suffix Array
 */
function buildLPS(pattern) {
    const lps = new Array(pattern.length).fill(0);
    let len = 0; // Length of the previous longest prefix suffix
    let i = 1;   // Start from the second character

    while (i < pattern.length) {
        if (pattern[i] === pattern[len]) {
            // Prefix matches suffix!
            len++;
            lps[i] = len;
            i++;
        } else {
            // Mismatch!
            if (len !== 0) {
                // Fall back to the previous longest prefix suffix
                // Note: We do NOT increment i here. We re-test the same character.
                len = lps[len - 1];
            } else {
                // Reached the absolute beginning, no prefix matches
                lps[i] = 0;
                i++;
            }
```

```
        }
    }
    return lps;
}

// --- Usage ---
console.log(kmpSearch("ABABDABACDABABCABAB", "ABABCABAB")); // Output: 10
```

---

5. LeetCode questions it applies to

- **Find the Index of the First Occurrence in a String (LeetCode 28):** This is the exact implementation of KMP. You are given a `haystack` (text) and a `needle` (pattern). The code provided above solves this natively.
- **Repeated Substring Pattern (LeetCode 459):** A beautiful trick. You build the LPS array for the given string. If a string is created by repeating a substring, the last value in the LPS array will tell you the length of the repeated portion! Check if `lps[n-1] !== 0` and `n % (n - lps[n-1]) === 0`.
- **Shortest Palindrome (LeetCode 214):** A hard problem made easy by LPS. You want to find the longest palindromic prefix of a string. You create a new string: `s + "#" + reversed(s)`. You build the LPS array for this new string. The very last value in the LPS array gives you the exact length of the longest palindromic prefix!

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations                  | Big O
--------------------------|-----------------------------|-------------
1. Build LPS Array        | Traverse pattern (size M)   | O(M)
                          | While loop falls back, but  |
                          | total steps bounded by M.   |

2. Search Text            | Traverse text (size N)      | O(N)
                          | Pointer 'i' NEVER goes      |
                          | backwards. Max N steps.     |
-----------------------------------------------------------
Total Time Complexity: O(N) + O(M) = O(N + M)

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Array Size                  | Big O
```

```
------------------------|---------------------------|-------------
1. Store LPS Array       | Size of pattern (M)        | O(M)
-----------------------------------------------------------
```

---

7. Variations and Common Tweaks

Variation A: Find ALL Occurrences (Not just the first) **The Problem:** You want an array of every index where the pattern appears, not just the first one. **The Tweak:** When `j === pattern.length`, instead of returning immediately, you push the found index to an array. Then, you force the algorithm to pretend it just had a mismatch at the very end, causing `j` to fall back via the LPS array and continue searching.

Snippet for Finding All:

```
const results = [];
// ... inside the search loop:
if (j === pattern.length) {
    results.push(i - j);  // Save the found index
    j = lps[j - 1];       // TWEAK: Fall back to continue finding more!
}
```

Variation B: The "Concat and Find" Trick (For Cyclic Arrays / Palindromes) **The Problem:** You need to find if string A is a rotation of string B, or you need to find the longest overlapping prefix and suffix between two different strings. **The Tweak:** You don't use the full search algorithm! You only use the `buildLPS` helper function. You combine the two strings with a special delimiter (like `#`) that guarantees they don't bleed into each other.

Snippet for Overlapping Strings:

```
// Find the longest prefix of string 'B' that is a suffix of string 'A'
const combined = B + "#" + A;
const lps = buildLPS(combined);

// The very last value in the LPS array is our answer!
const overlapLength = lps[lps.length - 1];
```

Boyer-Moore voting algorithm

Here is a complete guide to understanding and using the Boyer-Moore Voting Algorithm.

---

1. Problem it solves and Why we need it

The Problem: You are given an array of numbers, and you need to find the **Majority Element**. By mathematical definition, a "Majority Element" is an element that appears **strictly more than N / 2 times** in an array of size N.

Why we need it: There are several ways to solve this, but they all have drawbacks:

- **The Hashmap Approach:** You count the frequencies of every number. This takes O(N) time but requires **O(N) extra memory** to store the map.
- **The Sorting Approach:** You sort the array and pick the number right in the middle. This takes O(1) memory, but sorting takes **O(N log N) time**, which is slow.

The Boyer-Moore Voting Algorithm is pure magic because it solves the problem in **O(N) time and O(1) space**. It processes the array in a single pass without storing any frequencies.

---

2. What is it?

Think of this algorithm as a "Battle Royale" or "Gang Warfare".

Imagine every unique number in the array is a different army.

- If two soldiers from the *same* army meet, they join forces (Count increases).
- If two soldiers from *different* armies meet, they fight and **cancel each other out** (Both die, Count decreases).

The Golden Rule of the Algorithm: Because the majority army has *more* soldiers than all the other armies combined (more than N / 2), even if all the other armies team up to fight the majority army, the majority army will always have at least one soldier left standing at the end.

---

3. How does it work? (Walkthrough)

We only need to keep track of two variables:

1. `candidate` (The army currently holding the hill)
2. `count` (How many soldiers that army has on the hill)

Let's look at two scenarios to see how the mathematical cancellation works.

Scenario A: The clear majority **Input Array:** `[7, 7, 5, 7, 1]`

```
Start: candidate = null, count = 0

| Index | Value | Action                                | candidate | count |
|-------|-------|---------------------------------------|-----------|-------|
|   0   |   7   | Count is 0! New candidate is 7.       |     7     |   1   |
|   1   |   7   | Matches candidate. Count + 1.         |     7     |   2   |
|   2   |   5   | Mismatch! Fight! Count - 1.           |     7     |   1   |
|   3   |   7   | Matches candidate. Count + 1.         |     7     |   2   |
|   4   |   1   | Mismatch! Fight! Count - 1.           |     7     |   1   |

Result: 7 survives. The majority element is 7.
```

Scenario B: The hidden majority (More Complex) What happens if the majority element gets wiped out early on? Let's trace it. **Input Array:** `[5, 5, 1, 1, 1, 5, 1]` (The majority is 1)

```
Start: candidate = null, count = 0

| Idx | Val | Action                                    | candidate | count |
|-----|-----|-------------------------------------------|-----------|-------|
|  0  |  5  | Count is 0. New candidate is 5.           |     5     |   1   |
|  1  |  5  | Matches 5. Count + 1.                     |     5     |   2   |
|  2  |  1  | Mismatch! Fight! Count - 1.               |     5     |   1   |
|  3  |  1  | Mismatch! Fight! Count - 1.               |     5     |   0   |
                ^^^ [Army 5 and Army 1 wiped out!] ^^^
|  4  |  1  | Count is 0. New candidate is 1.           |     1     |   1   |
|  5  |  5  | Mismatch! Fight! Count - 1.               |     1     |   0   |
                ^^^ [Army 1 and Army 5 wiped out!] ^^^
|  6  |  1  | Count is 0. New candidate is 1.           |     1     |   1   |

Result: 1 survives. The majority element is 1.
```

Notice how at Index 3 and Index 5, the count hit `0`. Whenever the count hits `0`, it means *all elements processed so far have perfectly cancelled each other out.* We just forget the past and start fresh with the next number!

---

4. Code Sample (JavaScript)

```javascript
/**
 * Finds the majority element in an array using Boyer-Moore Voting.
 * Note: This assumes a majority element ALWAYS exists in the array.
 * @param {number[]} nums
 * @return {number}
 */
function majorityElement(nums) {
```

```
    let candidate = null;
    let count = 0;

    for (let i = 0; i < nums.length; i++) {
        const currentNumber = nums[i];

        // If our count is 0, the previous candidate was wiped out.
        // We assign the current number as the new candidate.
        if (count === 0) {
            candidate = currentNumber;
        }

        // If the current number is our candidate, add a soldier.
        // Otherwise, they fight and cancel out (subtract a soldier).
        if (currentNumber === candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

// --- Usage ---
console.log(majorityElement([2, 2, 1, 1, 1, 2, 2])); // Output: 2
```

---

5. LeetCode questions it applies to

- **Majority Element (LeetCode 169):** This is the exact algorithm. The problem explicitly states "You may assume that the majority element always exists in the array." You just run the standard algorithm and return the candidate.
- **Majority Element II (LeetCode 229):** A famous step-up. You need to find all elements that appear strictly more than `N / 3` times. You solve this by tracking *two* candidates and *two* counts simultaneously. (See Variation B below).

---

6. Time and Space Complexity

```
Time Complexity Derivation:
-------------------------------------------------------------
Task                      | Operations           | Big O
```

```
-------------------------|----------------------|--------
1. Initialize variables  | 2 assignments        | O(1)
2. Loop through array    | N iterations         | O(N)
3. Check/Update count    | 1 if/else per step   | O(1)
-----------------------------------------------------------
Total Time Complexity: O(1) + O(N) * O(1) = O(N)

Space Complexity Derivation:
-----------------------------------------------------------
Task                     | Memory Used          | Big O
-------------------------|----------------------|--------
1. Track 'candidate'     | 1 variable           | O(1)
2. Track 'count'         | 1 variable           | O(1)
-----------------------------------------------------------
Total Space Complexity: O(1) + O(1) = O(1) Constant Space
```

---

7. Variations and Common Tweaks

Variation A: The Majority Element is NOT Guaranteed (The Verification Pass) **The Problem:** What if the interviewer says, "Find the majority element, but if no element appears more than N/2 times, return -1." **The Trap:** If you pass `[1, 2, 3]` to Boyer-Moore, it will return `3`. But `3` only appears once (which is not $> 3/2$). The algorithm assumes a majority exists. If it doesn't, the last surviving element is just a false positive. **The Tweak:** You must do a second loop over the array to actually count how many times your surviving `candidate` appears to verify it truly is a majority.

Snippet for Verification Pass:

```javascript
let candidate = majorityElement(nums); // Run standard Boyer-Moore

// Verification Pass
let actualCount = 0;
for (let i = 0; i < nums.length; i++) {
    if (nums[i] === candidate) {
        actualCount++;
    }
}

if (actualCount > nums.length / 2) {
    return candidate;
} else {
    return -1; // No majority exists
}
```

Variation B: The N / 3 Majority (LeetCode 229) **The Problem:** Find all elements appearing more than N / 3 times. **The Logic:** Mathematically, there can be at most **two** elements that appear more than N / 3 times (e.g., 34% and 34%. You can't have three things take up 34% each). **The Tweak:** We track `candidate1`, `count1`, `candidate2`, and `count2`. If a new number matches neither, we decrement *both* counts. We also must do a Verification Pass at the end for both candidates.

Snippet for Two Candidates:

```
let candidate1 = null, count1 = 0;
let candidate2 = null, count2 = 0;

for (let i = 0; i < nums.length; i++) {
    let num = nums[i];

    if (num === candidate1) {
        count1++;
    } else if (num === candidate2) {
        count2++;
    } else if (count1 === 0) {
        candidate1 = num;
        count1++;
    } else if (count2 === 0) {
        candidate2 = num;
        count2++;
    } else {
        // Mismatch with BOTH candidates. Both armies take a hit.
        count1--;
        count2--;
    }
}
// (Follow this with a verification loop for both candidate1 and candidate2)
```

DFS

Here is a complete guide to understanding and using the Depth-First Search (DFS) algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are lost in a massive maze. How do you find your way out? If you just walk randomly, you might walk in circles forever. You need a systematic way to explore every single corridor until you find the exit, ensuring you never explore the exact same path twice.

Why we need it: In computer science, we frequently represent data as interconnected webs called "Graphs" (like a network of friends on social media, or a map of city roads) or hierarchical structures called "Trees" (like the folders on your computer).

We need DFS because it gives us a structured, foolproof way to visit every single piece of data (node) in these structures. Compared to its sibling, Breadth-First Search (BFS), DFS is incredibly memory-efficient because it only needs to remember the specific path it is currently walking down, rather than remembering the entire perimeter of its search area.

---

2. What is it?

Depth-First Search is exactly what it sounds like: a "dive deep first" exploration strategy.

Its core philosophy is: **Go as far down a path as you possibly can. When you hit a dead end, take one step back (backtrack), and try the next available path.**

It achieves this by using a **Stack**. A stack is a Last-In, First-Out structure (like a stack of plates). You can either create your own stack array in code, or—much more commonly—you just use recursion, which automatically uses the computer's built-in "Call Stack" to remember where to backtrack to.

---

3. How does it work? (Walkthrough)

Let's look at two scenarios: a simple tree (which naturally flows downward) and a tricky graph (which can loop back on itself).

Scenario A: The Binary Tree In a tree, data flows from the Root down to the Leaves. There are no loops.

```
      [A]
     /   \
   [B]   [C]
   / \     \
 [D] [E]   [F]
```

Walkthrough:

1. Start at the root: **A**.
2. A has two paths (Left to B, Right to C). DFS always picks one direction and commits. Let's go Left.
3. Move to **B**.
4. B has paths to D and E. Let's go Left.

5. Move to **D**.
6. D has no children! It is a dead end. We **backtrack** to the last decision point (B).
7. At B, the left path is done. We take the right path.
8. Move to **E**.
9. E is a dead end. We backtrack to B. B's paths are all done. We backtrack to A.
10. At A, the left side is entirely explored. We take the right path.
11. Move to **C**.
12. C has no left child, so we go right. Move to **F**.
13. F is a dead end. Backtrack to C. Backtrack to A. Done!

Order Visited: A, B, D, E, C, F.

Scenario B: The Graph with a Cycle (The Trap) Graphs are dangerous because nodes can point back to each other. If you aren't careful, DFS will bounce back and forth forever in an infinite loop.

```
[1] --- [2]
 |       |
[3] --- [4]
```

The Fix: The Visited Set To prevent infinite loops, we carry a notepad called a `visited` set. Before we step on a node, we check if it is in our notepad. If it is, we treat it as a dead end. If it isn't, we step on it and immediately write it down.

Walkthrough (Starting at 1):

1. Start at **1**. Add 1 to notepad. (Notepad: {1})
2. Node 1 connects to 2 and 3. Go to 2.
3. Move to **2**. Add 2 to notepad. (Notepad: {1, 2})
4. Node 2 connects to 1 and 4. We check 1. It is in the notepad! Ignore it. We check 4. Not in notepad.
5. Move to **4**. Add 4 to notepad. (Notepad: {1, 2, 4})
6. Node 4 connects to 2 and 3. Node 2 is in the notepad. Ignore it. Go to 3.
7. Move to **3**. Add 3 to notepad. (Notepad: {1, 2, 4, 3})
8. Node 3 connects to 1 and 4. Both are in the notepad! Node 3 has no unexplored paths.
9. Backtrack all the way back up. Every node has been visited.

---

4. Code Sample (Javascript)

Here is the standard recursive template for DFS on a Graph using an Adjacency List.

```javascript
/**
 * Explores a graph using Depth-First Search
 * @param {Object} graph - An adjacency list representing the graph
 * @param {string} startNode - The node to begin the search
 */
function dfs(graph, startNode) {
    // 1. Create the notepad to prevent infinite loops
    const visited = new Set();
    const resultOrder = [];

    // 2. Define the recursive helper function
    function explore(currentNode) {
        // Base Case: If we have already seen this node, stop and turn back
        if (visited.has(currentNode)) {
            return;
        }

        // Action: Mark as visited and process the node
        visited.add(currentNode);
        resultOrder.push(currentNode);

        // Recursive Step: Look at all the neighbors of this node
        const neighbors = graph[currentNode];
        for (let i = 0; i < neighbors.length; i++) {
            const nextNode = neighbors[i];

            // Dive down the next path!
            explore(nextNode);
        }
    }

    // 3. Kick off the search from the starting node
    explore(startNode);

    return resultOrder;
}

// --- Usage ---
// This adjacency list represents the Scenario B graph above
const myGraph = {
    '1': ['2', '3'],
    '2': ['1', '4'],
    '3': ['1', '4'],
    '4': ['2', '3']
};
```

```
console.log(dfs(myGraph, '1'));
// Output: [ '1', '2', '4', '3' ]
```

---

5. LeetCode questions that it applies to

- **Number of Islands (LeetCode 200):** You are given a grid of 1s (land) and 0s (water). You loop through the grid. Whenever you find a 1, you increase your island count, and then instantly trigger a DFS. The DFS explores up, down, left, and right, turning every connected 1 into a 0 (sinking the island). This ensures you don't count the same island twice.
- **Max Area of Island (LeetCode 695):** Identical to Number of Islands, but instead of just sinking the island, your DFS function returns the number 1 plus the area of its neighbors. You keep track of the largest area returned.
- **Lowest Common Ancestor of a Binary Tree (LeetCode 236):** You use DFS to search the left and right subtrees. If a node finds target A in its left tree and target B in its right tree, that node is the lowest common ancestor.
- **Clone Graph (LeetCode 133):** You use DFS to traverse the original graph. As you visit each node, you make a new copy of it and store the copy in a Hashmap. If your DFS hits a node it has already seen, it just returns the clone from the Hashmap to wire up the connections correctly.

---

6. Time and Space Complexity

Let V equal the number of Vertices (Nodes). Let E equal the number of Edges (Connections).

```
Time Complexity Derivation:
-------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Visit every Vertex     | 1 call per node      | O(V)
2. Check every Edge       | Look at neighbors    | O(E)
-------------------------------------------------------
Total Time: We visit every node once and check every
connection once. The time is exactly O(V + E).

Space Complexity Derivation:
-------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Visited Set            | Stores up to V items | O(V)
2. Recursion Call Stack   | V calls deep (Worst) | O(V)
```

```
-------------------------------------------------------
Total Space: O(V) + O(V) = O(V)
*Note: In the worst-case scenario (a graph that is just one
long straight line), the recursion stack goes V levels deep.
```

---

7. Variations and Common Tweaks

Variation A: Tree Traversals (Pre-order, In-order, Post-order) When doing DFS on a Binary Tree, you don't need a `visited` set because trees don't have cycles. The only thing that changes is *when* you process the node's value. **The Tweak:** Move the `result.push(node.val)` line before, between, or after the recursive calls.

Snippet:

```
function treeDFS(node) {
    if (node === null) return;

    // 1. PRE-ORDER: Process here (Root, Left, Right)

    treeDFS(node.left);

    // 2. IN-ORDER: Process here (Left, Root, Right) - Gives sorted order for BST!

    treeDFS(node.right);

    // 3. POST-ORDER: Process here (Left, Right, Root) - Good for deleting trees
}
```

Variation B: Backtracking (The "Un-visit" Tweak) **The Problem:** In games like Chess, or puzzles like "Word Search" on a grid, you want to explore a path, but if it's wrong, you want to pretend you were never there so you can use those squares again for a different path. **The Tweak:** After your recursive calls finish, you remove the current node from the `visited` set. This is the hallmark of Backtracking.

Snippet:

```
function backtrack(node) {
    visited.add(node);      // Step on the square

    explore(node.next);     // Dive deep

    visited.delete(node);   // TWEAK: Step off the square so other paths can use it!
}
```

Variation C: Iterative DFS (Preventing Stack Overflow) **The Problem:** If a graph has 100,000 nodes in a straight line, recursive DFS will crash the program with a "Maximum Call Stack Size Exceeded" error. **The Tweak:** Don't use recursion. Create an actual array to act as your stack and use a `while` loop.

Snippet:

```
function iterativeDFS(startNode) {
    const stack = [startNode];
    const visited = new Set();

    while (stack.length > 0) {
        // Pop from the TOP of the stack
        const current = stack.pop();

        if (!visited.has(current)) {
            visited.add(current);
            // Process node here...

            // Push neighbors onto the stack
            for (let neighbor of graph[current]) {
                stack.push(neighbor);
            }
        }
    }
}
```

Post-Order DFS

Here is a complete guide to understanding and implementing the Post-Order Depth-First Search (DFS) algorithm.

---

1. Problem it solves and Why we need it

The Problem: In standard DFS (Pre-Order), you process a node the moment you first touch it. But what if you are trying to delete a folder on your computer? You cannot delete the parent folder until you have completely deleted all the files and subfolders inside of it.

Similarly, imagine a math equation represented as a tree: `(2 + 3) * 4`. You cannot perform the multiplication at the top of the tree until you have fully calculated the addition at the bottom of the tree.

Why we need it: We need Post-Order DFS whenever a problem requires a **"Bottom-Up"** approach. It guarantees that you will never process a parent

node until you have completely processed 100% of its children. It solves problems where a parent's answer directly depends on the answers returned by its children.

---

2. What is it?

Post-Order DFS is a specific flavor of Depth-First Search used primarily on Trees.

The name "Post-Order" tells you exactly when the current node is processed: *after* its children. The strict, unbreakable rule of Post-Order traversal is:

1. **Go Left:** Recursively visit the entire left subtree.
2. **Go Right:** Recursively visit the entire right subtree.
3. **Process Node (Self):** Look at the current node's value and do your work.

You can remember it as: **Left -> Right -> Root**.

---

3. How does it work? (Walkthrough)

Let's walk through two scenarios to see exactly how the computer's Call Stack forces this bottom-up behavior.

Scenario A: The Standard Binary Tree Input Tree:

```
        [A]
       /   \
    [B]    [C]
    / \      \
  [D] [E]    [F]
```

Walkthrough:

1. Start at **A**. Rule says: Go Left. (A is put on hold).
2. Arrive at **B**. Rule says: Go Left. (B is put on hold).
3. Arrive at **D**. Rule says: Go Left. No left child! Go Right. No right child!

- *Action:* Process Node. We print **D**.
- D is finished. We backtrack up to B.

4. Back at **B**. Left is done. Rule says: Go Right. (B remains on hold).
5. Arrive at **E**. Go Left (null), Go Right (null).

- *Action:* Process Node. We print **E**.
- E is finished. We backtrack up to B.

6. Back at **B**. Left is done. Right is done.

- *Action:* Process Node. We print **B**.
- B is finished. We backtrack up to A.

7. Back at **A**. Left is entirely done! Rule says: Go Right. (A remains on hold).
8. Arrive at **C**. Go Left (null). Go Right.
9. Arrive at **F**. Go Left (null), Go Right (null).

- *Action:* Process Node. We print **F**.
- F is finished. Backtrack up to C.

10. Back at **C**. Left is done. Right is done.

- *Action:* Process Node. We print **C**.
- C is finished. Backtrack up to A.

11. Back at **A**. Left is done. Right is done.

- *Action:* Process Node. We print **A**.

Final Post-Order: D, E, B, F, C, A. *(Notice how the Root 'A' is the absolute last thing printed!)*

Scenario B: The Math Expression Tree Let's see why this is so powerful for calculations. We want to evaluate: `(5 + 3) * 2`

```
      [*]
     /   \
   [+]    [2]
   / \
 [5] [3]
```

Walkthrough:

1. Start at `[*]`. Go Left.
2. Arrive at `[+]`. Go Left.
3. Arrive at `[5]`. It's a leaf (a number). Return `5` up to the parent.
4. Back at `[+]`. Go Right.
5. Arrive at `[3]`. It's a leaf. Return `3` up to the parent.
6. Back at `[+]`. Both children returned!

- *Action:* Evaluate `Left + Right` -> `5 + 3 = 8`.
- Return `8` up to the root.

7. Back at `[*]`. Left returned `8`. Go Right.
8. Arrive at `[2]`. It's a leaf. Return `2` up to the parent.
9. Back at `[*]`. Both children returned!

- *Action:* Evaluate `Left * Right` -> `8 * 2 = 16`.
- Final Answer is 16.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 * this.val = (val===undefined ? 0 : val)
 * this.left = (left===undefined ? null : left)
 * this.right = (right===undefined ? null : right)
 * }
 */

/**
 * Standard Recursive Post-Order Traversal
 * @param {TreeNode} root
 * @return {number[]}
 */
function postorderTraversal(root) {
    const result = [];

    function traverse(node) {
        // Base case: If we hit a dead end, turn back
        if (node === null) {
            return;
        }

        // 1. Go completely down the Left side
        traverse(node.left);

        // 2. Go completely down the Right side
        traverse(node.right);

        // 3. Process the current Node (Self)
        // We only push to the result array after BOTH children are fully processed
        result.push(node.val);
    }

    // Kick off the recursion
    traverse(root);

    return result;
}
```

---

5. LeetCode questions it applies to

- **Maximum Depth of Binary Tree (LeetCode 104):** You can't know how deep you are until your children tell you how deep they are. You ask the left child for its depth, ask the right child for its depth, and return `Math.max(leftDepth, rightDepth) + 1` to your parent.
- **Lowest Common Ancestor of a Binary Tree (LeetCode 236):** You search left and right for the two target nodes. If the left side returns Target A, and the right side returns Target B, then the *current node* (where the two paths meet) is the Lowest Common Ancestor.
- **Binary Tree Maximum Path Sum (LeetCode 124):** A "Hard" problem that becomes easy with Post-Order. A parent node asks its children: "What is the maximum straight-line path you can offer me?" The parent then calculates if connecting the left path, itself, and the right path forms a new global maximum, before returning the best single path up to its own parent.

---

6. Time and Space Complexity

Let N equal the total number of nodes in the tree. Let H equal the height of the tree (from root to furthest leaf).

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Visit every Node       | 1 call per node      | O(N)
2. Process Node Data      | O(1) push to array   | O(1)
-----------------------------------------------------------
Total Time: We visit every single node exactly once.
Time Complexity is strictly O(N).

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Recursion Call Stack   | Varies by Tree Shape | O(H)
-----------------------------------------------------------
Total Space: O(H).
* Note: If the tree is perfectly balanced, H is log2(N),
so space is O(log2(N)). If the tree is a worst-case
straight line (like a linked list), H is N, making the
space O(N).
```

---

7. Variations and Common Tweaks

Variation A: N-ary Tree Post-Order **The Problem:** What if a node doesn't just have a `left` and `right` child, but an array of 10 children (like a file directory system)? **The Tweak:** Instead of explicitly calling `traverse(left)` and `traverse(right)`, you use a loop to iterate through the `children` array. You process the parent node only after the loop finishes.

Snippet for N-ary Tree:

```
function traverseNary(node) {
    if (node === null) return;

    // 1. Recursively visit ALL children first
    for (let i = 0; i < node.children.length; i++) {
        traverseNary(node.children[i]);
    }

    // 2. Process Self last!
    result.push(node.val);
}
```

Variation B: Iterative Post-Order (The Two-Stack Hack) **The Problem:** Recursive DFS can cause a "Stack Overflow" error if the tree is massively deep. You need an iterative version using a `while` loop. However, writing a true iterative Post-Order is notoriously difficult because you have to visit a node twice (once on the way down, once on the way up) before processing it. **The Tweak:** We cheat. Pre-Order is Root -> Left -> Right. If we slightly tweak Pre-Order to be **Root -> Right -> Left**, and then we **reverse** the final result array, we magically get **Left -> Right -> Root** (which is Post-Order!). We use one stack for traversal, and another array to hold the reversed result.

Snippet for Iterative Two-Stack Hack:

```
function iterativePostOrder(root) {
    if (root === null) return [];

    const stack = [root];
    const result = [];

    while (stack.length > 0) {
        const current = stack.pop();

        // Push current value (acts like Root)
        result.push(current.val);

        // Push Left first, so Right is popped first
        // Result order being built: Root -> Right -> Left
```

```
        if (current.left !== null) stack.push(current.left);
        if (current.right !== null) stack.push(current.right);
    }

    // Reverse the result to get: Left -> Right -> Root
    return result.reverse();
}
```

Post-Order DFS for Topological Sorting

Here is a complete guide to understanding and implementing Post-Order DFS for Topological Sorting.

---

1. Problem it solves and Why we need it

The Problem: You have a set of tasks with strict dependencies (e.g., you must put on your socks before your shoes). You need to find a valid linear order to perform these tasks without violating any rules. This means dealing with a Directed Acyclic Graph (DAG) and finding a Topological Sort.

Why we need it (when we already have Kahn's Algorithm / BFS): While Kahn's Algorithm (the BFS approach using In-Degrees) is fantastic, Post-Order DFS offers a completely different, elegant recursive approach. We need this DFS version because:

1. **Seamless Integration:** If you are already doing a deep recursive search through a graph for other reasons, it is much easier to just add a single line of code to build your topological sort along the way, rather than rewriting everything into a BFS queue.
2. **Cycle Detection Magic:** The DFS approach uses a brilliant "3-State" memory system that makes detecting circular dependencies (like "Task A requires Task B, and Task B requires Task A") incredibly efficient.

---

2. What is it?

We already know that **DFS** dives as deep as possible into a graph, and **Post-Order** means we process a node *only after* we have fully processed all of its children/neighbors.

If Task A depends on Task B (A -> B), DFS will start at A, immediately travel down to B, and finish B first. Because B is finished, B is added to our `result` list. Then, DFS backtracks to A. A is now finished, so A is added to the `result` list.

Our list currently looks like this: `[B, A]`. Because we added the deepest dependencies first, the list is built completely backwards! To get the correct Topological Sort, all we have to do is **reverse the final list**.

---

3. How does it work? (Walkthrough)

To make this work safely on a graph, we must use a **3-State Visited Map** (often called 3-Coloring). Instead of just "visited" or "not visited", every node is in one of three states:

- **0 (Unvisited):** We haven't touched this node yet.
- **1 (Visiting):** We are currently exploring this node or its neighbors. It is active in our current path.
- **2 (Fully Visited):** We have completely finished exploring this node and all its downstream paths. It is safe.

Let's walk through two scenarios.

Scenario A: The Standard Schedule (Clean DAG) Tasks: 0, 1, 2, 3. Rules: 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3.

```
Graph:
       [0]
      /   \
    [1]   [2]
      \   /
       [3]
```

Walkthrough:

1. **Start at 0:** Mark 0 as *Visiting* (State 1). Go to neighbor 1.
2. **Arrive at 1:** Mark 1 as *Visiting* (State 1). Go to neighbor 3.
3. **Arrive at 3:** Mark 3 as *Visiting* (State 1).

- Node 3 has no neighbors!
- Mark 3 as *Fully Visited* (State 2).
- Add 3 to `result`. -> `result = [3]`
- Backtrack to 1.

4. **Back at 1:** Node 1 has no more neighbors to check.

- Mark 1 as *Fully Visited* (State 2).
- Add 1 to `result`. -> `result = [3, 1]`
- Backtrack to 0.

5. **Back at 0:** Node 0 has another neighbor: 2. Go to 2.
6. **Arrive at 2:** Mark 2 as *Visiting* (State 1). Go to neighbor 3.

- *Check 3:* State is 2 (Fully Visited). We safely ignore it!

- Node 2 has no more neighbors.
- Mark 2 as *Fully Visited* (State 2).
- Add 2 to `result`. -> `result = [3, 1, 2]`
- Backtrack to 0.

7. **Back at 0:** Node 0 has no more neighbors.

- Mark 0 as *Fully Visited* (State 2).
- Add 0 to `result`. -> `result = [3, 1, 2, 0]`

Final Step: Reverse the result array. Output: `[0, 2, 1, 3]`. A perfect topological sort!

Scenario B: The Deadly Cycle What happens if there is a loop? `A -> B -> C -> A`.

Walkthrough:

1. Start at A. Mark A as *Visiting* (State 1). Go to B.
2. Arrive at B. Mark B as *Visiting* (State 1). Go to C.
3. Arrive at C. Mark C as *Visiting* (State 1). Go to A.
4. **The Trap:** We check A. A is currently in State 1 (*Visiting*)!

- If we hit a node that is *currently actively being visited* in our own current path, it means we circled back around to ourselves.
- **CYCLE DETECTED!** We immediately abort and return an error.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Topological Sort using Post-Order DFS with 3-State Cycle Detection
 * @param {number} numNodes - Total number of tasks
 * @param {number[][]} edges - Array of [prerequisite, task] edges
 * @return {number[]} - Valid order, or empty array if cycle exists
 */
function topologicalSortDFS(numNodes, edges) {
    // 1. Build Adjacency List
    const adjList = new Map();
    for (let i = 0; i < numNodes; i++) {
        adjList.set(i, []);
    }
    for (let i = 0; i < edges.length; i++) {
        const prereq = edges[i][0];
        const task = edges[i][1];
        adjList.get(prereq).push(task);
    }
```

```
// 2. Setup the 3-State Tracking Array
// 0 = Unvisited, 1 = Visiting, 2 = Fully Visited
const state = new Array(numNodes).fill(0);
const result = [];
let hasCycle = false;

// 3. Define the Post-Order DFS Helper
function dfs(node) {
    // If we hit a node we are currently visiting, we found a loop!
    if (state[node] === 1) {
        hasCycle = true;
        return;
    }
    // If we hit a node that is completely finished, it's safe. Ignore it.
    if (state[node] === 2) {
        return;
    }

    // Mark as Visiting (State 1)
    state[node] = 1;

    // Recursively visit all neighbors
    const neighbors = adjList.get(node);
    for (let i = 0; i < neighbors.length; i++) {
        dfs(neighbors[i]);
        if (hasCycle) return; // Abort early if cycle is found deep down
    }

    // Mark as Fully Visited (State 2)
    state[node] = 2;

    // POST-ORDER ACTION: Add to result only after all neighbors are done
    result.push(node);
}

// 4. Launch DFS from every unvisited node (handles disconnected graphs)
for (let i = 0; i < numNodes; i++) {
    if (state[i] === 0 && !hasCycle) {
        dfs(i);
    }
}

// 5. If a cycle was detected, it's impossible to sort
if (hasCycle) {
    return [];
}
```

```
    // 6. The list was built bottom-up. Reverse it for the final answer!
    return result.reverse();
}
```

---

5. LeetCode questions it applies to

- **Course Schedule II (LeetCode 210):** The absolute perfect fit for this exact algorithm. You are given courses and prerequisites, and you must return the valid order. The 3-state DFS handles it beautifully.
- **Find Eventual Safe States (LeetCode 802):** A node is "safe" if every possible path starting from it leads to a terminal node (a node with no outgoing edges). This problem is essentially asking you to identify nodes that are *not* part of a cycle. The 3-State DFS handles this natively. Any node that successfully reaches State 2 is a safe state!
- **Longest Increasing Path in a Matrix (LeetCode 329):** While not explicitly asked to return a sorted array, the core logic relies on Post-Order DFS to evaluate a cell's neighbors before concluding the cell's own maximum path length. Because water strictly flows downhill (no cycles), you just memoize the fully visited states.

---

6. Time and Space Complexity

Let V equal the number of Vertices (Tasks). Let E equal the number of Edges (Dependencies).

```
Time Complexity Derivation:
-------------------------------------------------------
Task                     | Operations           | Big O
-------------------------|----------------------|--------
1. Build Graph           | Iterate all Edges    | O(E)
2. Outer Loop            | Check every Vertex   | O(V)
3. DFS Traversal         | Traverse Graph once  | O(V + E)
4. Reverse Result Array  | Swap V elements      | O(V)
-------------------------------------------------------
Total Time Complexity: O(V + E)
Even with the outer loop, every node is only explored
exactly once.

Space Complexity Derivation:
-------------------------------------------------------
Task                     | Memory Used          | Big O
-------------------------|----------------------|--------
1. Adjacency List        | Stores V and E       | O(V + E)
```

```
2. State Array           | Size V               | O(V)
3. Result Array          | Size V               | O(V)
4. DFS Call Stack        | Recursion depth      | O(V)
-------------------------------------------------------
Total Space Complexity: O(V + E)
The graph representation takes the most memory. The
recursion stack takes O(V) in the worst-case scenario
(a single straight line of dependencies).
```

---

7. Variations and Common Tweaks

Variation A: Validating a Tree Structure **The Problem:** You are given an undirected graph and asked if it forms a valid tree. A valid tree must be fully connected and contain exactly zero cycles. **The Tweak:** You use a standard DFS with a cycle check, but because the graph is undirected, you don't need the 3-State map. You just need a standard `visited` set and you must track the `parent` node that called the current node so you don't falsely identify the immediate backward path as a cycle.

Snippet for Undirected Cycle Check:

```javascript
function hasCycle(node, parent, visited, adjList) {
    visited.add(node);

    for (let neighbor of adjList.get(node)) {
        if (!visited.has(neighbor)) {
            if (hasCycle(neighbor, node, visited, adjList)) return true;
        } else if (neighbor !== parent) {
            // We hit a visited node that isn't the node we just came from!
            return true;
        }
    }
    return false;
}
```

Variation B: Finding All Possible Topological Sorts (Backtracking) **The Problem:** An interviewer asks you to return *every single valid combination* of task schedules, not just one. **The Tweak:** You cannot use Kahn's Algorithm or standard DFS here. You must use **Backtracking**. You maintain an In-Degree array. In a recursive function, you loop through all nodes. If a node has an In-Degree of 0 and isn't visited, you add it to your temporary path, decrease the In-Degrees of its neighbors, and recurse. Then, you *undo* all those actions (backtrack) to try the next available node.

Snippet for All Topological Sorts:

```
function findAllSorts(path) {
    // Base Case: We used all nodes, record the path!
    if (path.length === numNodes) {
        allResults.push([...path]);
        return;
    }

    for (let i = 0; i < numNodes; i++) {
        if (inDegree[i] === 0 && !visited[i]) {
            // Choose
            visited[i] = true;
            path.push(i);
            for (let neighbor of adjList.get(i)) inDegree[neighbor]--;

            // Explore
            findAllSorts(path);

            // Un-Choose (Backtrack)
            for (let neighbor of adjList.get(i)) inDegree[neighbor]++;
            path.pop();
            visited[i] = false;
        }
    }
}
```

More Applications of Post-Order DFS

It is a fantastic idea to double down on Post-Order DFS. Once you master the "bottom-up" mindset, an entire category of "Hard" tree problems suddenly becomes incredibly predictable.

Whenever a problem implies that a parent node is completely helpless until its children report back their status, you should immediately reach for Post-Order DFS.

Here are four excellent LeetCode problems where Post-Order DFS is the undisputed secret weapon, ranging from standard to advanced.

---

1. Diameter of Binary Tree (LeetCode 543)

The Problem: Find the length of the longest path between any two nodes in a tree. The tricky part? The longest path might not even pass through the absolute root of the tree!

Why Post-Order is the Secret Weapon: A node cannot know the longest

path going through it until it knows exactly how deep its left and right subtrees go.

1. The Left child reports its max depth.
2. The Right child reports its max depth.
3. **The Parent's Action:** It calculates the local diameter (`Left Depth + Right Depth`) and updates a global maximum if it's the biggest seen so far. Finally, it returns `max(Left, Right) + 1` up to *its* parent.

```
Walkthrough of a local calculation:
       [A]
      /   \
    [B]   [C]  <-- C reports a depth of 1
    /
  [D]          <-- D reports a depth of 1 (so B's left depth is 2)
  /
[E]

When we process [A], its Left side reports depth 3. Its Right reports 1.
Local Diameter at [A] = 3 + 1 = 4.
[A] reports depth 4 up to its parent.
```

The Tweak:

```
let maxDiameter = 0;

function postOrder(node) {
    if (node === null) return 0;

    let leftDepth = postOrder(node.left);
    let rightDepth = postOrder(node.right);

    // Process Self: Update global max if this local 'bridge' is bigger
    maxDiameter = Math.max(maxDiameter, leftDepth + rightDepth);

    // Return the deepest single path back up to the parent
    return Math.max(leftDepth, rightDepth) + 1;
}
```

---

2. House Robber III (LeetCode 337)

The Problem: You are a thief robbing houses arranged in a binary tree. If you rob a parent node, the security system triggers if you also rob its direct children. Find the maximum amount of money you can steal.

Why Post-Order is the Secret Weapon: This is "Dynamic Programming on a Tree". A parent node needs to make a choice: "Do I rob myself, or do I skip

myself?" It cannot make this choice without knowing exactly how much money its children would bring in under both scenarios. Every child must return an array of two numbers to its parent: `[Max money if skipped, Max money if robbed]`.

The Tweak:

```
function robDFS(node) {
    if (node === null) return [0, 0]; // [skip, rob]

    let left = robDFS(node.left);
    let right = robDFS(node.right);

    // Process Self:
    // Option 1: We ROB this node. We MUST SKIP the children.
    let robThis = node.val + left[0] + right[0];

    // Option 2: We SKIP this node. We take the best possible outcome from the children.
    let skipThis = Math.max(left[0], left[1]) + Math.max(right[0], right[1]);

    // Return both options up to the parent
    return [skipThis, robThis];
}
```

---

3. Delete Nodes And Return Forest (LeetCode 1110)

The Problem: You are given a tree and a list of values to delete. When you delete a node, its children become the roots of new, independent trees (a "forest"). Return a list of all the new roots.

Why Post-Order is the Secret Weapon: Imagine deleting a folder on your desktop. If you use Pre-Order (Top-Down) and delete the parent folder immediately, you instantly lose the memory pointers to all the files inside it! You must use Post-Order to dive all the way to the bottom, process the children, and *then* look at the parent to see if it needs to be deleted.

The Tweak:

```
function deleteDFS(node) {
    if (node === null) return null;

    // 1 & 2. Clean up children first!
    node.left = deleteDFS(node.left);
    node.right = deleteDFS(node.right);

    // 3. Process Self: Are we on the hit list?
    if (toDeleteSet.has(node.val)) {
```

```
        // If we have surviving children, they are now independent roots!
        if (node.left) forest.push(node.left);
        if (node.right) forest.push(node.right);

        // Delete ourselves by returning null to OUR parent
        return null;
    }

    // If not deleted, return ourselves normally
    return node;
}
```

---

4. Distribute Coins in Binary Tree (LeetCode 979)

The Problem: You have a tree with N nodes and exactly N coins total, but they are distributed unevenly (some nodes have 3 coins, some have 0). In one move, you can slide one coin across one edge. How many total moves does it take to give every node exactly 1 coin?

Why Post-Order is the Secret Weapon: A node cannot balance its own checkbook until its children are balanced. We work bottom-up. A leaf node looks at its coins. If it has 0, it tells its parent: "I need 1 coin (Debt = -1)". If it has 3 coins, it keeps 1 for itself and tells its parent: "Here are 2 extra coins (Excess = +2)". Every time a coin (or a request for a coin) moves across an edge, that equals 1 move!

```
Walkthrough:
        [0] (Root)
      /   \
    [3]   [0]
```

1. Left child `[3]` keeps 1, passes `+2` excess up. (2 moves required).
2. Right child `[0]` needs 1, passes `-1` debt up. (1 move required).
3. Root `[0]` receives +2 and -1. It keeps 1 for itself and the tree is balanced. Total moves = 3.

The Tweak:

```
let totalMoves = 0;

function coinDFS(node) {
    if (node === null) return 0;

    let leftBalance = coinDFS(node.left);
    let rightBalance = coinDFS(node.right);

    // Every time balance moves up an edge (positive or negative), it costs a move!
```

```
    totalMoves = totalMoves + Math.abs(leftBalance) + Math.abs(rightBalance);

    // Process Self: Calculate what we are passing up to OUR parent
    // (Our coins) + (what kids gave us) - (1 coin we must keep)
    return node.val + leftBalance + rightBalance - 1;
}
```

BFS

Here is a complete guide to understanding and implementing the Breadth-First Search (BFS) algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are playing a video game and you drop a stone into a calm pond. The ripples spread outward uniformly in all directions. If you want to find the absolute closest piece of land to where the stone dropped, how would you search? If you use Depth-First Search (DFS), you might swim in one single direction for a mile before realizing there was an island just 10 feet behind you.

Why we need it: Breadth-First Search (BFS) explores data like that water ripple. It looks at everything 1 step away, then everything 2 steps away, then 3 steps away. We strictly need BFS whenever a problem asks for the **"Shortest Path"** or **"Minimum Number of Steps"** in an unweighted graph or grid. DFS cannot guarantee the shortest path; BFS guarantees it by design.

---

2. What is it?

Breadth-First Search is a traversal algorithm that explores data level-by-level.

Its core philosophy is: **Explore all immediate neighbors before moving on to the neighbors of those neighbors.**

It achieves this by using a **Queue**. A queue is a First-In, First-Out (FIFO) data structure, exactly like a line at a grocery store. The first node you discover gets to the front of the line, and you must finish processing the front of the line before looking at anyone newly added to the back.

---

3. How does it work? (Walkthrough)

Let's look at two scenarios: a simple tree and a cyclic graph.

Scenario A: The Binary Tree (Level Order Traversal) Trees have no cycles, so we don't need to worry about infinite loops.

```
      [A]          <-- Level 0
     /   \
   [B]   [C]       <-- Level 1
   / \     \
 [D] [E]   [F]     <-- Level 2
```

Walkthrough:

1. **Start:** Put the root `[A]` into the Queue. (Queue: `[A]`)
2. **Pop A:** Remove `A` from the front of the queue. Process it.

- Add its children `[B, C]` to the back of the queue. (Queue: `[B, C]`)

3. **Pop B:** Remove `B` from the front. Process it.

- Add its children `[D, E]` to the back. (Queue: `[C, D, E]`)
- *Notice:* `C` is still at the front of the line! We are finishing Level 1 before touching Level 2.

4. **Pop C:** Remove `C` from the front. Process it.

- Add its child `[F]` to the back. (Queue: `[D, E, F]`)

5. **Pop D, E, F:** None of them have children, so we just pop and process them one by one until the Queue is empty.

Order Visited: A, B, C, D, E, F. (Perfectly level by level!)

Scenario B: The Graph with a Cycle (The Shortest Path) Graphs can loop. We want to find the shortest path from **Node 1** to **Node 4**. To prevent infinite loops, just like in DFS, we must use a `visited` set.

```
[1] --- [2]
 |       |
[3] --- [4]
```

Walkthrough:

1. **Start:** Put `1` in the Queue. Mark `1` as visited. (Queue: `[1]`, Visited: `{1}`)
2. **Pop 1:** Neighbors of 1 are `2` and `3`.

- Neither are visited. Add them to Queue. Mark them visited.
- (Queue: `[2, 3]`, Visited: `{1, 2, 3}`)

3. **Pop 2:** Neighbors of 2 are `1` and `4`.

- `1` is already visited. Ignore it.
- `4` is not visited! Add it to Queue. Mark it visited.
- (Queue: `[3, 4]`, Visited: `{1, 2, 3, 4}`)

4. **Pop 3:** Neighbors of 3 are `1` and `4`. Both are already visited! Do nothing.

- (Queue: `[4]`)

5. **Pop 4:** We found our target! We reached it in 2 steps.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Explores a graph using Breadth-First Search
 * @param {Object} graph - An adjacency list representing the graph
 * @param {string} startNode - The node to begin the search
 */
function bfs(graph, startNode) {
    // 1. Create the Queue and add the starting node
    const queue = [startNode];

    // 2. Create the notepad (Set) to prevent infinite loops, mark start as visited
    const visited = new Set();
    visited.add(startNode);

    const resultOrder = [];

    // 3. Loop until the queue is completely empty
    while (queue.length > 0) {
        // Pop the FIRST person in line (First-In, First-Out)
        // Note: shift() is O(N) in JS. In strict interviews, you might
        // use an index pointer 'head' to simulate popping in O(1) time.
        const currentNode = queue.shift();

        // Process the node
        resultOrder.push(currentNode);

        // 4. Look at all immediate neighbors
        const neighbors = graph[currentNode];
        for (let i = 0; i < neighbors.length; i++) {
            const nextNode = neighbors[i];

            // If we haven't seen this neighbor yet, put them in line!
            if (!visited.has(nextNode)) {
                visited.add(nextNode);   // Mark visited IMMEDIATELY when entering queue
                queue.push(nextNode);    // Add to the back of the line
            }
        }
    }
```

```
    return resultOrder;
}

// --- Usage ---
const myGraph = {
    '1': ['2', '3'],
    '2': ['1', '4'],
    '3': ['1', '4'],
    '4': ['2', '3']
};

console.log(bfs(myGraph, '1'));
// Output: [ '1', '2', '3', '4' ]
```

---

5. LeetCode questions it applies to

- **Binary Tree Level Order Traversal (LeetCode 102):** The classic introduction to BFS. You return an array of arrays, where each inner array contains the values of one specific level of the tree. You use the "Level Tracking" tweak (explained below).
- **Rotting Oranges (LeetCode 994):** You have a grid with fresh and rotten oranges. Every minute, rotten oranges infect adjacent fresh ones. You use a "Multi-Source BFS" where you put *all* initially rotten oranges into the queue at the start, and track how many minutes (levels) it takes to infect everything.
- **Word Ladder (LeetCode 127):** A famous hard problem. Find the shortest transformation sequence from a start word to an end word, changing only one letter at a time. Every word is a node, and edges exist between words that differ by one letter. Since it asks for the *shortest sequence*, it screams BFS!

---

6. Time and Space Complexity

Let V equal the number of Vertices (Nodes). Let E equal the number of Edges (Connections).

```
Time Complexity Derivation:
-------------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Add/Remove from Queue  | 1 time per Vertex    | O(V)
2. Check Neighbors        | 1 time per Edge      | O(E)
-------------------------------------------------------------
```

```
Total Time Complexity: O(V + E)
(Every node is processed once, and every connection is looked at once).

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Visited Set            | Stores up to V items | O(V)
2. Queue                  | Stores up to V items | O(V)
-----------------------------------------------------------
Total Space Complexity: O(V) + O(V) = O(V)
*Note: The Queue gets largest at the widest level of the graph.
For a perfectly balanced binary tree, the bottom level holds
N/2 nodes, meaning the Queue space is O(V/2), which drops the constant to O(V).
```

---

7. Variations and Common Tweaks

Variation A: Tracking Levels / Distance (The "Inner Loop" Tweak)
The Problem: Standard BFS just processes nodes one by one. It doesn't know *when* Level 1 ends and Level 2 begins. You need this to find the "Number of steps" or to return nodes grouped by level. **The Tweak:** Take a "snapshot" of the queue's length at the start of the `while` loop. That length is exactly how many nodes are on the current level. Use a `for` loop to process exactly that many nodes before moving on.

Snippet for Level Tracking:

```javascript
let distance = 0;

while (queue.length > 0) {
    // TWEAK: Take a snapshot of the current level's size
    let levelSize = queue.length;

    // Process ONLY the nodes that were on this level
    for (let i = 0; i < levelSize; i++) {
        const current = queue.shift();

        if (current === TARGET) return distance;

        // ... add unvisited neighbors to queue ...
    }

    // Once the inner loop finishes, a whole level is done!
    distance++;
}
```

Variation B: Multi-Source BFS (Rotting Oranges / Walls and Gates) **The Problem:** Instead of a single starting point (like dropping one stone in the pond), you have multiple starting points at the exact same time (like dropping 5 stones at once). **The Tweak:** Before the `while` loop begins, instead of pushing just `startNode` into the queue, you iterate through your grid and push *every single* starting point into the queue. They will all expand outward simultaneously, level by level.

Snippet for Multi-Source BFS:

```
const queue = [];
const visited = new Set();

// 1. Find ALL starting points and load the queue
for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
        if (grid[r][c] === ROTTEN) {
            queue.push([r, c]);
            visited.add(r + "," + c);
        }
    }
}

// 2. Proceed with the Level-Tracking BFS loop!
// (The ripples will spread from all sources at the same time)
```

Variation C: Bi-Directional BFS **The Problem:** You want to find the shortest path between a Start node and an End node, but the graph is massively huge. **The Tweak:** Instead of expanding a giant circle from the Start node until it hits the End node, you start one BFS from the Start node, and a *second* BFS from the End node. They expand toward each other. When their `visited` sets intersect, you've found the shortest path! This cuts the search space drastically.

Multi-Source BFS

Here is a complete guide to understanding and mastering the Multi-Source Breadth-First Search algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine a city with three fire stations and a fire breaks out at a random house. You want to know the shortest distance from *any* fire station to that house. Or, conversely, a forest fire starts in five different locations at the exact same time, and you need to simulate how long it takes for the entire forest to burn.

Why the "Naive" Approach fails: If you use standard BFS, you would have to start a separate search from Fire Station A, then completely reset and start a new search from Fire Station B, and then again from Fire Station C. If your map has N total locations, and you have S starting points (sources), running BFS for every single source takes O(S * N) time. If there are 10,000 sources on a map of 1,000,000 cells, your computer will freeze trying to calculate all 10,000,000,000 operations.

Why we need Multi-Source BFS: This algorithm completely eliminates the need for multiple runs. By slightly tweaking how we start our BFS, we can simulate all starting points expanding simultaneously. This brings the time complexity down to a beautiful, highly optimal **O(N)** time—a single pass through the map!

---

2. What is it?

Multi-Source BFS is the exact same engine as standard Breadth-First Search (it uses a Queue to process nodes level-by-level).

The only difference is the **Setup Phase**. In a standard BFS, you put exactly one starting node into your Queue before starting your `while` loop. In Multi-Source BFS, you scan your entire grid/graph first, find *every single* starting point, and push ALL of them into the Queue at distance 0.

Think of dropping a handful of pebbles into a pond all at once. The ripples spread outward at the exact same speed. When a ripple hits a target, it is mathematically guaranteed to be the shortest path from *one* of the pebbles, and you don't even care which one!

---

3. How does it work? (Walkthrough)

Let's walk through the classic "Rotting Oranges" grid scenario.

- `2` = Rotten Orange (Our Starting Sources)
- `1` = Fresh Orange (Our Targets)
- `0` = Empty Space (Walls we cannot cross)

Input Grid:

```
Column:  0  1  2
       ---------
Row 0: | 2  1  1 |
Row 1: | 1  1  0 |
Row 2: | 0  1  2 |
```

Step 1: The Setup Scan Before we expand, we loop through the entire grid to find our sources and count our targets.

- We find a `2` at (Row 0, Col 0). Push to Queue!
- We find a `2` at (Row 2, Col 2). Push to Queue!
- Total Fresh Oranges (`1`s) found = 5.

```
Initial Queue: [ (0,0), (2,2) ]
Fresh Count: 5
Minute: 0
```

Step 2: The Multi-Source Expansion (Minute 1) We process the queue level-by-level. Currently, there are 2 items in our level.

- **Pop (0,0):** Look Up, Down, Left, Right.
- Right is (0,1). It's fresh! Make it rotten. Add to next Queue. Fresh Count = 4.
- Down is (1,0). It's fresh! Make it rotten. Add to next Queue. Fresh Count = 3.
- **Pop (2,2):** Look Up, Down, Left, Right.
- Up is (1,2). It's an empty space (0). Ignore.
- Left is (2,1). It's fresh! Make it rotten. Add to next Queue. Fresh Count = 2.

```
Grid After Minute 1:   Queue: [ (0,1), (1,0), (2,1) ]
| 2  2  1 |            Fresh Count: 2
| 2  1  0 |            Minute: 1
| 0  2  2 |
```

(Notice how the rot expanded from BOTH corners simultaneously!)

Step 3: The Multi-Source Expansion (Minute 2) There are 3 items in our current level queue.

- **Pop (0,1):** Right is (0,2). It's fresh! Make it rotten. Add to next Queue. Fresh Count = 1.
- **Pop (1,0):** Right is (1,1). It's fresh! Make it rotten. Add to next Queue. Fresh Count = 0.
- **Pop (2,1):** Up is (1,1). We already rotted it! Ignore.

```
Grid After Minute 2:   Queue: [ (0,2), (1,1) ]
| 2  2  2 |            Fresh Count: 0
| 2  2  0 |            Minute: 2
| 0  2  2 |
```

Step 4: Finish up Our Fresh Count is 0. We successfully rotted every orange on the grid. It took exactly 2 minutes!

---

4. Code Sample (JavaScript)

```javascript
/**
 * Calculates the minimum minutes to rot all oranges.
 * @param {number[][]} grid
 * @return {number} - minutes taken, or -1 if impossible
 */
function orangesRotting(grid) {
    const rows = grid.length;
    const cols = grid[0].length;
    const queue = [];

    let freshOranges = 0;
    let minutes = 0;

    // 1. SETUP PHASE: Scan the grid to find all sources and count targets
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (grid[r][c] === 2) {
                // It's rotten! Add to our starting queue
                queue.push([r, c]);
            } else if (grid[r][c] === 1) {
                // It's fresh! Track how many we need to infect
                freshOranges++;
            }
        }
    }

    // Edge case: If there are no fresh oranges, it takes 0 minutes
    if (freshOranges === 0) return 0;

    // Standard directions array for moving Up, Down, Left, Right
    const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];

    // 2. BFS EXPANSION PHASE
    while (queue.length > 0 && freshOranges > 0) {
        // Take a snapshot of the current level's size
        let levelSize = queue.length;

        // Process ONLY the oranges that rotted in the previous minute
        for (let i = 0; i < levelSize; i++) {
```

```
            // Remove front of queue
            const [currentRow, currentCol] = queue.shift();

            // Look in all 4 directions
            for (let [dr, dc] of directions) {
                const newRow = currentRow + dr;
                const newCol = currentCol + dc;

                // Boundary check AND check if it is a fresh orange
                if (
                    newRow >= 0 && newRow < rows &&
                    newCol >= 0 && newCol < cols &&
                    grid[newRow][newCol] === 1
                ) {
                    // Infect it!
                    grid[newRow][newCol] = 2; // Mark as rotten (acts as our visited set)
                    freshOranges--;           // One less fresh orange in the world
                    queue.push([newRow, newCol]); // Add to queue for the NEXT minute
                }
            }
        }

        // After fully processing the current level, one minute has passed
        minutes++;
    }

    // 3. FINAL CHECK
    // If we finished our BFS but there are still fresh oranges left,
    // it means some oranges were blocked by walls and can never be reached.
    if (freshOranges > 0) {
        return -1;
    }

    return minutes;
}
```

---

5. LeetCode questions it applies to

- **Rotting Oranges (LeetCode 994):** The exact algorithm described above. Find all 2s, put them in a queue, count the 1s, and expand outwards until the 1s hit 0.
- **01 Matrix (LeetCode 542):** You are given a grid of 0s and 1s. You need to find the distance to the nearest 0 for every cell. **The Trick:** Treat ALL `0`s as your sources! Push every `0` into the queue, and let them expand

outwards to fill in the distances for the 1s.

- **Walls and Gates (LeetCode 286):** You have a grid with Rooms, Walls, and Gates. You must fill each Room with its distance to the nearest Gate. Treat all Gates as your starting sources, push them into the queue, and expand outwards into the empty Rooms.
- **As Far from Land as Possible (LeetCode 1162):** You have a grid of land (1) and water (0). Find a water cell such that its distance to the nearest land cell is maximized. Treat all land cells (1s) as your sources and expand into the water. The very last water cell you reach is your answer!

---

6. Time and Space Complexity

Let R equal the number of Rows. Let C equal the number of Columns. Total grid cells = R * C.

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Initial Grid Scan      | Visit every cell     | O(R * C)
2. Add/Remove from Queue  | 1 time per cell      | O(R * C)
3. Check 4 Neighbors      | 4 times per cell     | O(R * C)
-----------------------------------------------------------
Total Time: We visit every cell a constant number of times.
Time Complexity is exactly O(R * C).

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Queue Storage          | Up to R * C items    | O(R * C)
-----------------------------------------------------------
Total Space: O(R * C)
*Note: In the worst-case scenario where the entire grid is
filled with Rotten Oranges, we push all (R * C) cells into
the queue right at the start.
```

---

7. Variations and Common Tweaks

Variation A: The Reverse Viewpoint (01 Matrix) **The Problem:** The question asks "Find the shortest distance from every Node X to *any* Node Y". If there are many X's and few Y's, doing a BFS from every X is too slow. **The Tweak:** Reverse your perspective! Instead of trying to find the path from X

to Y, push all the Y's into the queue and let them branch outwards to find the X's. In the `01 Matrix` problem, you push all `0`s into the queue initially, and as you expand into the `1`s, the distance traveled is recorded directly into the grid.

Snippet for Reverse Viewpoint Update:

```
// Inside the neighbor check loop:
if (grid[newRow][newCol] === 1) {
    // The distance to this '1' is simply the distance of the cell
    // we just came from + 1 step!
    grid[newRow][newCol] = grid[currentRow][currentCol] + 1;
    queue.push([newRow, newCol]);
}
```

Variation B: Unreachable Targets Validation **The Problem:** What if a fresh orange is completely surrounded by walls (0s)? The rotting ripple will never reach it. **The Tweak:** As shown in the main code block, you don't just rely on the Queue becoming empty. You must maintain a `targetCount` variable. You count all targets during the initial Setup Phase. You decrement it every time you infect/reach a target. At the very end of the function, if `targetCount > 0`, you return `-1` (impossible).

Variation C: Graph Multi-Source (Non-Grid) **The Problem:** Multi-Source BFS isn't strictly for 2D grids. You might have an adjacency list graph and need to find the shortest path from a group of starting nodes. **The Tweak:** The logic is identical. Instead of looping through `rows` and `cols`, you loop through your given array of `startingNodes`, push them all into the Queue, and add them all to a `visited` Set before starting the `while(queue.length > 0)` loop.

DFS and BFS on Trees vs Graphs

Here is a complete guide to understanding the crucial differences between using DFS and BFS on Trees versus Graphs.

---

1. Problem it solves and Why we need it

The Problem: You have a collection of data points (nodes) connected to each other. You need to explore them to find a specific target, calculate distances, or just read all the data.

Why we need it: While DFS (Depth-First Search) and BFS (Breadth-First Search) are the two undisputed kings of traversal, **you cannot blindly use the exact same code for a Tree and a Graph.** * **Trees** are clean, organized, and flow in one direction (top to bottom).

- **Graphs** are chaotic. They can loop back on themselves (cycles), and some parts of the graph might be completely disconnected from others.

If you use a Tree traversal algorithm on a Graph, your program will likely get stuck in an infinite loop and crash. Understanding the difference allows you to safely navigate any data structure an interviewer throws at you.

---

2. What is it?

Both DFS and BFS are exploration strategies:

- **DFS (The Maze Runner):** Dives as deep as possible down one path using a **Stack** (or recursion). Good for finding dead ends or fully exploring branches.
- **BFS (The Water Ripple):** Expands equally in all directions level-by-level using a **Queue**. Good for finding the shortest path.

The "Tree vs. Graph" Difference: A Tree is actually just a special, very strict type of Graph.

1. **The Root Rule:** A tree has exactly one starting point (the Root). A graph can start anywhere.
2. **The Cycle Rule:** A tree never flows backwards. If you go down, you can't accidentally end up where you started. Graphs can have endless loops (cycles).

Because of the Cycle Rule, **Graph algorithms must use a "Visited" memory (usually a Set) to remember where they have been.** Tree algorithms don't need this extra memory.

---

3. How does it work? (Walkthrough)

Let's look at how DFS and BFS behave differently depending on the structure they are exploring.

Scenario A: Exploring a Tree (No Visited Set Needed)

```
      [1]
     /   \
   [2]   [3]
   /       \
 [4]       [5]
```

Tree DFS (Left, then Right):

1. Start at `1`. Dive left to `2`.
2. Dive left to `4`. Dead end! Backtrack to `2`.

3. 2 has no right child. Backtrack to 1.
4. Dive right to 3. Dive right to 5. *Order:* 1, 2, 4, 3, 5

Tree BFS (Level by Level):

1. Look at Level 0: 1.
2. Look at Level 1: 2, 3.
3. Look at Level 2: 4, 5. *Order:* 1, 2, 3, 4, 5

Scenario B: Exploring a Graph (The Cycle Trap)

```
[1] ----- [2]
 |         |
[3] ----- [4]
```

Notice how the nodes form a square.

Graph DFS without a Visited Set (DISASTER!):

1. Start at 1. Go to 2.
2. From 2, go to 4.
3. From 4, go to 3.
4. From 3, go to 1.
5. From 1, go to 2… INFINITE LOOP!

Graph DFS WITH a Visited Set (The Fix):

1. Start at 1. Mark 1 as visited. (Set: {1})
2. Go to 2. Mark 2 as visited. (Set: {1, 2})
3. Go to 4. Mark 4 as visited. (Set: {1, 2, 4})
4. Go to 3. Mark 3 as visited. (Set: {1, 2, 4, 3})
5. From 3, we look at neighbors: 1 and 4.
6. *Crucial Check:* Both 1 and 4 are in our Visited Set! We ignore them. Dead end reached safely. Backtrack!

---

4. Code Sample (JavaScript)

Here is a side-by-side comparison showing how Tree code and Graph code differ for BFS.

```javascript
// ==========================================
// 1. TREE BFS (Clean and Simple)
// ==========================================
function treeBFS(root) {
    if (root === null) return [];

    const queue = [root];
    const result = [];
```

```
    while (queue.length > 0) {
        const current = queue.shift();
        result.push(current.val);

        // No visited set needed! Just push children if they exist.
        if (current.left !== null) queue.push(current.left);
        if (current.right !== null) queue.push(current.right);
    }
    return result;
}

// ==========================================
// 2. GRAPH BFS (Requires Visited Set)
// ==========================================
function graphBFS(graph, startNode) {
    const queue = [startNode];
    const result = [];

    // THE DIFFERENCE: We MUST track where we have been
    const visited = new Set();
    visited.add(startNode);

    while (queue.length > 0) {
        const current = queue.shift();
        result.push(current);

        // Loop through all neighbors
        for (let neighbor of graph[current]) {
            // THE DIFFERENCE: Only add to queue if NOT visited
            if (!visited.has(neighbor)) {
                visited.add(neighbor);
                queue.push(neighbor);
            }
        }
    }
    return result;
}
```

---

5. LeetCode questions it applies to

- **Tree DFS -> Lowest Common Ancestor (LeetCode 236):** You traverse down a tree. Because there are no cycles, you just ask the left child and right child if they found the target, and bubble the answer back up.

- **Tree BFS -> Binary Tree Right Side View (LeetCode 199):** You do a level-by-level BFS. For each level, you only add the very last node processed to your result array.
- **Graph DFS -> Clone Graph (LeetCode 133):** You must copy a graph. You use a Hashmap as your `visited` set. If you visit a node you've already cloned, you just wire up the connection to the existing clone instead of getting stuck in a loop.
- **Graph BFS -> Shortest Path in Binary Matrix (LeetCode 1091):** You are finding a path in a 2D grid (which is just a graph where each cell connects to 8 neighbors). You use BFS to guarantee the shortest path, marking grid cells as 'visited' as you step on them so you don't walk backwards.

---

6. Time and Space Complexity

Trees use **N** (Total Nodes). Graphs use **V** (Vertices/Nodes) and **E** (Edges/Connections).

```
Tree Complexity Derivation:
-----------------------------------------------------------
Time: O(N)
We visit every node in the tree exactly once.

Space (DFS): O(H)
Where H is the height of the tree. The recursion stack goes
as deep as the tree is tall.

Space (BFS): O(W)
Where W is the maximum width of the tree. The queue holds
one entire level at a time.
-----------------------------------------------------------

Graph Complexity Derivation:
-----------------------------------------------------------
Time: O(V + E)
We visit every Vertex exactly once, and we loop through
every Edge (neighbor connection) exactly once.

Space (DFS & BFS): O(V)
Our Visited Set will eventually store all V vertices.
Our Queue/Stack can also hold up to V vertices.
Total space is O(V) + O(V) = O(V).
-----------------------------------------------------------
```

---

7. Variations and Common Tweaks

Variation A: The Disconnected Graph (The Outer Loop Tweak) **The Problem:** In a Tree, starting at the root guarantees you can reach everything. In a Graph, there might be "islands" of nodes that don't connect to each other. If you just call `graphDFS(node1)`, you will miss the disconnected islands! **The Tweak:** You wrap your DFS or BFS inside an outer loop that iterates over *every single node* in the graph. You only trigger the search if the node isn't already in the `visited` set.

Snippet for Disconnected Graphs:

```
const visited = new Set();
let disconnectedComponents = 0;

// Loop over EVERY node in the entire graph
for (let node in graph) {
    // Only launch a search if we haven't found this node yet
    if (!visited.has(node)) {
        disconnectedComponents++; // We found a new island!

        // Launch DFS or BFS to explore this entire island and
        // add all its connected nodes to the visited set.
        exploreIslandDFS(graph, node, visited);
    }
}
```

Variation B: Cycle Detection in an Undirected Graph **The Problem:** You want to know if a graph has a loop in it. **The Tweak:** You use Graph DFS with a `visited` set. However, in an undirected graph, A connects to B, and B connects to A. When you are at B, looking back at A doesn't count as a cycle—it's just the path you came from! You must pass a `parent` variable in your DFS. If you see a node that is already `visited`, AND that node is NOT your `parent`, you have found a true cycle.

Snippet for Cycle Detection:

```
function hasCycle(node, parent, visited, graph) {
    visited.add(node);

    for (let neighbor of graph[node]) {
        if (!visited.has(neighbor)) {
            // Dive deep, telling the neighbor that WE are its parent
            if (hasCycle(neighbor, node, visited, graph)) return true;
        } else if (neighbor !== parent) {
            // We hit a visited node that isn't the guy who just called us!
            // This means we circled back around. Cycle detected!
            return true;
```

```
        }
    }
    return false;
}
```

Inorder Traversal

Here is a complete guide to understanding and implementing the Inorder Traversal algorithm.

---

1. Problem it solves and Why we need it

The Problem: You have a Binary Tree, and you need to read all the data inside of it. But trees aren't flat like arrays; they branch out. If you have a specific type of tree called a **Binary Search Tree (BST)**—where smaller numbers are on the left and larger numbers are on the right—how do you extract the numbers so they come out in perfectly sorted, ascending order?

Why we need it: If you use Pre-Order DFS (Root, Left, Right), you get a jumbled mess of numbers. If you use Post-Order DFS (Left, Right, Root), you get a different jumbled mess.

We need Inorder Traversal because it is the mathematically perfect way to "flatten" a Binary Search Tree back into a sorted, straight line. It ensures we visit the smallest elements first, then the middle elements, and finally the largest elements.

---

2. What is it?

Inorder Traversal is a specific flavor of Depth-First Search (DFS) designed for binary trees.

The name "Inorder" tells you exactly when you process the current node: *in the middle* of processing its children. The strict, unbreakable rule of Inorder traversal is:

1. **Go Left:** Recursively visit the entire left subtree (find the smallest stuff).
2. **Process Node (Self):** Look at the current node's value and do your work.
3. **Go Right:** Recursively visit the entire right subtree (find the larger stuff).

You can remember it as: **Left -> Root -> Right**.

---

3. How does it work? (Walkthrough)

Let's walk through two scenarios to see how the computer's Call Stack naturally sorts the numbers.

Scenario A: The Standard Binary Search Tree Input Tree:

```
        [4]
       /   \
     [2]   [6]
     / \     \
   [1] [3]   [7]
```

Walkthrough:

1. Start at **Root** [**4**]. Rule says: Go Left. (4 is put on hold).
2. Arrive at [**2**]. Rule says: Go Left. (2 is put on hold).
3. Arrive at [**1**]. Rule says: Go Left. (It's null).

- *Action:* Left is done. Process Node! We print **1**.
- Rule says: Go Right. (It's null).
- [1] is entirely finished. We backtrack up to [2].

4. Back at [**2**]. Left subtree is done!

- *Action:* Process Node! We print **2**.
- Rule says: Go Right.

5. Arrive at [**3**]. Go Left (null).

- *Action:* Process Node! We print **3**.
- Go Right (null). Backtrack up to [2].

6. Back at [**2**]. Left is done, Self is done, Right is done. Backtrack up to [4].
7. Back at [**4**]. Its entire Left subtree is finally done!

- *Action:* Process Node! We print **4**.
- Rule says: Go Right.

8. Arrive at [**6**]. Go Left (null).

- *Action:* Process Node! We print **6**.
- Rule says: Go Right.

9. Arrive at [**7**]. Go Left (null).

- *Action:* Process Node! We print **7**.
- Go Right (null). Backtrack to [6], backtrack to [4]. Done!

Final Inorder: 1, 2, 3, 4, 6, 7. *(Look at that magic! The tree is now perfectly sorted).*

Scenario B: The "Lopsided" Tree What happens if the tree isn't a perfect BST, but just a strange, left-heavy tree? The rules never change.

```
        [A]
       /
     [B]
     / \
   [C] [D]
```

Walkthrough:

1. Start at [**A**]. Go Left.
2. Arrive at [**B**]. Go Left.
3. Arrive at [**C**]. Go Left (null). Print **C**. Go Right (null). Backtrack to [B].
4. Back at [**B**]. Left is done. Print **B**. Go Right.
5. Arrive at [**D**]. Go Left (null). Print **D**. Go Right (null). Backtrack to [B], then [A].
6. Back at [**A**]. Left is done. Print **A**. Go Right (null).

Final Inorder: C, B, D, A.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 * this.val = (val===undefined ? 0 : val)
 * this.left = (left===undefined ? null : left)
 * this.right = (right===undefined ? null : right)
 * }
 */

/**
 * Standard Recursive Inorder Traversal
 * @param {TreeNode} root
 * @return {number[]}
 */
function inorderTraversal(root) {
    const result = [];

    function traverse(node) {
        // Base case: If we hit a dead end, turn back
        if (node === null) {
            return;
        }
```

```
        // 1. Go completely down the Left side
        traverse(node.left);

        // 2. Process the current Node (Self)
        result.push(node.val);

        // 3. Go completely down the Right side
        traverse(node.right);
    }

    // Kick off the recursion from the root
    traverse(root);

    return result;
}
```

---

5. LeetCode questions it applies to

- **Binary Tree Inorder Traversal (LeetCode 94):** The direct implementation of this algorithm. You can use the recursive code above, but they will likely ask you to write the Iterative version (see Variations below).
- **Validate Binary Search Tree (LeetCode 98):** You need to check if a tree is a valid BST. If you do an Inorder Traversal, the numbers MUST come out in strictly increasing order. You just keep track of the `previousNodeValue`. If the `currentNodeValue` is ever less than or equal to the `previousNodeValue`, you immediately return false!
- **Kth Smallest Element in a BST (LeetCode 230):** You need to find the 3rd smallest number in a BST. Because Inorder Traversal visits nodes from smallest to largest, you just keep a counter. The moment you process a node, you increment your counter. When `counter === K`, you have found your answer and can stop traversing.

---

6. Time and Space Complexity

Let N equal the total number of nodes in the tree. Let H equal the height of the tree.

```
Time Complexity Derivation:
-------------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Visit every Node       | 1 call per node      | O(N)
2. Process Node Data      | 1 push to array      | O(1)
-------------------------------------------------------------
```

```
Total Time Complexity: O(N)
We visit every node exactly once.

Space Complexity Derivation:
-------------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Recursion Call Stack   | Varies by Tree Shape | O(H)
-------------------------------------------------------------
Total Space Complexity: O(H)
*Note: If the tree is perfectly balanced, the height H is
log2(N), making space O(log2(N)). If the tree is a straight
line (worst case), the height H is N, making space O(N).
```

---

7. Variations and Common Tweaks

Variation A: Iterative Inorder Traversal (The Stack Tweak) The Problem: In interviews, after you write the recursive version, the interviewer will almost always say: "Great. Now do it iteratively without using recursion." **The Tweak:** You have to manually simulate the computer's Call Stack. You use a `while` loop and an explicit array to act as your stack. You push nodes to the stack as you travel down the left side. When you hit a null, you pop from the stack, process the node, and move to its right child.

Snippet for Iterative Inorder:

```javascript
function iterativeInorder(root) {
    const result = [];
    const stack = [];
    let current = root;

    // Loop until we've explored everything and the stack is empty
    while (current !== null || stack.length > 0) {

        // 1. Reach the left most Node of the current Node
        while (current !== null) {
            stack.push(current);
            current = current.left;
        }

        // 2. Current must be null at this point. Backtrack!
        current = stack.pop();

        // Process the node
        result.push(current.val);
```

```
        // 3. We have visited the node and its left subtree. Now, it's right subtree's turn
        current = current.right;
    }

    return result;
}
```

Variation B: Reverse Inorder Traversal (Descending Order) The Problem: The interviewer asks you to find the "Kth *Largest* Element in a BST". Or, they ask you to convert a BST into a "Greater Tree" (where every key is changed to the original key plus the sum of all keys greater than it). **The Tweak:** You simply flip the order of the recursive calls! Instead of Left -> Root -> Right, you go **Right -> Root -> Left**. This forces the algorithm to visit the absolute largest number first, then the second largest, all the way down to the smallest.

Snippet for Reverse Inorder:

```
let runningSum = 0;

function reverseInorder(node) {
    if (node === null) return;

    // 1. Go RIGHT first (find the biggest numbers)
    reverseInorder(node.right);

    // 2. Process Self
    runningSum = runningSum + node.val;
    node.val = runningSum; // Update the node for the "Greater Tree" problem

    // 3. Go LEFT last
    reverseInorder(node.left);
}
```

Preorder Traversal

Here is a complete guide to understanding and implementing the Preorder Traversal algorithm.

---

1. Problem it solves and Why we need it

The Problem: You have a Binary Tree and you need to visit every node. But unlike Post-Order (where you process children first) or Inorder (where you

process the left, then the middle), you have a scenario where you absolutely must process the **Parent node before you even look at its children**.

Why we need it: Preorder traversal is the ultimate "Top-Down" approach. We strictly need it for two major real-world tasks:

1. **Copying / Cloning a Tree:** If you want to build an exact replica of a tree, you must create the root node first before you can attach any children to it.
2. **Serializing a Tree:** If you want to save a tree to a text file or send it across the internet, you need to flatten it into a string. Preorder is perfect for this because when you read the string back later to rebuild the tree, the very first value you read is guaranteed to be the absolute root!

---

2. What is it?

Preorder Traversal is a specific flavor of Depth-First Search (DFS) designed for binary trees.

The name "Pre-Order" tells you exactly when you process the current node: *before* its children. The strict, unbreakable rule of Preorder traversal is:

1. **Process Node (Self):** Look at the current node's value and do your work (print it, copy it, save it).
2. **Go Left:** Recursively visit the entire left subtree.
3. **Go Right:** Recursively visit the entire right subtree.

You can remember it as: **Root -> Left -> Right**.

---

3. How does it work? (Walkthrough)

Let's walk through two scenarios to see exactly how processing the root first changes the flow of the algorithm.

Scenario A: The Standard Binary Tree Input Tree:

```
        [A]
       /   \
     [B]   [C]
     / \     \
   [D] [E]   [F]
```

Walkthrough:

1. Start at **Root** [**A**]. Rule says: Process Node.

- *Action:* We print **A**.

- Rule says: Go Left.

2. Arrive at [**B**]. Rule says: Process Node.

- *Action:* We print **B**.
- Rule says: Go Left.

3. Arrive at [**D**]. Rule says: Process Node.

- *Action:* We print **D**.
- Rule says: Go Left (null). Go Right (null). Backtrack up to [B].

4. Back at [**B**]. Left subtree is done. Rule says: Go Right.
5. Arrive at [**E**]. Rule says: Process Node.

- *Action:* We print **E**.
- Go Left (null). Go Right (null). Backtrack up to [B], then backtrack up to [A].

6. Back at [**A**]. Left subtree is entirely done! Rule says: Go Right.
7. Arrive at [**C**]. Rule says: Process Node.

- *Action:* We print **C**.
- Rule says: Go Left (null). Go Right.

8. Arrive at [**F**]. Rule says: Process Node.

- *Action:* We print **F**.
- Go Left (null). Go Right (null). Backtrack to [C], backtrack to [A]. Done!

Final Preorder: A, B, D, E, C, F. *(Notice how the Root 'A' is the absolute first thing printed, and 'B' is printed before any of its children!)*

Scenario B: The "Missing Branches" Tree What happens if the tree has missing left or right branches? The algorithm just hits the `null` dead ends and immediately moves to the next step in its 1-2-3 rule list.

```
    [1]
   /
[2]
   \
    [3]
```

Walkthrough:

1. Start at [**1**]. Process Self: Print **1**. Go Left.
2. Arrive at [**2**]. Process Self: Print **2**. Go Left.
3. Arrive at `null` (left of 2). Turn back! Go Right.
4. Arrive at [**3**]. Process Self: Print **3**. Go Left (null). Go Right (null).
5. Backtrack to [2], backtrack to [1].
6. Back at [**1**]. Go Right.
7. Arrive at `null` (right of 1). Turn back! Done.

Final Preorder: 1, 2, 3.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 * this.val = (val===undefined ? 0 : val)
 * this.left = (left===undefined ? null : left)
 * this.right = (right===undefined ? null : right)
 * }
 */

/**
 * Standard Recursive Preorder Traversal
 * @param {TreeNode} root
 * @return {number[]}
 */
function preorderTraversal(root) {
    const result = [];

    function traverse(node) {
        // Base case: If we hit a dead end, turn back
        if (node === null) {
            return;
        }

        // 1. Process the current Node (Self) FIRST!
        result.push(node.val);

        // 2. Go completely down the Left side
        traverse(node.left);

        // 3. Go completely down the Right side
        traverse(node.right);
    }

    // Kick off the recursion from the root
    traverse(root);

    return result;
}
```

---

5. LeetCode questions it applies to

- **Binary Tree Preorder Traversal (LeetCode 144):** The direct implementation. Just like Inorder, interviewers will often ask you to write the Iterative version of this after you finish the recursive one.
- **Serialize and Deserialize Binary Tree (LeetCode 297):** A famous "Hard" problem. You flatten the tree into a string using Preorder (e.g., "1,2,null,null,3"). When you deserialize it, you read the string left-to-right. Because it's Preorder, the first number is guaranteed to be your root, making it incredibly easy to rebuild the tree top-down.
- **Construct Binary Tree from Preorder and Inorder Traversal (LeetCode 105):** You are given two arrays. You look at the very first element of the Preorder array to instantly know what the Root of the tree is. Then, you find that root in the Inorder array, which magically tells you exactly how many nodes belong in the left subtree versus the right subtree!
- **Invert Binary Tree (LeetCode 226):** The problem famously failed by the creator of Homebrew. You process the current node by swapping its left and right child pointers, and *then* you recursively travel down the left and right sides to keep swapping.

---

6. Time and Space Complexity

Let N equal the total number of nodes in the tree. Let H equal the height of the tree.

```
Time Complexity Derivation:
-------------------------------------------------------
Task                     | Operations           | Big O
-------------------------|----------------------|--------
1. Visit every Node      | 1 call per node      | O(N)
2. Process Node Data     | 1 push to array      | O(1)
-------------------------------------------------------
Total Time Complexity: O(N)
We visit every node exactly once.

Space Complexity Derivation:
-------------------------------------------------------
Task                     | Memory Used          | Big O
-------------------------|----------------------|--------
1. Recursion Call Stack  | Varies by Tree Shape | O(H)
-------------------------------------------------------
Total Space Complexity: O(H)
*Note: If the tree is perfectly balanced, the height H is
log2(N). If the tree is a straight line, H is N, making
```

space O(N).

---

7. Variations and Common Tweaks

Variation A: Iterative Preorder Traversal (The Stack Tweak) **The Problem:** You must write this algorithm without using system recursion to avoid stack overflow errors on massively deep trees. **The Tweak:** Unlike Iterative Inorder (which is quite tricky), Iterative Preorder is surprisingly straightforward. You use an array as your stack. The only "gotcha" is that because a stack is Last-In, First-Out (LIFO), you must push the **Right child first**, and the **Left child second**. This guarantees the Left child gets popped off and processed before the Right child!

Snippet for Iterative Preorder:

```
function iterativePreorder(root) {
    if (root === null) return [];

    const result = [];
    const stack = [root]; // Start by pushing the root

    while (stack.length > 0) {
        // 1. Pop the top of the stack and process it immediately
        const current = stack.pop();
        result.push(current.val);

        // 2. Push RIGHT child first (so it stays at the bottom)
        if (current.right !== null) {
            stack.push(current.right);
        }

        // 3. Push LEFT child second (so it ends up on top and gets processed next!)
        if (current.left !== null) {
            stack.push(current.left);
        }
    }

    return result;
}
```

Variation B: Tree Serialization (The Null Marker Tweak) **The Problem:** If you just output the values of a tree, you lose its structural shape. The trees `[1, 2]` (left child) and `[1, null, 2]` (right child) will both output as just "1, 2". **The Tweak:** When you hit a `null` dead end, you must explicitly record it in your string/array (e.g., push an "X" or "null"). This strictly preserves the

shape of the tree so it can be rebuilt perfectly.

Snippet for Serialization:

```
function serializePreorder(node, result) {
    if (node === null) {
        result.push("null"); // TWEAK: Explicitly record the dead end
        return;
    }

    // Process Self
    result.push(node.val.toString());

    // Go Left, then Right
    serializePreorder(node.left, result);
    serializePreorder(node.right, result);
}
```

Topological Sort

Here is a complete guide to understanding and implementing the Topological Sort algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are trying to get a computer science degree. You want to take "Machine Learning", but the university says you must pass "Calculus" and "Statistics" first. But wait, to take "Statistics", you must first pass "Basic Math".

When you have a massive list of tasks where certain tasks **must be completed before others**, how do you figure out the correct order to do them in?

Why we need it: If you just pick tasks randomly, you will constantly hit roadblocks ("I can't build the roof, I haven't poured the foundation yet!").

Topological Sort is an algorithm that takes a messy web of dependencies and flattens them out into a single, perfectly ordered straight line. If you follow that line from left to right, you are mathematically guaranteed to never hit a roadblock. It is the backbone of task scheduling, software build systems (like npm or Webpack), and spreadsheet formula evaluations.

---

2. What is it?

Topological Sort is an algorithm designed strictly for **Directed Acyclic Graphs (DAGs)**.

- **Directed:** The connections have arrows (A points to B). It means A is a prerequisite for B.
- **Acyclic:** There are no cycles (loops). If A requires B, and B requires A, it is impossible to finish either task! Topological sort only works if the graph has no loops.

The Output: A linear list of nodes where, for every directed edge from Node U to Node V, Node U appears *before* Node V in the list.

---

3. How does it work? (Walkthrough)

There are two ways to do this (DFS and BFS). The most intuitive way is the BFS-based approach called **Kahn's Algorithm**.

Kahn's Algorithm relies on a concept called **In-Degree**.

- **In-Degree** is simply the number of arrows pointing *in* to a node.
- In plain English: It is the number of prerequisites a task has left to complete.
- If a task has an In-Degree of `0`, it means it has no prerequisites! You can do it right now.

Scenario A: The College Schedule (Standard DAG) Let's say we have 4 classes:

- Class 0: No prerequisites.
- Class 1: Requires Class 0.
- Class 2: Requires Class 0.
- Class 3: Requires Class 1 AND Class 2.

```
Graph View:
       [0]
      /   \
    [1]   [2]
      \   /
       [3]
```

Step 1: Build the Adjacency List and In-Degree Table We count how many arrows point into each node.

```
Node | Prerequisites (In-Degree) | Leads To (Neighbors)
-------------------------------------------------------
  0  |            0              | [1, 2]
  1  |            1              | [3]
```

```
2  |            1              | [3]
3  |            2              | []
```

Step 2: Find all nodes with 0 In-Degree and put them in a Queue
Only Node 0 has 0 prerequisites.

- **Queue:** `[0]`
- **Result Array:** `[]`

Step 3: Process the Queue until empty

- **Pop 0:** Add it to our Result.
- Node 0 is done! We can now cross it off the prerequisite lists for its neighbors (Node 1 and Node 2).
- Node 1 In-Degree drops from 1 to 0. (It's free! Add to Queue).
- Node 2 In-Degree drops from 1 to 0. (It's free! Add to Queue).
- **Queue:** `[1, 2]`
- **Result:** `[0]`
- **Pop 1:** Add it to Result.
- Node 1 is done! Its neighbor is Node 3.
- Node 3 In-Degree drops from 2 to 1. (Still not 0, so we don't queue it yet. It still waiting on Node 2).
- **Queue:** `[2]`
- **Result:** `[0, 1]`
- **Pop 2:** Add it to Result.
- Node 2 is done! Its neighbor is Node 3.
- Node 3 In-Degree drops from 1 to 0. (It's free! Add to Queue).
- **Queue:** `[3]`
- **Result:** `[0, 1, 2]`
- **Pop 3:** Add it to Result.
- Node 3 has no neighbors.
- **Queue:** `[]` (Empty! We are done).

Final Answer: `[0, 1, 2, 3]` (or `[0, 2, 1, 3]`, both are valid topological sorts!)

Scenario B: The Impossible Schedule (Cycle Detection) What if Class A requires Class B, and Class B requires Class A?

```
Graph: [A] <---> [B]

In-Degree Table:
A: 1 (waiting on B)
B: 1 (waiting on A)
```

Walkthrough:

1. We look for nodes with an In-Degree of 0 to put in our Queue.
2. There are none! Both are stuck at 1.
3. The Queue starts empty. The `while` loop never runs.
4. Our Result array is empty.

The Cycle Check Rule: If the length of our final Result array does NOT equal the total number of nodes in our graph, it means some nodes got permanently stuck. **We have detected a cycle!**

---

4. Code Sample (JavaScript)

```javascript
/**
 * Topological Sort using Kahn's Algorithm (BFS)
 * @param {number} numNodes - Total number of tasks/nodes
 * @param {number[][]} edges - Array of [prerequisite, task] pairs
 * @return {number[]} - Valid order, or empty array if a cycle exists
 */
function topologicalSort(numNodes, edges) {
    // 1. Initialize Adjacency List and In-Degree array
    const adjList = new Map();
    const inDegree = new Array(numNodes).fill(0);

    for (let i = 0; i < numNodes; i++) {
        adjList.set(i, []);
    }

    // 2. Build the Graph and populate In-Degrees
    for (let i = 0; i < edges.length; i++) {
        const prereq = edges[i][0];
        const task = edges[i][1];

        adjList.get(prereq).push(task); // prereq points TO task
        inDegree[task]++;               // task has one more prerequisite
    }
```

```
    // 3. Find all nodes with 0 prerequisites to start our queue
    const queue = [];
    for (let i = 0; i < numNodes; i++) {
        if (inDegree[i] === 0) {
            queue.push(i);
        }
    }

    const result = [];

    // 4. Process the Queue
    while (queue.length > 0) {
        // Take a task that has no remaining prerequisites
        const currentTask = queue.shift();
        result.push(currentTask);

        // Look at all the tasks that were waiting on this currentTask
        const neighbors = adjList.get(currentTask);
        for (let i = 0; i < neighbors.length; i++) {
            const nextTask = neighbors[i];

            // "Cross off" the prerequisite
            inDegree[nextTask]--;

            // If the next task now has 0 prerequisites, it is ready to be processed!
            if (inDegree[nextTask] === 0) {
                queue.push(nextTask);
            }
        }
    }

    // 5. Cycle Detection Check
    // If we didn't process every node, it means there was a cycle (a deadlock).
    if (result.length !== numNodes) {
        return []; // Impossible to finish!
    }

    return result;
}

// --- Usage ---
// 4 nodes (0, 1, 2, 3). Edges: 0->1, 0->2, 1->3, 2->3
const numClasses = 4;
const prerequisites = [[0, 1], [0, 2], [1, 3], [2, 3]];

console.log(topologicalSort(numClasses, prerequisites));
```

```
// Output: [ 0, 1, 2, 3 ]
```

---

5. LeetCode questions it applies to

- **Course Schedule (LeetCode 207):** You are given a list of courses and their prerequisites. You just need to return `true` if it's possible to finish all courses, or `false` if it's impossible. You run Kahn's Algorithm and simply return `result.length === numCourses`.
- **Course Schedule II (LeetCode 210):** Exactly the same as Course Schedule I, but instead of returning a boolean, you must return the actual `result` array. (The code provided in Section 4 is the exact solution for this).
- **Alien Dictionary (LeetCode 269):** A famous hard problem. You are given a sorted dictionary of an alien language. You must deduce the alphabetical order of their letters. You compare adjacent words to figure out which letter comes before which (e.g., if "art" comes before "bat", then 'a' must come before 'b'). You build a graph from these rules and run a Topological Sort to get the alphabet!

---

6. Time and Space Complexity

Let V equal the number of Vertices (Nodes/Tasks). Let E equal the number of Edges (Dependencies/Prerequisites).

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Build Graph/InDegree   | Loop over all Edges  | O(E)
2. Find 0 InDegree nodes  | Loop over all Nodes  | O(V)
3. Process Queue          | Visit each Node once | O(V)
4. Reduce InDegrees       | Check each Edge once | O(E)
-----------------------------------------------------------
Total Time Complexity: O(V + E)
We look at every task and every dependency exactly once.

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Adjacency List         | Stores V keys, E vals| O(V + E)
2. In-Degree Array        | Stores V items       | O(V)
3. Queue                  | Stores up to V items | O(V)
```

```
-------------------------------------------------------
Total Space Complexity: O(V + E)
```

---

7. Variations and Common Tweaks

Variation A: Topological Sort using DFS (The Recursive Way) **The Problem:** Kahn's Algorithm (BFS) is great, but sometimes a problem specifically requires a Depth-First Search approach, or you are already traversing a graph recursively and want to build the topological sort simultaneously. **The Tweak:** You use a standard Graph DFS. The magic trick is that you **only add a node to your result array AFTER all of its neighbors have been fully explored.** Because we add nodes at the very end of their exploration, the result array will be built backwards. We simply reverse the array at the end!

Snippet for DFS Topological Sort:

```javascript
const visited = new Set();
const result = [];

function dfs(node) {
    visited.add(node);

    const neighbors = adjList.get(node);
    for (let i = 0; i < neighbors.length; i++) {
        const nextNode = neighbors[i];
        if (!visited.has(nextNode)) {
            dfs(nextNode);
        }
    }

    // TWEAK: Only add to result AFTER all children are processed (Post-Order)
    result.push(node);
}

// Run DFS on all unvisited nodes...
// Then at the very end:
return result.reverse();
```

(Note: To handle cycle detection in DFS, you need a second set called `visiting` *or* `path` *to track the nodes currently in your recursive call stack. If you hit a node in the* `visiting` *set, you've found a cycle).*

Variation B: Lexicographically Smallest Topological Sort **The Problem:** Sometimes there are multiple valid ways to schedule tasks. For example, if Task 1 and Task 2 both have 0 prerequisites, you could do either one first.

The problem might ask: "If there is a tie, always pick the task with the smaller ID/Alphabetical letter first." **The Tweak:** In Kahn's algorithm, we use a standard array as our Queue. A standard queue just pops whatever entered first. To fix this, we replace the Queue with a **Min-Priority Queue (Min-Heap)**.

Snippet for Min-Heap Tweak:

```
// Instead of const queue = [];
const minHeap = new MinPriorityQueue();

// When finding 0 in-degree nodes:
if (inDegree[i] === 0) {
    minHeap.enqueue(i); // Enqueues based on the smallest value
}

// Inside the while loop:
// We are now guaranteed to always process the smallest available task first!
const currentTask = minHeap.dequeue().element;
```

Disjoint Set Union (DSU) (Union Find)

Here is a complete guide to understanding and implementing the Disjoint Set Union (DSU) algorithm, also famously known as Union-Find.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are building a social network. You have millions of users, and they form friend groups. You need a system that can instantly answer two questions:

1. "Are User A and User B in the same friend group?"
2. "User A and User C just became friends. How do we merge their entire friend groups together?"

Why the "Naive" Approaches fail:

- **Adjacency List + DFS/BFS:** You could build a graph. To check if A and B are connected, you run DFS from A to see if you hit B. But what if the graph is huge? DFS takes O(V + E) time. If you have to answer 100,000 queries, running DFS every single time is far too slow.
- **Array of Group IDs:** You could give everyone a Group ID (e.g., `group[A] = 1`, `group[B] = 1`). Checking if they are in the same group is instant O(1). But what happens when you merge Group 1 and Group 2? You have to loop through millions of users and update all the '2's to '1's. This takes O(N) time per merge, which is also too slow.

Why we need DSU: DSU is a specialized data structure that solves this exact "dynamic connectivity" problem. It can both check if two items are connected **and** merge two massive groups together in practically **O(1)** time.

---

2. What is it?

Disjoint Set Union manages a collection of elements partitioned into non-overlapping (disjoint) subsets.

It operates on a single brilliant concept: **The Representative (or Root).** Instead of updating everyone's group ID when two groups merge, we organize each group like a family tree. One specific node is chosen as the "Root" (the absolute boss) of that group.

- If you want to know which group you belong to, you just follow the chain of parents up to the Root.
- If two nodes have the *same* Root, they are in the same group!

It has two primary operations:

1. **Find:** Follow the parent pointers to find the Root of an element.
2. **Union:** Find the Roots of two elements. If they are different, make one Root the parent of the other Root. This instantly merges the two entire trees!

---

3. How does it work? (Walkthrough)

To make DSU blazing fast, we must use two critical optimizations: **Path Compression** and **Union by Rank/Size**. Without them, DSU can degrade to O(N) time.

- **Path Compression (Optimizes Find):** When you call `Find(X)` and walk up the tree to the Root, you take a second to update `X` (and every node you passed along the way) to point *directly* to the Root. The next time you call `Find(X)`, it takes exactly 1 step!
- **Union by Rank (Optimizes Union):** When merging two trees, you don't want to attach a massive tree to the bottom of a tiny tree (which would make the tree taller and slower to climb). You always attach the shorter tree's root to the taller tree's root.

Walkthrough: Merging Friend Groups Let's start with 5 isolated users: 1, 2, 3, 4, 5. We represent this with a `parent` array. Initially, everyone is their own boss. `parent = [0, 1, 2, 3, 4, 5]` (Index 0 is ignored for simplicity).

Step 1: Union(1, 2)

- Root of 1 is 1. Root of 2 is 2.

- They are the same size, so just make 1 the parent of 2.

```
Array: parent = [0, 1, 1, 3, 4, 5]

Tree View:
   [1]        [3]   [4]   [5]
    |
   [2]
```

Step 2: Union(3, 4)

- Root of 3 is 3. Root of 4 is 4.
- Make 3 the parent of 4.

```
Array: parent = [0, 1, 1, 3, 3, 5]

Tree View:
   [1]        [3]         [5]
    |          |
   [2]        [4]
```

Step 3: Union(2, 5)

- Root of 2 is 1. Root of 5 is 5.
- Tree 1 is taller (contains 1 and 2). Tree 5 is just one node.
- *Union by Rank:* Make 1 the parent of 5.

```
Array: parent = [0, 1, 1, 3, 3, 1]

Tree View:
      [1]            [3]
     /   \            |
   [2]   [5]         [4]
```

Step 4: Union(5, 4)

- Find(5): Parent of 5 is 1. Root is 1.
- Find(4): Parent of 4 is 3. Root is 3.
- They have different roots, so we merge! Both trees have a height of 2. Let's make 1 the parent of 3.

```
Array: parent = [0, 1, 1, 1, 3, 1]

Tree View:
          [1]
        /  |  \
      [2] [5] [3]
               |
              [4]
```

Step 5: Path Compression in Action! Now, what if we call `Find(4)`? We look at 4. Its parent is 3. 3's parent is 1. Root is 1! Before we return 1, **Path Compression** kicks in. We say: "Why is 4 pointing to 3? Let's wire it directly to 1 to save time later!" We update `parent[4] = 1`.

```
Tree View after Find(4):
         [1]
       / / | \
     [2][5][3][4]  <-- The tree is incredibly flat!
```

---

4. Code Sample (JavaScript)

```javascript
/**
 * Disjoint Set Union (Union-Find) class
 */
class DSU {
    constructor(n) {
        // parent[i] points to the parent of node i
        this.parent = new Array(n);
        // rank[i] tracks the depth/size of the tree to keep it flat
        this.rank = new Array(n).fill(1);

        // Initially, every node is its own parent (its own root)
        for (let i = 0; i < n; i++) {
            this.parent[i] = i;
        }
    }

    /**
     * Finds the root of the given element 'x'
     * Includes Path Compression optimization!
     */
    find(x) {
        // If x is not its own parent, it is not the root.
        if (this.parent[x] !== x) {
            // Recursively find the root, and point x DIRECTLY to it.
            // This flattens the tree.
            this.parent[x] = this.find(this.parent[x]);
        }
        return this.parent[x];
    }

    /**
     * Unites the groups containing x and y.
     * Includes Union by Rank optimization!
```

```
     * @return {boolean} - Returns true if a merge happened, false if already in same group
     */
    union(x, y) {
        let rootX = this.find(x);
        let rootY = this.find(y);

        // They already have the same root. No merge needed.
        if (rootX === rootY) {
            return false;
        }

        // Union by Rank: Attach the smaller tree under the taller tree
        if (this.rank[rootX] > this.rank[rootY]) {
            this.parent[rootY] = rootX;
        } else if (this.rank[rootX] < this.rank[rootY]) {
            this.parent[rootX] = rootY;
        } else {
            // If they are the same height, pick one to be the root
            // and increase its rank by 1
            this.parent[rootY] = rootX;
            this.rank[rootX] += 1;
        }

        return true;
    }

    /**
     * Checks if x and y belong to the same group
     */
    isConnected(x, y) {
        return this.find(x) === this.find(y);
    }
}

// --- Usage ---
const dsu = new DSU(6); // 6 nodes: 0 through 5
dsu.union(1, 2);
dsu.union(3, 4);
dsu.union(2, 5);

console.log(dsu.isConnected(1, 5)); // Output: true
console.log(dsu.isConnected(1, 4)); // Output: false
```

---

5. LeetCode questions it applies to

- **Number of Provinces (LeetCode 547):** You are given a matrix of connected cities. You initialize a DSU with N cities. Loop through the matrix, and whenever city i is connected to city j, you call `dsu.union(i, j)`. At the end, you just count how many unique roots exist in the graph (i.e., how many nodes have `parent[i] === i`).
- **Redundant Connection (LeetCode 684):** A tree is an undirected graph with no cycles. You are given a tree that has one extra edge added to it, creating a cycle. You iterate through the edges calling `dsu.union(u, v)`. The very first time `dsu.union(u, v)` returns `false` (meaning `u` and `v` already share a root), that specific edge is the redundant one that formed the cycle!
- **Accounts Merge (LeetCode 721):** Users have multiple emails. If two accounts share an email, they belong to the same person. You use DSU to `union` the first email of an account with the rest of the emails in that account. After merging all intersecting accounts, you group the emails by their DSU root.

---

6. Time and Space Complexity

The time complexity of DSU uses a very special mathematical function called the **Inverse Ackermann function**, usually written as `alpha(N)`. For all practical values of N (even numbers larger than the atoms in the universe), `alpha(N)` is strictly less than 5. Therefore, we consider the operations to be **nearly O(1)** or "Amortized Constant Time".

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Initialize DSU         | Loop N times         | O(N)
2. Find (with comp.)      | Climb flattened tree | O(alpha(N)) ~ O(1)
3. Union (with rank)      | Two Finds + 1 link   | O(alpha(N)) ~ O(1)
4. isConnected            | Two Finds            | O(alpha(N)) ~ O(1)
-----------------------------------------------------------
Total Time: Building takes O(N). Queries are practically O(1).

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Parent Array           | Stores N items       | O(N)
2. Rank Array             | Stores N items       | O(N)
-----------------------------------------------------------
```

```
Total Space Complexity: O(N)
```

---

7. Variations and Common Tweaks

Variation A: Tracking Group Sizes (Max Area of Island) **The Problem:** Standard DSU merges groups, but it doesn't easily tell you *how many items* are in that merged group. If a problem asks "What is the size of the largest connected component?", you need a tweak. **The Tweak:** Instead of tracking the `rank` (the height of the tree), you track the `size` (total number of nodes in the tree). When you merge Root Y into Root X, you add the size of Y to the size of X.

Snippet for Tracking Sizes:

```
// In constructor:
this.size = new Array(n).fill(1); // Every group starts with size 1

// Inside union(x, y):
let rootX = this.find(x);
let rootY = this.find(y);

if (rootX !== rootY) {
    // Always attach the smaller sized group to the larger sized group
    if (this.size[rootX] >= this.size[rootY]) {
        this.parent[rootY] = rootX;
        this.size[rootX] += this.size[rootY]; // TWEAK: Update size!
    } else {
        this.parent[rootX] = rootY;
        this.size[rootY] += this.size[rootX]; // TWEAK: Update size!
    }
}
```

Variation B: String IDs instead of Number Indices **The Problem:** In problems like "Accounts Merge", the nodes aren't numbered 0 to N. The nodes are strings (Email addresses). You can't use an array like `parent = new Array(n)`. **The Tweak:** Swap out the Arrays for Hash Maps (Objects or `Map` in JS).

Snippet for String DSU:

```
class StringDSU {
    constructor() {
        this.parent = new Map();
    }

    // TWEAK: Automatically initialize a node the first time we see it
```

```
    find(x) {
        if (!this.parent.has(x)) {
            this.parent.set(x, x);
        }

        if (this.parent.get(x) !== x) {
            this.parent.set(x, this.find(this.parent.get(x)));
        }
        return this.parent.get(x);
    }
    // ... union remains mostly the same
}
```

Djikstra's Algorithm (SSSP)

Here is a complete guide to understanding and implementing Dijkstra's Algorithm for the Single Source Shortest Path (SSSP) problem.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are a GPS navigation system. A user wants to drive from their House (Node A) to the Airport (Node Z). There are many different routes, but every road has a different speed limit, traffic level, and physical length. You need to find the absolute fastest route.

Why we need it (when we already have BFS): Breadth-First Search (BFS) is fantastic for finding the shortest path, but it has one massive blind spot: **It assumes every single road takes the exact same amount of time to travel.** In a standard BFS, a 100-mile highway and a 1-mile dirt road are both treated as "1 step".

If your graph has **weights** (costs, distances, or times associated with moving from one node to another), BFS will confidently give you the wrong answer. Dijkstra's Algorithm was invented to solve this exact problem. It reliably finds the cheapest, fastest, or shortest path in a weighted graph.

The only catch: Dijkstra's Algorithm completely breaks if there are **negative weights** (e.g., a road that magically gives you back 5 minutes of time). For negative weights, you would need the Bellman-Ford algorithm.

---

2. What is it? (Pseudocode & Flow)

Dijkstra's Algorithm is essentially BFS on steroids. Instead of a standard First-In-First-Out (FIFO) Queue, it uses a **Min-Priority Queue** (Min-Heap).

The core philosophy is **Greedy Exploration**: "Always explore the absolute cheapest known path next."

Here is the language-agnostic pseudocode logic:

```
DIJKSTRA'S ALGORITHM WORKFLOW:

[SETUP]
1. Create a 'distances' map/array. Set distance to Start Node = 0.
2. Set distance to all other Nodes = Infinity.
3. Create a Min-Priority Queue (PQ).
4. Insert (Start Node, Total Cost: 0) into PQ.

[MAIN LOOP]
While PQ is not empty:
   |
   |-- Pop the node with the SMALLEST Total Cost from PQ.
   |   Let's call it 'CurrentNode'.
   |
   |-- [CHECK] Is the cost to reach 'CurrentNode' strictly greater
   |           than the best distance we already recorded in our
   |           'distances' map?
   |           --> If YES: We found a better path earlier.
   |                       Throw this one away! (Continue loop)
   |
   |-- For each 'Neighbor' of 'CurrentNode':
         |
         |-- Calculate newCost = Cost to reach CurrentNode + Edge Weight to Neighbor
         |
         |-- [CHECK] Is newCost < 'distances[Neighbor]'?
               |
               |--> If YES: We found a faster route to this Neighbor!
                            1. Update 'distances[Neighbor]' = newCost
                            2. Push (Neighbor, newCost) into PQ.
               |
               |--> If NO: Ignore it. Our old route was better.

[END]
Return the 'distances' map!
```

---

3. How does it work? (Walkthrough)

Let's trace a scenario where the "shortest" path in terms of steps is actually the "longest" path in terms of weight.

Scenario: The Greedy Trap We want to go from **A** to **C**.

- Path 1: A directly to C (Weight: 10)
- Path 2: A to B (Weight: 2), then B to C (Weight: 1)

```
Graph Layout:
      (10)
  A --------> C
   \         /
 (2)\       /(1)
     v     v
        B
```

Walkthrough:

Step 0: Setup

- Distances: `{ A: 0, B: Infinity, C: Infinity }`
- Priority Queue: `[ (Node: A, Cost: 0) ]`

Step 1: Pop A

- Pop the cheapest item from PQ: **A** (Cost: 0).
- Look at A's neighbors: B and C.
- **Neighbor C:** New cost is $0 + 10 = 10$.
- Is $10 <$ Infinity? YES!
- Update Distances: `{ A: 0, B: Infinity, C: 10 }`
- Add to PQ: `(Node: C, Cost: 10)`
- **Neighbor B:** New cost is $0 + 2 = 2$.
- Is $2 <$ Infinity? YES!
- Update Distances: `{ A: 0, B: 2, C: 10 }`
- Add to PQ: `(Node: B, Cost: 2)`

Current PQ State: `[ (B, 2), (C, 10) ]` *(Notice B is at the front because 2 is smaller than 10!)*

Step 2: Pop B

- Pop the cheapest item from PQ: **B** (Cost: 2).
- Look at B's neighbor: C.
- **Neighbor C:** New cost is Cost to reach B (which is 2) + Edge Weight to C (which is 1) = 3.
- Is 3 < Distances[C] (which is 10)? **YES!** We found a massive shortcut!
- Update Distances: `{ A: 0, B: 2, C: 3 }`
- Add to PQ: `(Node: C, Cost: 3)`

Current PQ State: `[ (C, 3), (C, 10) ]` *(Notice C is in there twice! That's okay, our check will handle it).*

Step 3: Pop C (The shortcut)

- Pop the cheapest item from PQ: **C** (Cost: 3).
- C has no neighbors. Loop continues.

Step 4: Pop C (The old, slow path)

- Pop the next item: **C** (Cost: 10).
- *The crucial check:* Is this cost (10) strictly greater than our currently recorded Distances[C] (which is 3)? YES. This is an obsolete, slow path. We immediately throw it away and do nothing.

Result: The loop finishes. Our final shortest distances from A are `{ A: 0, B: 2, C: 3 }`.

---

4. Code Sample (JavaScript)

Note: JavaScript does not have a built-in Priority Queue. For interview purposes, you can often use a simple array and sort it, but for a true O(E log V) implementation, you need a Min-Heap. I will simulate the Min-Heap here with an array `.sort()` *for brevity, but I will explicitly mark where the true Heap operations occur.*

```javascript
/**
 * Dijkstra's Algorithm for Single Source Shortest Path
 * @param {Object} graph - Adjacency list: { node: [{target: 'B', weight: 2}] }
 * @param {string} startNode - The node to begin calculating distances from
 * @return {Object} - A map of the shortest distances to every node
 */
function dijkstra(graph, startNode) {
    // 1. Setup the distances map
    const distances = {};
    for (let node in graph) {
        distances[node] = Infinity;
    }
    distances[startNode] = 0;

    // 2. Setup the Priority Queue
    // In a real optimized scenario, this is a MinHeap class.
    // Elements are stored as objects: { node: string, cost: number }
    const pq = [];
    pq.push({ node: startNode, cost: 0 });

    // 3. Main BFS-style loop
```

```
    while (pq.length > 0) {
        // [SIMULATING MIN-HEAP POP]
        // We sort descending and pop the end to get the smallest cost in O(1) time.
        // (A true MinHeap does this in O(log V) time without full sorting).
        pq.sort((a, b) => b.cost - a.cost);
        const current = pq.pop();

        const currentNode = current.node;
        const currentCost = current.cost;

        // [STALE RECORD CHECK]
        // If we found a faster route to this node earlier, discard this old data.
        if (currentCost > distances[currentNode]) {
            continue;
        }

        // 4. Explore neighbors
        const neighbors = graph[currentNode] || [];
        for (let i = 0; i < neighbors.length; i++) {
            const neighbor = neighbors[i].target;
            const weight = neighbors[i].weight;

            // Calculate the total cost to reach this neighbor via currentNode
            const newTotalCost = currentCost + weight;

            // 5. Relaxation Step: Did we find a shorter path?
            if (newTotalCost < distances[neighbor]) {
                // Update the shortest distance
                distances[neighbor] = newTotalCost;

                // Add the neighbor to the priority queue to be explored later
                pq.push({ node: neighbor, cost: newTotalCost });
            }
        }
    }

    return distances;
}

// --- Usage ---
const myGraph = {
    'A': [{ target: 'B', weight: 2 }, { target: 'C', weight: 10 }],
    'B': [{ target: 'C', weight: 1 }],
    'C': []
};
```

```
console.log(dijkstra(myGraph, 'A'));
// Output: { A: 0, B: 2, C: 3 }
```

---

5. LeetCode questions it applies to

- **Network Delay Time (LeetCode 743):** This is the textbook Dijkstra problem. You are given a list of servers and the time it takes for a signal to travel between them. You run standard Dijkstra from the starting server. The answer is simply the *maximum* value in your final `distances` object (the time it takes for the very last server to receive the signal).
- **Path with Maximum Probability (LeetCode 1514):** A fantastic math variation. Instead of *adding* distances and looking for the *minimum*, you are given probabilities (e.g., 0.5 chance of success). You *multiply* the probabilities together, and you want the *maximum* probability. You switch to a Max-Priority Queue and change the initialization to 0 instead of Infinity.
- **Path With Minimum Effort (LeetCode 1631):** You are traversing a 2D grid of mountain heights. The "cost" of a path is the *maximum absolute height difference* between any two adjacent cells on that path. You want to minimize this maximum effort. Your calculation changes from `currentCost + weight` to `Math.max(currentCost, edgeWeight)`.

---

6. Time and Space Complexity

Let **V** equal the number of Vertices (Nodes). Let **E** equal the number of Edges.

```
Time Complexity Derivation:
-------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Initialize distances   | Loop over V nodes    | O(V)
2. Priority Queue Push    | Once per Edge found  | O(log V) per push
3. Priority Queue Pop     | Once per Edge found  | O(log V) per pop
4. Explore Edges          | Check E edges total  | O(E)
-------------------------------------------------------
Total Time Complexity: O((V + E) * log(V))
*Why log(V)? The Priority Queue can hold up to V nodes at
a time. Inserting or popping from a heap takes logarithmic
time relative to its size. We do this for roughly every edge.

Space Complexity Derivation:
-------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
```

```
1. Distances Map         | Stores V items       | O(V)
2. Priority Queue        | Can hold V items     | O(V)
3. Graph Representation  | V nodes, E edges     | O(V + E)
-------------------------------------------------------
Total Space Complexity: O(V + E)
```

---

7. Variations and Common Tweaks

Variation A: The Early Exit (Point A to Point B) **The Problem:** The standard algorithm calculates the distance from the Start Node to *every single other node* on the map. If you are a GPS and the user just wants to go to the grocery store, calculating the distance to every house in the country is a massive waste of time. **The Tweak:** You pass a `targetNode` parameter. The absolute moment you pop the `targetNode` from the Priority Queue, you are mathematically guaranteed you have found the shortest path to it. You instantly `return distances[targetNode]` and stop the loop.

Snippet for Early Exit:

```
// Inside the while loop:
const current = pq.pop();
const currentNode = current.node;

// TWEAK: Stop searching the moment we pull the target out of the queue!
if (currentNode === targetNode) {
    return current.cost;
}
```

Variation B: Reconstructing the Actual Path **The Problem:** The standard algorithm tells you that the trip will take 3 minutes, but it doesn't tell you *which roads to take* to get there! **The Tweak:** You add a `previous` map. Whenever you find a shorter path to a neighbor, you record *who* gave you that shorter path. At the very end, you trace backwards from your destination using the `previous` map to build the exact turn-by-turn route.

Snippet for Path Reconstruction:

```
const previous = {}; // Tracks the optimal parent for each node

// Inside the relaxation step (when we find a better path):
if (newTotalCost < distances[neighbor]) {
    distances[neighbor] = newTotalCost;

    // TWEAK: Record exactly how we got here!
    previous[neighbor] = currentNode;
```

```
    pq.push({ node: neighbor, cost: newTotalCost });
}

// After the while loop finishes, build the path backwards:
const shortestPath = [];
let step = targetNode;
while (step) {
    shortestPath.push(step);
    step = previous[step]; // Walk backwards
}
return shortestPath.reverse(); // Flip it to Start -> End
```

Variation C: Dijkstra with State (e.g., K Stops Limit) **The Problem:** In LeetCode 787 (Cheapest Flights Within K Stops), you want the cheapest route, but you are not allowed to take more than `K` connecting flights. **The Tweak:** Standard Dijkstra might take a 10-flight route because it's $5 cheaper than a 2-flight route. You must add `stops` to your Priority Queue objects. The "Stale Record Check" gets more complex: you might accept a *more expensive* path if it takes *fewer stops*, because it might be the only valid way to reach the end within the limit!

Bellman-Ford Algorithm (SSSP) for negative weights

Here is a complete guide to understanding and implementing the Bellman-Ford Algorithm.

---

1. Problem it solves and Why we need it

The Problem: You need to find the shortest path from a starting node to all other nodes in a graph. However, this graph contains **negative weights** (e.g., a financial transaction network where some paths represent debt/loss, and others represent profit/gain).

Why we need it (when we already have Dijkstra's): Dijkstra's algorithm is greedy and assumes that adding more edges to a path will *always* increase the total cost.

Imagine you are at Node A.

- Path 1 goes directly to Node B (Cost: 5).
- Path 2 goes to Node C (Cost: 10). Dijkstra will permanently lock in the cost to B as 5, because it assumes going through C (which already costs 10) can only get worse. But what if there is a magical path from C to

B with a cost of **-8**? The true cost to B is A -> C -> B (10 - 8 = 2). Dijkstra completely misses this.

Bellman-Ford solves this by abandoning the greedy approach. Instead of guessing the best path, it brute-forces the problem by repeatedly checking and updating *every single edge* in the graph.

It also solves a critical secondary problem: **Negative Cycles**. If a graph has a loop where the total weight is negative, you could loop around it infinitely to get a cost of negative infinity. Bellman-Ford acts as an alarm system, safely detecting if a negative cycle exists so your program doesn't get trapped in an infinite loop.

---

2. What is it? (Pseudocode & Flow)

The algorithm is based on the principle of **"Relaxation"**. Relaxing an edge simply means asking: *"If I travel through this specific edge, does it give me a shorter path to my destination than the one I currently know?"*

The golden rule of Bellman-Ford: **In a graph with V vertices, the longest possible path without going in circles can only have V - 1 edges.** Therefore, if we relax every edge V - 1 times, we are mathematically guaranteed to find the absolute shortest path.

```
BELLMAN-FORD WORKFLOW:

[SETUP]
1. Let V = number of vertices.
2. Create 'distances' array. Set distance to Start Node = 0.
3. Set distance to all other Nodes = Infinity.

[MAIN LOOP]
Repeat exactly (V - 1) times:
   |
   |-- For EVERY edge in the graph (from U to V with weight W):
         |
         |-- [RELAXATION CHECK]
         |   Is distances[U] + W < distances[V]?
               |
               |--> If YES: We found a cheaper path!
                            distances[V] = distances[U] + W
               |--> If NO: Do nothing.

[CYCLE DETECTION LOOP]
Run ONE more time through every edge:
   |
```

```
|-- Is distances[U] + W < distances[V]?
      |
      |--> If YES: A path just got cheaper even after (V - 1)
                   passes. This is mathematically impossible
                   unless there is a NEGATIVE CYCLE!
                   Throw an Error / Return False.

[END]
Return the 'distances' array!
```

---

3. How does it work? (Walkthrough)

Let's look at two scenarios: a graph that breaks Dijkstra, and a graph with a deadly negative loop.

Scenario A: The Delayed Discount (Dijkstra's Nightmare) We want the shortest paths from **A**.

- Total Vertices (V) = 3.
- We must loop (V - 1) = 2 times.

```
Graph Layout:
      (10)
  A --------> C
   \         /
 (5)\       /(-8)
     v     v
        B
```

Edges List: `[A->B (5), A->C (10), C->B (-8)]`

Step 0: Setup `Distances = { A: 0, B: Infinity, C: Infinity }`

Step 1: Pass 1 of 2 We check every edge in our list:

1. `A->B (5)`: 0 + 5 < Infinity? YES. `Distances[B] = 5`.
2. `A->C (10)`: 0 + 10 < Infinity? YES. `Distances[C] = 10`.
3. `C->B (-8)`: Cost to C is 10. 10 + (-8) = 2. Is 2 < Distances[B] (which is 5)? YES! `Distances[B] = 2`. *End of Pass 1 Distances:* `{ A: 0, B: 2, C: 10 }`

Step 2: Pass 2 of 2 We check every edge again:

1. `A->B (5)`: 0 + 5 < 2? NO.
2. `A->C (10)`: 0 + 10 < 10? NO.
3. `C->B (-8)`: 10 + (-8) < 2? NO. *End of Pass 2 Distances:* `{ A: 0, B: 2, C: 10 }`

Step 3: Negative Cycle Check (Pass 3) We check every edge one last time. Nothing updates. We are safe! Final distances are correct.

Scenario B: The Infinite Money Glitch (Negative Cycle)

- Total Vertices (V) = 3.
- Loop (V - 1) = 2 times.

```
Graph Layout:
      (2)
  A --------> B
               \ (2)
   (-5)         v
  <------------- C
```

Edges List: `[A->B (2), B->C (2), C->A (-5)]`

Pass 1:

- `A->B (2)`: `Distances[B] = 2`
- `B->C (2)`: `Distances[C] = 4` (since B is 2, 2+2=4)
- `C->A (-5)`: `Distances[A] = -1` (since C is 4, 4-5=-1)

Pass 2:

- `A->B (2)`: `Distances[B]` becomes `-1 + 2 = 1`. (It shrank!)
- `B->C (2)`: `Distances[C]` becomes `1 + 2 = 3`. (It shrank!)
- `C->A (-5)`: `Distances[A]` becomes `3 - 5 = -2`. (It shrank!)

Pass 3 (The Detection Check): We check `A->B (2)`. Cost to A is -2. `-2 + 2 = 0`. Is `0` less than `Distances[B]` (which is 1)? YES. Because a value shrank on the Vth pass, we have proven a negative cycle exists. The algorithm raises an alarm and halts.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Bellman-Ford Algorithm for Single Source Shortest Path
 * @param {number} numVertices - Total number of nodes (V)
 * @param {number[][]} edges - Array of edges [source, destination, weight]
 * @param {number} startNode - The starting node
 * @return {number[]|null} - Array of distances, or null if negative cycle found
 */
function bellmanFord(numVertices, edges, startNode) {
    // 1. Setup distances array filled with Infinity
    const distances = new Array(numVertices).fill(Infinity);
    distances[startNode] = 0;
```

```
    // 2. Main Loop: Run exactly (V - 1) times
    for (let i = 0; i < numVertices - 1; i++) {

        // We track if ANY changes happened in this pass.
        // If no changes happened, we can stop early to save time!
        let updated = false;

        for (let j = 0; j < edges.length; j++) {
            const u = edges[j][0];
            const v = edges[j][1];
            const weight = edges[j][2];

            // Relaxation Step
            // Ensure we don't add weight to Infinity (which causes weird math issues)
            if (distances[u] !== Infinity && distances[u] + weight < distances[v]) {
                distances[v] = distances[u] + weight;
                updated = true;
            }
        }

        // Optimization: If nothing changed, we are already perfectly optimized.
        if (!updated) {
            break;
        }
    }

    // 3. Cycle Detection Loop: Run ONE more time
    for (let j = 0; j < edges.length; j++) {
        const u = edges[j][0];
        const v = edges[j][1];
        const weight = edges[j][2];

        if (distances[u] !== Infinity && distances[u] + weight < distances[v]) {
            console.error("Graph contains a negative weight cycle!");
            return null; // Return null or throw an error
        }
    }

    return distances;
}

// --- Usage ---
// 3 vertices (0, 1, 2). Edges: 0->1(5), 0->2(10), 2->1(-8)
const v = 3;
const edgesList = [
    [0, 1, 5],
```

```
    [0, 2, 10],
    [2, 1, -8]
];

console.log(bellmanFord(v, edgesList, 0));
// Output: [ 0, 2, 10 ]
```

---

5. LeetCode questions it applies to

- **Cheapest Flights Within K Stops (LeetCode 787):** A legendary problem. You use a tweaked version of Bellman-Ford (see Variation B below). Because Bellman-Ford naturally processes the graph one "edge depth" at a time, running the main loop exactly `K + 1` times guarantees you find the cheapest flight taking at most K connecting flights!
- **Network Delay Time (LeetCode 743):** While usually solved with Dijkstra's algorithm (because times are strictly positive), you can absolutely solve this with Bellman-Ford. You just run it and find the maximum value in your final distances array.
- **Path with Maximum Probability (LeetCode 1514):** You can use Bellman-Ford by initializing distances to `0` instead of Infinity, multiplying probabilities instead of adding weights, and checking if `distances[u] * weight > distances[v]`.

---

6. Time and Space Complexity

Let **V** equal the number of Vertices (Nodes). Let **E** equal the number of Edges.

```
Time Complexity Derivation:
-------------------------------------------------------
Task                     | Operations           | Big O
-------------------------|----------------------|--------
1. Initialize array      | V elements           | O(V)
2. Main Loop             | Loops (V - 1) times  | O(V)
3. Inner Edge Loop       | Loops E times        | O(E)
4. Cycle Detection Loop  | Loops E times        | O(E)
-------------------------------------------------------
Total Time: The nested loops dictate the time. We run an
O(E) operation roughly O(V) times.
Time Complexity is exactly O(V * E).

Space Complexity Derivation:
-------------------------------------------------------
Task                     | Memory Used          | Big O
-------------------------|----------------------|--------
```

```
1. Distances Array       | Stores V items       | O(V)
-------------------------------------------------------
Total Space Complexity: O(V)
*Note: This is drastically more memory efficient than
Dijkstra's O(V + E) space because we don't need to maintain
an Adjacency List or a Priority Queue. We just iterate
straight over the raw Edges array.
```

---

7. Variations and Common Tweaks

Variation A: The Early Exit Optimization **The Problem:** The standard algorithm rigidly runs V - 1 times. If the shortest paths are found on pass #2, passes 3 through V-1 are completely wasted computation. **The Tweak:** You add a boolean flag (like `updated` in the code sample above). You set it to `false` at the start of each pass. If any edge successfully relaxes, set it to `true`. If a full pass finishes and `updated` is still `false`, it means the graph has settled. You can safely `break` out of the loop early.

Variation B: Distance Limit / At Most K Edges (LeetCode 787) **The Problem:** You want the cheapest route, but you are not allowed to take more than `K` edges (flights). **The Tweak:** The beauty of Bellman-Ford is that after exactly 1 pass, it finds the shortest paths using at most 1 edge. After 2 passes, it finds the shortest paths using at most 2 edges. So, to limit it to K stops (which is K + 1 edges), you just change the outer loop to run `K + 1` times instead of `V - 1` times. *The Crucial Trap:* During a single pass, a chain reaction could happen where Node A updates Node B, and then Node B immediately updates Node C in the same pass. That equals 2 edges traveled in 1 pass! To fix this, you must **clone** the `distances` array at the start of every pass, and only use the clone for your math checks, updating the real array.

Snippet for K Stops Tweak:

```javascript
// Run exactly K + 1 times
for (let i = 0; i < K + 1; i++) {

    // TWEAK: Clone the array so updates in this pass don't chain forward!
    const tempDistances = [...distances];

    for (let j = 0; j < edges.length; j++) {
        const u = edges[j][0];
        const v = edges[j][1];
        const weight = edges[j][2];

        // TWEAK: Read from the CLONE, write to the REAL array
        if (tempDistances[u] !== Infinity && tempDistances[u] + weight < distances[v]) {
```

```
            distances[v] = tempDistances[u] + weight;
        }
    }
}
```

Bellman-Ford Algorithm (SSSP) for max K iterations

Here is a complete guide to understanding and implementing the Bellman-Ford Algorithm specifically modified for a **Maximum of K Iterations**.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are booking a flight from New York to Tokyo. You want the absolute cheapest ticket possible. However, you absolutely refuse to take a route that has more than 2 layovers (stops).

Why standard algorithms fail:

- **Standard Dijkstra's Algorithm:** It is greedy. It will happily find a $300 flight route, completely ignoring the fact that it takes 6 layovers to get there. It prioritizes pure cost, not the number of edges.
- **Standard Bellman-Ford:** It finds the absolute shortest path by looping (Total Nodes - 1) times. Like Dijkstra, it doesn't inherently respect a strict "edge limit" if left unmodified.

Why we need this algorithm: By intentionally restricting the Bellman-Ford algorithm to run exactly `K + 1` times (where K is your layover limit), we can find the cheapest path that uses *at most* K stops. It perfectly blends cost-optimization with a strict limit on how many steps (edges) you are allowed to take.

---

2. What is it? (Pseudocode & Flow)

This is a heavily controlled version of Bellman-Ford. In graph theory, 1 "stop" (layover) means traveling across 2 edges. So, if your limit is K stops, you are allowed to travel across `K + 1` edges.

The most critical part of this algorithm is the **Prevention of Chaining**. In a single loop, if Node A updates Node B, and then Node B immediately updates Node C, you just traveled 2 edges in a single iteration! To prevent this, we must use a **Snapshot** (a temporary clone) of the distances array at the start of every loop.

```
BELLMAN-FORD (MAX K ITERATIONS) WORKFLOW:

[SETUP]
1. Create a 'distances' array. Set distance to Start Node = 0.
2. Set distance to all other Nodes = Infinity.
3. Determine Max Edges allowed = K + 1.

[MAIN LOOP]
Repeat exactly (K + 1) times:
   |
   |-- [THE SNAPSHOT]
   |   Create 'tempDistances' as an exact copy of 'distances'.
   |   (This prevents chain-reactions during a single loop).
   |
   |-- For EVERY edge in the graph (from U to V with weight W):
         |
         |-- [RELAXATION CHECK]
         |   Look at the 'tempDistances' for U, but write to 'distances' for V!
         |   Is tempDistances[U] + W < distances[V]?
               |
               |--> If YES: distances[V] = tempDistances[U] + W
               |--> If NO: Do nothing.

[END]
Return the 'distances' array!
```

---

3. How does it work? (Walkthrough)

Let's look at the "Chaining Trap" to see exactly why the Snapshot array is the secret to making this algorithm work.

Scenario: The Chaining Trap We want to fly from **A** to **C**. Limit: **0 Layovers** (This means exactly 1 Edge maximum!)

- Flight 1: A to B (Cost: 100)
- Flight 2: B to C (Cost: 100)
- Flight 3: A to C (Cost: 500)

```
Graph Layout:
      (500)
  A --------> C
   \          ^
(100)\      /(100)
      v    /
        B
```

Edges List: `[A->C (500), A->B (100), B->C (100)]`

Walkthrough WITHOUT the Snapshot (The Wrong Way):

- *Iteration 1 (Allowed 1 edge):*

1. Check `A->C (500)`: 0 + 500 < Infinity. `Distances[C] = 500`.
2. Check `A->B (100)`: 0 + 100 < Infinity. `Distances[B] = 100`.
3. Check `B->C (100)`: `Distances[B]` is now 100. 100 + 100 = 200. Is 200 < 500? YES! `Distances[C] = 200`.

- *Result:* The algorithm says the cheapest flight with 0 layovers costs 200. **THIS IS A LIE!** The route A->B->C has 1 layover. It traveled 2 edges in 1 iteration because B updated C immediately after A updated B.

Walkthrough WITH the Snapshot (The Correct Way):

- *Iteration 1 (Allowed 1 edge):*
- `Distances = { A: 0, B: Infinity, C: Infinity }`
- **Snapshot = { A: 0, B: Infinity, C: Infinity }**

1. Check `A->C (500)`: Snapshot[A] is 0. 0 + 500 < Infinity. `Distances[C] = 500`.
2. Check `A->B (100)`: Snapshot[A] is 0. 0 + 100 < Infinity. `Distances[B] = 100`.
3. Check `B->C (100)`: Snapshot[B] is Infinity! Infinity + 100 is Infinity. Is Infinity < Distances[C]? NO!

- *Result:* The loop ends. `Distances[C]` remains 500. The algorithm correctly identifies that the only valid flight with 0 layovers is the direct $500 flight!

---

4. Code Sample (JavaScript)

```javascript
/**
 * Modified Bellman-Ford to find shortest path within K stops
 * @param {number} n - Number of cities (nodes)
 * @param {number[][]} flights - Array of [source, destination, price]
 * @param {number} src - Starting city
 * @param {number} dst - Destination city
 * @param {number} k - Maximum number of stops allowed
 * @return {number} - Cheapest price, or -1 if unreachable
 */
function findCheapestPrice(n, flights, src, dst, k) {
    // 1. Initialize distances array with Infinity
    let distances = new Array(n).fill(Infinity);
    distances[src] = 0;
```

```
    // K stops means we are allowed to take K + 1 edges (flights)
    const maxEdges = k + 1;

    // 2. Main Loop: Run exactly 'maxEdges' times
    for (let i = 0; i < maxEdges; i++) {

        // THE SNAPSHOT: Clone the array to prevent chaining in the same iteration
        // Using spread operator [...] creates a shallow copy
        const tempDistances = [...distances];

        // Track if any updates happened. If not, we can stop early!
        let updated = false;

        // 3. Loop through all edges
        for (let j = 0; j < flights.length; j++) {
            const u = flights[j][0]; // Source city
            const v = flights[j][1]; // Destination city
            const cost = flights[j][2]; // Price of flight

            // Relaxation Step
            // VERY IMPORTANT: We read from tempDistances[u], but we update distances[v]
            if (tempDistances[u] !== Infinity && tempDistances[u] + cost < distances[v]) {
                distances[v] = tempDistances[u] + cost;
                updated = true;
            }
        }

        // Optimization: If no distances were updated in this pass,
        // further passes will also do nothing. Break early.
        if (!updated) {
            break;
        }
    }

    // 4. Return result
    // If the distance to destination is still Infinity, we never reached it
    if (distances[dst] === Infinity) {
        return -1;
    }
    return distances[dst];
}

// --- Usage ---
// 3 cities. Flights: 0->1(100), 1->2(100), 0->2(500)
const cities = 3;
const flightRoutes = [[0,1,100], [1,2,100], [0,2,500]];
```

```
console.log(findCheapestPrice(cities, flightRoutes, 0, 2, 0));
// Output: 500 (Because K=0 stops forces the direct flight)

console.log(findCheapestPrice(cities, flightRoutes, 0, 2, 1));
// Output: 200 (Because K=1 stop allows the 0 -> 1 -> 2 route)
```

---

5. LeetCode questions it applies to

- **Cheapest Flights Within K Stops (LeetCode 787):** This is the flagship question for this algorithm. The code provided in Section 4 is the exact, optimal solution for this problem.
- **Maximum Probability Path with Edge Limits (Variation of LeetCode 1514):** If a problem asks you to find the path with the highest probability of success, but limits the number of connections you can make, you use this exact same logic. You initialize with `0` instead of `Infinity`, use multiplication instead of addition, and flip the check to `tempDistances[u] * prob > distances[v]`.

---

6. Time and Space Complexity

Let **V** equal the number of Vertices (Cities). Let **E** equal the number of Edges (Flights). Let **K** equal the maximum number of stops allowed.

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Initialize arrays      | V elements           | O(V)
2. Main Loop              | Loops K + 1 times    | O(K)
3. Inner Edge Loop        | Loops E times        | O(E)
4. Array Cloning          | V elements cloned    | O(V) inside loop
-----------------------------------------------------------
Total Time: We run an O(E) edge check and an O(V) array
clone exactly K times.
Time Complexity is O(K * E + K * V), which simplifies
to O(K * (E + V)).

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Distances Array        | Stores V items       | O(V)
2. Temp/Snapshot Array    | Stores V items       | O(V)
```

```
-------------------------------------------------------
Total Space Complexity: O(V) + O(V) = O(V)
This is incredibly memory efficient compared to building
adjacency lists or Priority Queues for Dijkstra.
```

---

7. Variations and Common Tweaks

Variation A: Finding paths with EXACTLY K edges **The Problem:** The standard algorithm finds the cheapest path using *at most* K stops (meaning a direct flight of 0 stops is a valid answer even if K=5). What if the problem says "You must take a route that uses EXACTLY K stops"? **The Tweak:** Instead of initializing the `tempDistances` array as a direct clone of `distances` (`[...distances]`), you fill the entire `tempDistances` array with `Infinity` at the start of every loop, except for the results of the previous step. This ensures that any path recorded in iteration 3 absolutely *must* have taken exactly 3 edges to get there, wiping out any 1-edge or 2-edge shortcuts from memory.

Snippet for EXACTLY K edges:

```
// Before the main loop
let previousDistances = new Array(n).fill(Infinity);
previousDistances[src] = 0;

for (let i = 0; i < k + 1; i++) {
    // TWEAK: Start completely fresh every iteration!
    let currentDistances = new Array(n).fill(Infinity);

    for (let j = 0; j < flights.length; j++) {
        const u = flights[j][0];
        const v = flights[j][1];
        const cost = flights[j][2];

        // Read from previous, write to current
        if (previousDistances[u] !== Infinity && previousDistances[u] + cost < currentDistar
            currentDistances[v] = previousDistances[u] + cost;
        }
    }

    // The current becomes the previous for the next iteration
    previousDistances = currentDistances;
}
return previousDistances[dst];
```

Variation B: Solving with BFS instead of Bellman-Ford **The Problem:** Some interviewers might ask: "Can you solve the K-stops problem

using Breadth-First Search (BFS) instead of Bellman-Ford?" **The Tweak:** Yes, you can! You use a standard Queue, but you push an object containing `[currentNode, currentCost, currentStops]`. You process the queue level-by-level (each level is 1 stop). If the popped node has `stops > K`, you ignore its neighbors. You also need a `minCost` array to track the cheapest way you've reached each node so far, ignoring paths that arrive at a node with a worse cost.

(Note: While BFS works conceptually well here because the "levels" naturally limit the stops, the Bellman-Ford array-cloning approach is mathematically identical in its layer-by-layer expansion but requires much less code and memory overhead!)

Floyd-Warshall Algorithm (APSP)

Here is a complete guide to understanding and implementing the Floyd-Warshall Algorithm for the All-Pairs Shortest Path (APSP) problem.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are an airline planning flight routes. You don't just want to know the shortest path from New York to everywhere else (Single-Source). You need a massive cheat sheet that instantly tells you the shortest path between *every single pair of cities* in your entire network.

Why we need it (when we already have Dijkstra and Bellman-Ford): Dijkstra's and Bellman-Ford algorithms are "Single-Source" algorithms. They find paths from *one* specific starting point. If you have a graph with V vertices and you want the shortest path between every possible pair, you would have to run Dijkstra's algorithm V separate times. If your graph has negative weights, you'd have to run Bellman-Ford V separate times, which is incredibly slow.

The **Floyd-Warshall Algorithm** computes the shortest paths between **all pairs of nodes simultaneously** in a few lines of incredibly elegant code. It handles negative weights perfectly, and it is the absolute go-to solution when graphs are dense (lots of edges) and you need a complete distance matrix.

---

2. What is it? (Pseudocode & Flow)

Floyd-Warshall is a **Dynamic Programming** algorithm. Instead of navigating the graph like a maze, it builds a 2D grid (an Adjacency Matrix) and progressively refines it.

The core philosophy is based on introducing one "Intermediate Node" at a time. It asks one simple question millions of times: **"Is the path from A to B**

shorter if I take a detour through K?"

Here is the logic structure:

```
FLOYD-WARSHALL WORKFLOW:

[SETUP]
1. Create a 2D Matrix called 'dist' of size V x V.
2. Fill dist[i][j] with the direct edge weight from node i to node j.
3. If there is no direct edge, set it to Infinity.
4. Set the diagonal to 0 (distance from a node to itself is 0).

[MAIN LOOP]
Let 'k' be the Intermediate Node.
Let 'i' be the Start Node.
Let 'j' be the End Node.

FOR k from 0 to V-1:
  FOR i from 0 to V-1:
    FOR j from 0 to V-1:
       |
       |-- [THE DETOUR CHECK]
       |   Is (dist[i][k] + dist[k][j]) < dist[i][j]?
       |
       |--> If YES: We found a faster route using 'k' as a stepping stone!
                    Update dist[i][j] = dist[i][k] + dist[k][j]

[END]
Return the 'dist' matrix!
```

Crucial Warning: The intermediate node `k` MUST be the outermost loop. If you put `k` on the inside, the dynamic programming chain breaks, and you won't find the true shortest paths.

---

3. How does it work? (Walkthrough)

Let's trace a scenario to see how the matrix evolves.

Scenario: The Hidden Detour We have 3 nodes: A (Index 0), B (Index 1), C (Index 2).

- Direct path A -> C takes 10 hours.
- Path A -> B takes 3 hours.
- Path B -> C takes 2 hours.

```
Graph Layout:
      (10)
```

```
  A --------> C
   \          ^
 (3)\        /(2)
     v      /
        B
```

Step 0: The Initial Matrix We fill the grid with exactly what we see right now. No detours allowed yet.

```
dist = [
  // A   B   C
  [  0,  3, 10 ], // A to anywhere
  [Inf,  0,  2 ], // B to anywhere
  [Inf, Inf, 0 ]  // C to anywhere
]
```

Step 1: Allow detours through A (k = 0) We check every pair (i, j) to see if routing through A helps.

- Example: Can B get to C faster by going through A? `dist[B][A] + dist[A][C] -> Infinity + 10 = Infinity`. No.
- *Result:* No changes. A is a dead end for incoming traffic.

Step 2: Allow detours through A and B (k = 1) We check every pair (i, j) to see if routing through B helps.

- Example: Can A get to C faster by going through B? Let i = 0 (A), j = 2 (C). `dist[0][1] + dist[1][2] -> 3 + 2 = 5`. Is 5 < `dist[0][2]` (which is currently 10)? **YES!**
- *Action:* Update `dist[0][2] = 5`.

```
Matrix Updates:
dist = [
  [  0,  3,  5 ], <-- A to C updated to 5!
  [Inf,  0,  2 ],
  [Inf, Inf, 0 ]
]
```

Step 3: Allow detours through A, B, and C (k = 2) We check if routing through C helps anyone. C has no outgoing edges, so `dist[C][anywhere]` is Infinity. Nothing updates.

Final Result: The matrix instantly tells us the cheapest route between any two points.

---

4. Code Sample (JavaScript)

```javascript
/**
 * Floyd-Warshall Algorithm for All-Pairs Shortest Path
```

```
 * @param {number} n - Number of vertices (0 to n-1)
 * @param {number[][]} edges - Array of [u, v, weight]
 * @return {number[][]} - 2D matrix of shortest distances
 */
function floydWarshall(n, edges) {
    // 1. Initialize the 2D distance matrix with Infinity
    const dist = new Array(n);
    for (let i = 0; i < n; i++) {
        dist[i] = new Array(n).fill(Infinity);
        // Distance from a node to itself is 0
        dist[i][i] = 0;
    }

    // 2. Populate the matrix with initial direct edge weights
    for (let i = 0; i < edges.length; i++) {
        const u = edges[i][0];
        const v = edges[i][1];
        const weight = edges[i][2];

        // Assuming a directed graph. (If undirected, also set dist[v][u] = weight)
        dist[u][v] = weight;
    }

    // 3. Main Algorithm: The Triple Nested Loop
    // 'k' MUST be the outer loop!
    for (let k = 0; k < n; k++) {
        for (let i = 0; i < n; i++) {
            for (let j = 0; j < n; j++) {

                // Prevent Javascript Infinity math bugs (Infinity + Infinity)
                if (dist[i][k] !== Infinity && dist[k][j] !== Infinity) {

                    // The Detour Check: Is going through 'k' shorter?
                    const detourDistance = dist[i][k] + dist[k][j];
                    if (detourDistance < dist[i][j]) {
                        dist[i][j] = detourDistance;
                    }
                }
            }
        }
    }

    return dist;
}

// --- Usage ---
```

```
// 3 nodes. Edges: A->B(3), B->C(2), A->C(10)
const numNodes = 3;
const graphEdges = [ [0, 1, 3], [1, 2, 2], [0, 2, 10] ];

const shortestPaths = floydWarshall(numNodes, graphEdges);
console.log(shortestPaths[0][2]); // Output: 5 (The shortest path from A to C)
```

---

5. LeetCode questions it applies to

- **Find the City With the Smallest Number of Neighbors at a Threshold Distance (LeetCode 1334):** This is the ultimate Floyd-Warshall problem. You need to know the distance from *every* city to *every other* city to count how many are within the threshold. You run the algorithm, then just loop over the resulting matrix to count neighbors.
- **Course Schedule IV (LeetCode 1462):** You are asked multiple queries: "Is Course A a prerequisite for Course B?". This is asking for "Reachability". You can use the Transitive Closure variation of Floyd-Warshall (explained below) to build a boolean matrix that answers these queries in O(1) time.
- **Evaluate Division (LeetCode 399):** You are given equations like `a / b = 2.0` and `b / c = 3.0`, and asked to evaluate `a / c`. This is a graph where the edge weight is the division multiplier. You use Floyd-Warshall, but instead of adding weights, you *multiply* them! `dist[i][j] = dist[i][k] * dist[k][j]`.

---

6. Time and Space Complexity

Let **V** equal the number of Vertices (Nodes).

```
Time Complexity Derivation:
-------------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Matrix Initialization  | V rows * V cols      | O(V^2)
2. Outer Loop (k)         | Runs V times         | O(V)
3. Middle Loop (i)        | Runs V times         | O(V)
4. Inner Loop (j)         | Runs V times         | O(V)
-------------------------------------------------------------
Total Time: The triple nested loop dominates the execution.
V * V * V = O(V^3).
Because it is exactly V^3 operations, this algorithm is
only suitable for small to medium graphs (V <= 400 approx).

Space Complexity Derivation:
```

```
-------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Distance Matrix        | V rows * V cols      | O(V^2)
-------------------------------------------------------
Total Space Complexity: O(V^2)
```

---

7. Variations and Common Tweaks

Variation A: Transitive Closure (Reachability) **The Problem:** You don't care about distances or edge weights. You only want to know: "Is it mathematically possible to get from Node A to Node B?" **The Tweak:** Instead of an array of numbers, you use a boolean matrix (`true` or `false`). The mathematical addition turns into a logical `AND`, and the mathematical comparison turns into a logical `OR`.

Snippet for Transitive Closure:

```
// Initialize reachMatrix with false, and true if a direct edge exists
for (let k = 0; k < n; k++) {
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            // "I can reach j from i IF I could ALREADY reach j from i,
            // OR if I can reach k from i AND I can reach j from k."
            reachMatrix[i][j] = reachMatrix[i][j] || (reachMatrix[i][k] && reachMatrix[k][j]
        }
    }
}
```

Variation B: Negative Cycle Detection **The Problem:** Just like Bellman-Ford, you need to know if the graph contains an infinite money glitch (a cycle where the total weight is negative). **The Tweak:** Remember that in step 0, we set the diagonal `dist[i][i]` to 0 (the distance from a node to itself). If a negative cycle exists, the algorithm will eventually find a "detour" from a node back to itself that costs less than 0!

Snippet for Negative Cycle Detection:

```
// Run the normal Floyd-Warshall algorithm first...

// Then, just check the main diagonal of the matrix!
for (let i = 0; i < n; i++) {
    if (dist[i][i] < 0) {
        return "Negative Cycle Detected!";
    }
}
```

Variation C: Minimax / Maximin Paths **The Problem:** You are driving a truck through a network of bridges. You want to find a path from A to B that *maximizes* the weight limit of the *weakest* bridge on your route. **The Tweak:** Instead of `dist[i][k] + dist[k][j]`, you change the inner logic to use `Math.min` and `Math.max` to evaluate the bottleneck capacities of the paths.

Prim's Algorithm (MST)

Here is a complete guide to understanding and implementing Prim's Algorithm for finding a Minimum Spanning Tree (MST).

---

1. Problem it solves and Why we need it

The Problem: Imagine you are laying down fiber-optic internet cables to connect 5 different cities. Laying cable is incredibly expensive, and the cost varies depending on the distance and terrain between any two cities. You don't need every city to be directly connected to every other city; you just need to ensure that every city can reach the network somehow.

Your goal: Find the absolutely cheapest way to connect all the cities together without forming any redundant loops (which waste money).

Why we need it: This exact problem is called the **Minimum Spanning Tree (MST)** problem.

- "Spanning" means it touches every single vertex (node).
- "Tree" means there are no cycles (loops).
- "Minimum" means the total weight (cost) of the edges is as small as mathematically possible.

If you try to brute-force this by guessing every possible tree combination, the number of possibilities explodes astronomically (using Cayley's formula, a graph with 10 nodes has 100 million possible trees). Prim's Algorithm gives us a fast, guaranteed way to find the absolute cheapest tree.

---

2. What is it? (Pseudocode & Flow)

Prim's Algorithm is a **Greedy Algorithm**. It works by starting at any random node and progressively "growing" the tree. At every step, it looks at all the available connections reaching out from the *currently built tree* to the *unvisited nodes*, and it greedily grabs the cheapest one.

Because it grows outward like a puddle, it guarantees that all selected nodes remain connected.

```
PRIM'S ALGORITHM WORKFLOW:

[SETUP]
1. Create a 'visited' set to keep track of nodes already in our tree.
2. Create a 'totalCost' variable set to 0.
3. Create a Min-Priority Queue (PQ) to store edges.
4. Pick ANY random node to start (e.g., Node 0). Mark it 'visited'.
5. Add all edges connected to Node 0 into the PQ.

[MAIN LOOP]
While PQ is not empty AND 'visited' size < total nodes:
   |
   |-- Pop the CHEAPEST edge from the PQ.
   |   Let's say this edge connects Node U (in tree) to Node V.
   |
   |-- [CYCLE CHECK]
   |   Is Node V already in the 'visited' set?
   |   --> If YES: Taking this edge would create a loop!
   |               Throw it away. (Continue loop)
   |
   |   --> If NO: This is a valid, cheap expansion!
               1. Mark Node V as 'visited'.
               2. Add the edge's weight to 'totalCost'.
               3. Add all edges connected to Node V into the PQ
                  (only if they lead to unvisited nodes).

[END]
Return 'totalCost'! (Or return the list of selected edges).
```

---

3. How does it work? (Walkthrough)

Let's trace a scenario to see how the algorithm builds the tree and avoids cycles.

Scenario: Wiring the Neighborhood We want to connect 4 houses (A, B, C, D) with the cheapest wire.

```
Graph Layout (Undirected):
           (1)
        A ----- B
        | \     |
     (4)|  \(3) | (2)
        |   \   |
        C ----- D
           (5)
```

Step 0: Setup

- Pick a starting node: Let's start at **A**.
- Visited: `{ A }`
- Total Cost: `0`
- Edges in PQ: `[ A-B(cost: 1), A-D(cost: 3), A-C(cost: 4) ]`

Step 1: Pop the cheapest edge

- Pop **A-B (cost: 1)**.
- Check B: Is B in Visited? No.
- Action: Add B to Visited. Add 1 to total cost.
- Add B's edges to PQ: B-D(cost: 2). (We ignore B-A because A is visited).
- Visited: `{ A, B }`
- Total Cost: `1`
- PQ: `[ B-D(2), A-D(3), A-C(4) ]` *(Notice PQ automatically sorts by cheapest!)*

Step 2: Pop the cheapest edge

- Pop **B-D (cost: 2)**.
- Check D: Is D in Visited? No.
- Action: Add D to Visited. Add 2 to total cost.
- Add D's edges to PQ: D-C(cost: 5). (We ignore D-A and D-B).
- Visited: `{ A, B, D }`
- Total Cost: `1 + 2 = 3`
- PQ: `[ A-D(3), A-C(4), D-C(5) ]`

Step 3: Pop the cheapest edge (The Cycle Trap)

- Pop **A-D (cost: 3)**.
- Check D: Is D in Visited? **YES!**
- If we connect A to D, we create the triangle A-B-D. We don't want loops!
- Action: Throw it away. Do nothing.

Step 4: Pop the next cheapest edge

- Pop **A-C (cost: 4)**.
- Check C: Is C in Visited? No.
- Action: Add C to Visited. Add 4 to total cost.
- Add C's edges: None lead to unvisited nodes.
- Visited: `{ A, B, D, C }`
- Total Cost: `3 + 4 = 7`

End: We have visited all 4 nodes. The algorithm stops. The Minimum Spanning Tree costs exactly 7!

---

4. Code Sample (JavaScript)

Note: Just like with Dijkstra, JavaScript lacks a native Min-Priority Queue. I will simulate it using an array and `.sort()`. In a production environment, you would use a proper Min-Heap class to guarantee optimal speed.

```javascript
/**
 * Prim's Algorithm for Minimum Spanning Tree
 * @param {number} numNodes - Total number of nodes (0 to numNodes - 1)
 * @param {Object} graph - Adjacency list: { 0: [{node: 1, weight: 4}] }
 * @return {number} - The total cost of the MST
 */
function primsAlgorithm(numNodes, graph) {
    const visited = new Set();
    let mstCost = 0;

    // The Priority Queue stores objects: { node: number, weight: number }
    const pq = [];

    // 1. Start at node 0 (Any node works!)
    visited.add(0);

    // Add all initial edges from node 0 to the queue
    const startEdges = graph[0] || [];
    for (let i = 0; i < startEdges.length; i++) {
        pq.push(startEdges[i]);
    }

    // 2. Main Loop
    // We stop when we have visited every node, OR we run out of edges
    while (pq.length > 0 && visited.size < numNodes) {

        // [SIMULATING MIN-HEAP POP]
        // Sort descending and pop from the end for the cheapest edge
        pq.sort((a, b) => b.weight - a.weight);
        const currentEdge = pq.pop();

        const targetNode = currentEdge.node;
        const weight = currentEdge.weight;

        // 3. Cycle Check: If we already visited this node, skip it!
        if (visited.has(targetNode)) {
            continue;
        }

        // 4. Valid Expansion: Add to our tree
```

```
        visited.add(targetNode);
        mstCost = mstCost + weight;

        // 5. Add new outgoing edges to the queue
        const neighbors = graph[targetNode] || [];
        for (let i = 0; i < neighbors.length; i++) {
            const neighborNode = neighbors[i].node;

            // Only add edges that lead to UNVISITED nodes to save space
            if (!visited.has(neighborNode)) {
                pq.push(neighbors[i]);
            }
        }
    }

    // Edge case: If the graph is disconnected, we can't form a full MST
    if (visited.size < numNodes) {
        return -1; // Or throw an error
    }

    return mstCost;
}

// --- Usage ---
// 4 Nodes. Edges: 0-1(1), 0-3(3), 0-2(4), 1-3(2), 2-3(5)
const myGraph = {
    0: [{node: 1, weight: 1}, {node: 3, weight: 3}, {node: 2, weight: 4}],
    1: [{node: 0, weight: 1}, {node: 3, weight: 2}],
    2: [{node: 0, weight: 4}, {node: 3, weight: 5}],
    3: [{node: 0, weight: 3}, {node: 1, weight: 2}, {node: 2, weight: 5}]
};

console.log(primsAlgorithm(4, myGraph));
// Output: 7
```

---

5. LeetCode questions it applies to

- **Min Cost to Connect All Points (LeetCode 1584):** You are given a list of coordinates on a 2D plane. The cost to connect them is the Manhattan distance `|x1 - x2| + |y1 - y2|`. You treat every point as a node, calculate the distance to every other unvisited point dynamically, and use Prim's to build the absolute cheapest web of connections.
- **Connecting Cities With Minimum Cost (LeetCode 1135):** The textbook direct application of the algorithm. You are given N cities and

connections with costs. Run Prim's, and return the `mstCost` (or -1 if the graph is disconnected).

- **Optimize Water Distribution in a Village (LeetCode 1168):** A brilliant trick question! You can build pipes between houses (edges), OR you can dig a well at a house (a flat cost). **The Trick:** Create a fake "Node 0" (the underground water source). Add an edge from Node 0 to every house, where the edge weight is the cost of digging a well at that house. Then, just run standard Prim's algorithm!

---

6. Time and Space Complexity

Let **V** equal the number of Vertices (Nodes). Let **E** equal the number of Edges.

```
Time Complexity Derivation (Assuming a True Min-Heap):
-------------------------------------------------------
Task                     | Operations          | Big O
-------------------------|---------------------|--------
1. Add Edges to PQ       | 1 push per Edge     | O(E * log E)
2. Pop Edges from PQ     | 1 pop per Edge      | O(E * log E)
3. Cycle Check           | O(1) Set lookup     | O(1)
-------------------------------------------------------
Total Time Complexity: O(E * log E)
Because the maximum number of edges in a graph is V^2,
log(E) is equivalent to log(V^2), which simplifies to
2 * log(V). Therefore, the time complexity is often written
interchangeably as O(E * log V).

Space Complexity Derivation:
-------------------------------------------------------
Task                     | Memory Used         | Big O
-------------------------|---------------------|--------
1. Priority Queue        | Can hold up to E    | O(E)
2. Visited Set           | Stores V items      | O(V)
3. Graph Representation  | V nodes, E edges    | O(V + E)
-------------------------------------------------------
Total Space Complexity: O(V + E)
```

---

7. Variations and Common Tweaks

Variation A: Prim's for Dense Graphs (No Priority Queue) **The Problem:** If a graph is incredibly "dense" (meaning almost every node is connected to every other node), pushing all those edges into a Priority Queue creates massive overhead. E approaches V^2, making PQ operations sluggish. **The Tweak:** Drop the Priority Queue entirely. Use an array called `minEdgeCost`

(size V) filled with Infinity. At each step, use a standard loop to scan the `minEdgeCost` array to find the smallest value for an unvisited node. This reduces the time complexity from `O(E log V)` to a pure **O(V^2)**, which is actually *faster* when `E` is very large!

Snippet for Dense Prim's:

```
// Instead of a PQ, use an array
const minEdgeCost = new Array(numNodes).fill(Infinity);
minEdgeCost[0] = 0; // Start at node 0

for (let step = 0; step < numNodes; step++) {
    // 1. Manually find the unvisited node with the cheapest edge
    let current = -1;
    let minCost = Infinity;

    for (let i = 0; i < numNodes; i++) {
        if (!visited.has(i) && minEdgeCost[i] < minCost) {
            minCost = minEdgeCost[i];
            current = i;
        }
    }

    // 2. Mark visited and add to total
    visited.add(current);
    mstCost += minCost;

    // 3. Update the minEdgeCost array for all neighbors!
    for (let neighbor of graph[current]) {
        if (!visited.has(neighbor.node) && neighbor.weight < minEdgeCost[neighbor.node]) {
            minEdgeCost[neighbor.node] = neighbor.weight;
        }
    }
}
```

Variation B: Pre-Connected / Existing Edges **The Problem:** The prompt says "You need to connect these cities, but City A and City C *already have a road* built between them." You want the minimum cost to build the *remaining* roads. **The Tweak:** Instead of starting with an empty tree, you force those pre-connected nodes into your tree immediately!

1. Add the pre-connected nodes to the `visited` set right at the start.
2. Push *all* outgoing edges from *all* those pre-connected nodes into the initial Priority Queue.
3. Start your main loop exactly as normal.

Tarjan's Algorithm (Strongly Connected Components)

Here is a complete guide to understanding and implementing Tarjan's Algorithm for finding Strongly Connected Components.

---

1. Problem it solves and Why we need it

The Problem: Imagine a network of one-way streets in a city. A "Strongly Connected Component" (SCC) is a neighborhood where, no matter which intersection you start at, you can legally drive to every single other intersection in that neighborhood.

If you are analyzing a Twitter network (who follows whom), or analyzing dependencies in a massive codebase, you need to find these "loops" where everything depends on everything else.

Why we need it: You could just run a standard Depth-First Search (DFS) from every single node to see who can reach whom. But in a graph with a million nodes, that takes O(N^2) time. Your program will time out.

Tarjan's Algorithm is a stroke of absolute genius. It finds *every single SCC* in a massive graph in a single, lightning-fast pass: **O(V + E)** time. It does this by using a DFS traversal combined with a brilliant tracking system to identify the "Root" of every cycle.

---

2. What is it? (The "Lifeline" Concept & Pseudocode)

Tarjan's Algorithm relies on two special numbers assigned to every node when we visit it:

1. **ID (Discovery Time):** A simple counter. The first node visited gets ID 1, the second gets ID 2, etc. This never changes.
2. **Low-Link Value (The Lifeline):** This is the magic number. It represents the *lowest ID* this node can reach. Initially, a node's Low-Link is equal to its own ID. But if it finds a path looping back to an older node, it updates its Low-Link to match that older node's ID.

The Stack: We also maintain a Stack. When we visit a node, we push it onto the Stack. When we realize we have fully explored an SCC, we pop all its nodes off the Stack together.

```
TARJAN'S ALGORITHM WORKFLOW:

[SETUP]
1. Initialize a global 'time' counter to 0.
```

```
2. Create arrays: 'ids', 'low', and a boolean array 'onStack'.
3. Create an empty 'stack'.

[DFS FUNCTION for Node U]
1. Set ids[U] = time, and low[U] = time. Increment time.
2. Push U to 'stack', mark onStack[U] = true.
3. For every neighbor V of U:
     |
     |-- IF V is unvisited:
     |     -> DFS(V)  (Dive deeper!)
     |     -> UPON RETURN: Update low[U] = min(low[U], low[V])
     |        (If my neighbor found a lifeline to an older node, I get it too!)
     |
     |-- ELSE IF V is currently on the stack:
           -> We hit a back-edge (a cycle)!
           -> Update low[U] = min(low[U], ids[V])
              (Grab the older node's ID as my new lifeline).

4. [THE ROOT CHECK]
   After checking all neighbors, is ids[U] == low[U]?
   --> If YES: Node U is the "Root" (the start) of an SCC!
               Pop nodes off the stack until you pop U.
               Group them together. That is one complete SCC!
```

---

3. How does it work? (Walkthrough)

Let's walk through a tricky scenario to see how the Low-Link values bubble up, and how the algorithm prevents false cycles.

Scenario: The Graph with a "Trap" Cross-Edge

```
Nodes: 0, 1, 2, 3
Edges: 0->1, 1->2, 2->0  (This is a triangle loop / SCC)
       2->3              (Node 2 escapes to Node 3)
       3->3              (Node 3 loops to itself / SCC)
```

Walkthrough:

Step 1: Start at Node 0

- `ids[0] = 0`, `low[0] = 0`. Stack: `[0]`
- Go to neighbor 1.

Step 2: Visit Node 1

- `ids[1] = 1`, `low[1] = 1`. Stack: `[0, 1]`
- Go to neighbor 2.

Step 3: Visit Node 2

- `ids[2] = 2`, `low[2] = 2`. Stack: `[0, 1, 2]`
- Node 2 has two neighbors: 0 and 3. Let's check 3 first.

Step 4: Visit Node 3

- `ids[3] = 3`, `low[3] = 3`. Stack: `[0, 1, 2, 3]`
- Node 3's neighbor is 3. It's already visited AND on the stack.
- Back-edge found! `low[3] = min(low[3], ids[3])` -> remains 3.
- Done with Node 3's neighbors.
- **Root Check for Node 3:** Is `ids[3] == low[3]`? (3 == 3). **YES!**
- *Action:* Pop from stack until we pop 3.
- Stack is now: `[0, 1, 2]`.
- **SCC Found: [3]**

Step 5: Backtrack to Node 2

- Node 2 just finished exploring Node 3. Node 3 is NOT on the stack anymore.
- Now Node 2 checks its other neighbor: Node 0.
- Node 0 is already visited AND is currently on the stack!
- Back-edge found! `low[2] = min(low[2], ids[0])` -> `min(2, 0)` -> **0**.
- Node 2's Low-Link is now 0. It found a lifeline back to the start!
- Done with Node 2's neighbors.
- **Root Check for Node 2:** Is `ids[2] == low[2]`? (2 == 0). NO. Backtrack to 1.

Step 6: Backtrack to Node 1

- Node 1 updates its Low-Link from its child (Node 2).
- `low[1] = min(low[1], low[2])` -> `min(1, 0)` -> **0**.
- **Root Check for Node 1:** Is `ids[1] == low[1]`? (1 == 0). NO. Backtrack to 0.

Step 7: Backtrack to Node 0

- Node 0 updates its Low-Link from its child (Node 1).
- `low[0] = min(low[0], low[1])` -> `min(0, 0)` -> **0**.
- **Root Check for Node 0:** Is `ids[0] == low[0]`? (0 == 0). **YES!**
- *Action:* Pop from stack until we pop 0.
- Stack was `[0, 1, 2]`. We pop 2, 1, 0.
- **SCC Found: [2, 1, 0]**

Final SCCs: `[3]` and `[2, 1, 0]`. The algorithm flawlessly separated the isolated loop from the main triangle!

---

4. Code Sample (JavaScript)

```javascript
/**
 * Tarjan's Algorithm to find Strongly Connected Components
 * @param {number} numNodes - Total number of nodes (0 to n-1)
 * @param {Object} graph - Adjacency list representation of the graph
 * @return {number[][]} - Array of SCCs (each SCC is an array of nodes)
 */
function tarjansAlgorithm(numNodes, graph) {
    let idCounter = 0;
    const ids = new Array(numNodes).fill(-1); // -1 means unvisited
    const low = new Array(numNodes).fill(0);
    const onStack = new Array(numNodes).fill(false);
    const stack = [];

    const sccs = [];

    // The core DFS function
    function dfs(at) {
        // 1. Assign ID and Low-Link
        ids[at] = idCounter;
        low[at] = idCounter;
        idCounter++;

        // 2. Add to stack
        stack.push(at);
        onStack[at] = true;

        // 3. Visit all neighbors
        const neighbors = graph[at] || [];
        for (let i = 0; i < neighbors.length; i++) {
            const to = neighbors[i];

            if (ids[to] === -1) {
                // Neighbor is unvisited. Dive deep!
                dfs(to);
                // On return, sync the Low-Link values
                low[at] = Math.min(low[at], low[to]);
            } else if (onStack[to] === true) {
                // Neighbor is already visited and on the stack! Back-edge found.
                // Notice we use ids[to], NOT low[to], for back-edges to prevent
                // cross-contamination between different SCCs.
                low[at] = Math.min(low[at], ids[to]);
            }
        }
```

```
        // 4. Root Check: Are we the start of an SCC?
        if (ids[at] === low[at]) {
            const currentSCC = [];
            // Pop everything off the stack until we pop ourselves
            while (true) {
                const poppedNode = stack.pop();
                onStack[poppedNode] = false;
                currentSCC.push(poppedNode);

                if (poppedNode === at) {
                    break;
                }
            }
            sccs.push(currentSCC);
        }
    }

    // 5. Kick off DFS for every unvisited node (handles disconnected graphs)
    for (let i = 0; i < numNodes; i++) {
        if (ids[i] === -1) {
            dfs(i);
        }
    }

    return sccs;
}
```

---

5. LeetCode questions it applies to

- **Critical Connections in a Network (LeetCode 1192):** This is the flagship problem for Tarjan's logic. You are given a network of servers (an undirected graph). You must find all the "Bridges" (connections that, if cut, would split the network into disconnected pieces). You use the exact `id` and `low` link logic from Tarjan's. (See Variations below for the one-line code change!).
- **Course Schedule (LeetCode 207) / Cycle Detection:** While normally solved with Kahn's Algorithm (Topological Sort) or standard DFS, Tarjan's can instantly tell you if a Directed Acyclic Graph has cycles. If any SCC has a length greater than 1, a cycle exists!
- **Number of Provinces (LeetCode 547):** Although typically solved with Union-Find or basic DFS, Tarjan's works perfectly here to group all connected nodes into isolated components.

---

6. Time and Space Complexity

Let V equal the number of Vertices (Nodes). Let E equal the number of Edges.

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Outer Loop             | Checks V nodes       | O(V)
2. DFS Traversal          | Visits every Node    | O(V)
3. Check Neighbors        | Looks at every Edge  | O(E)
4. Stack Popping          | Every node popped 1x | O(V)
-----------------------------------------------------------
Total Time Complexity: O(V + E)
Even though there are loops inside loops, every node is
added to the stack exactly once, popped exactly once, and
every edge is looked at exactly once.

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. IDs Array              | V integers           | O(V)
2. Low-Link Array         | V integers           | O(V)
3. On-Stack Array         | V booleans           | O(V)
4. Recursion / Stack      | V frames/items max   | O(V)
-----------------------------------------------------------
Total Space Complexity: O(V)
```

---

7. Variations and Common Tweaks

Variation: Finding Bridges (Critical Connections) The Problem: You have an undirected graph. You want to find edges that act as a single point of failure (Bridges). **The Tweak:** The logic for `ids` and `low` remains identical. The only differences are:

1. Because it's undirected, A connects to B, and B connects back to A. When B checks its neighbors, it must **ignore the parent that just called it**, otherwise it will falsely think the immediate reverse path is a cycle. Pass a `parent` variable in the DFS.
2. The Bridge Check: Instead of checking if a node is the root of an SCC at the end, you check the edge *immediately after the recursive call returns.* If `low[neighbor] > ids[currentNode]`, it means the neighbor could NOT find a lifeline back to you or anyone before you. Therefore, the edge between you and the neighbor is a Bridge!

Snippet for Finding Bridges:

```
// DFS takes the current node and the parent we just came from
function dfs(at, parent) {
    ids[at] = low[at] = idCounter++;

    for (let to of graph[at]) {
        // TWEAK 1: Don't look backwards at the node that just called us!
        if (to === parent) continue;

        if (ids[to] === -1) {
            dfs(to, at);
            low[at] = Math.min(low[at], low[to]);

            // TWEAK 2: The Bridge Check!
            // If the neighbor's lowest lifeline is STRICTLY GREATER than my ID,
            // it has no back-edges to safety. This edge is critical!
            if (low[to] > ids[at]) {
                bridges.push([at, to]);
            }
        } else {
            low[at] = Math.min(low[at], ids[to]);
        }
    }
}
```

Dynamic Programming (Top Down or Memoization)

Here is a complete guide to understanding and implementing Top-Down Dynamic Programming, also known as **Memoization**.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are asked to solve a complex math puzzle. You spend 10 minutes figuring it out and finally get the answer: 42. Five minutes later, someone walks up and asks you the *exact same math puzzle*. Do you spend another 10 minutes calculating it from scratch? Of course not! You remember the answer is 42 and tell them instantly.

Standard recursive algorithms have amnesia. If you ask a standard recursive function to calculate the 40th Fibonacci number, it will blindly recalculate the 2nd, 3rd, and 4th Fibonacci numbers millions of times over and over again. This is called the problem of **Overlapping Subproblems**.

Why we need it: Without help, overlapping subproblems cause exponential Time Complexity, like O(2^N). An algorithm that takes 0.01 seconds for an input of N=20 might take 30,000 years for N=100.

Top-Down Dynamic Programming (DP) introduces a "notepad" (called a cache or memo) to the recursion. It allows the algorithm to remember its past answers, transforming a 30,000-year computation into a fraction of a millisecond. It brings exponential O(2^N) time down to linear O(N) time.

---

2. What is it? (Pseudocode & Flow)

Top-Down DP is simply **Recursion + A Dictionary (Cache)**. It is called "Top-Down" because you start at the massive, final goal (the "Top") and recursively break it down into smaller and smaller pieces until you hit a basic base case.

Here is the language-agnostic pseudocode logic for Memoization:

```
TOP-DOWN DP WORKFLOW:

[SETUP]
1. Create a global 'memo' object/dictionary to act as our notepad.

[RECURSIVE FUNCTION: solve(state)]
   |
   |-- [THE MEMO CHECK]
   |   Have we already solved this exact 'state' before?
   |   (Does 'state' exist as a key in our 'memo'?)
   |   --> If YES: Stop right here! Return memo[state].
   |
   |-- [BASE CASES]
   |   Is this the smallest possible version of the problem?
   |   --> If YES: Return the obvious answer (e.g., if state == 0, return 0).
   |
   |-- [THE RECURSIVE WORK]
   |   We haven't seen this state, and it's not a base case.
   |   We must calculate it by calling solve() on smaller pieces.
   |   answer = perform logic using solve(state - 1), solve(state - 2), etc.
   |
   |-- [SAVE FOR LATER]
   |   Before returning the answer, write it down!
   |   memo[state] = answer
   |
   |-- Return answer.
```

---

3. How does it work? (Walkthrough)

Let's look at the classic "House Robber" problem to see how the notepad prunes a massive decision tree.

Scenario: The House Robber Decision Tree You are a thief. You can rob houses, but you cannot rob two *adjacent* houses or the alarm triggers. Houses have money: `[2, 7, 9, 3, 1]`. What is the max money you can steal?

At every house, you have two choices:

1. **Rob it:** You get its money, but you must skip the next house. Move to `Index + 2`.
2. **Skip it:** You get 0 money, but you can safely evaluate the very next house. Move to `Index + 1`.

Walkthrough WITHOUT Memoization (The Amnesia approach): To know the max money starting at House 0, you must explore branching paths.

```
Tree of choices (Nodes are the current House Index):
                        [0]
                      /     \
             Rob(0) /       \ Skip(0)
                   /         \
                 [2]         [1]
                /   \       /   \
               [4]  [3]    [3]   [2]
```

Notice the problem! We arrived at House `[2]` and House `[3]` multiple times through different timelines. The plain recursive algorithm will calculate the maximum profit from House `[3]` to the end twice. If the array had 100 houses, `[3]` would be recalculated millions of times!

Walkthrough WITH Memoization (The Top-Down approach): We add a `memo` dictionary. Keys are the House Index. Values are the Max Profit from that index to the end.

1. **Start [0]:** Check memo. Empty. Try Rob, try Skip.
2. **Path Rob -> [2]:** Check memo. Empty. (Eventually calculates max profit from index 2 to end is 10). *Action:* Save to notepad: `memo[2] = 10`. Return 10 up the chain.
3. **Path Skip -> [1]:** Check memo. Empty. Try Rob, try Skip.
4. **Path Skip -> [1] -> Rob -> [3]:** Check memo. Empty. (Eventually calculates profit from index 3 to end is 3). *Action:* Save to notepad: `memo[3] = 3`. Return 3 up the chain.
5. **Path Skip -> [1] -> Skip -> [2]:** *Action:* Check memo. Wait! `memo` has a key for `2`! It says `10`! *Magic:* We instantly return 10. We completely skip exploring the entire subtree under this node. The branch is "pruned".

---

4. Code Sample (JavaScript)

```javascript
/**
 * House Robber: Top-Down Dynamic Programming
 * @param {number[]} nums - Money at each house
 * @return {number} - Maximum profit
 */
function rob(nums) {
    // 1. Setup the notepad (Memo)
    // We use an array filled with -1 to represent "uncalculated" states.
    // A JavaScript Object {} or Map() works perfectly too!
    const memo = new Array(nums.length).fill(-1);

    // 2. Define the recursive helper function
    // 'currentIndex' represents our state. It tells us where we currently are.
    function dfs(currentIndex) {
        // [BASE CASE]
        // If we walked past the last house, there is no money left to steal.
        if (currentIndex >= nums.length) {
            return 0;
        }

        // [THE MEMO CHECK]
        // If the notepad has an answer for this index, use it instantly!
        if (memo[currentIndex] !== -1) {
            return memo[currentIndex];
        }

        // [THE RECURSIVE WORK]
        // Choice 1: Rob this house. We get its money, but must jump to index + 2
        const robChoice = nums[currentIndex] + dfs(currentIndex + 2);

        // Choice 2: Skip this house. We get 0 money, but can safely check index + 1
        const skipChoice = dfs(currentIndex + 1);

        // The best outcome for this specific state is the maximum of our two choices
        const maxProfit = Math.max(robChoice, skipChoice);

        // [SAVE FOR LATER]
        // Write the answer down in our notepad before returning!
        memo[currentIndex] = maxProfit;

        return maxProfit;
    }

    // 3. Kick off the recursion starting at the first house (index 0)
```

```
    return dfs(0);
}

// --- Usage ---
const houses = [2, 7, 9, 3, 1];
console.log(rob(houses));
// Output: 12 (Rob house 0 for 2, house 2 for 9, house 4 for 1)
```

---

5. LeetCode questions it applies to

- **Climbing Stairs (LeetCode 70):** You can take 1 step or 2 steps. How many ways to reach the top? The state is your `currentStep`. You return `dfs(currentStep + 1) + dfs(currentStep + 2)`.
- **Coin Change (LeetCode 322):** Find the minimum coins to make an amount. The state is `currentAmount`. You loop through all coins, doing `1 + dfs(currentAmount - coin)`. You memoize the `currentAmount` to avoid re-calculating the same remaining change repeatedly.
- **Word Break (LeetCode 139):** Can a string be segmented into dictionary words? The state is `startIndex`. You slice the string, and if a slice is in the dictionary, you recursively call `dfs(newStartIndex)`. Memoize the index so you don't re-check the same suffix of the string.
- **Longest Common Subsequence (LeetCode 1143):** Find the longest matching sequence between two strings. State requires TWO variables: `index1` for string A, and `index2` for string B. See Variation A below.

---

6. Time and Space Complexity

To calculate Top-Down DP complexity, use this golden formula: `Time = (Number of Unique States) * (Time spent computing each State once)`

```
Time Complexity Derivation (House Robber):
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Unique States          | N houses (indices)   | N
2. Work per State         | Math.max(a, b)       | O(1)
-----------------------------------------------------------
Total Time Complexity: N states * O(1) work = O(N)
Without memoization, it would branch into O(2^N). The memo
flattens the branching tree into a straight line of unique work.

Space Complexity Derivation:
-----------------------------------------------------------
```

```
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Memoization Array      | Stores N answers     | O(N)
2. Recursion Call Stack   | Goes N levels deep   | O(N)
-------------------------------------------------------------
Total Space Complexity: O(N) + O(N) = O(N)
```

---

7. Variations and Common Tweaks

Variation A: 2D Memoization (Multiple State Variables) **The Problem:** Sometimes, a single number isn't enough to describe a "State". For the "Longest Common Subsequence" problem, you are comparing two separate strings (`text1` and `text2`). Your state needs to know where you are in *both* strings simultaneously. **The Tweak:** Your `memo` notepad changes from a simple 1D array to a 2D matrix (or a Hash Map using a combined string key).

Snippet for 2D Memoization:

```javascript
// Setup a 2D Array filled with -1
const memo = Array(text1.length).fill(-1).map(() => Array(text2.length).fill(-1));

function dfs(i, j) {
    // Base cases (reached end of either string)
    if (i === text1.length || j === text2.length) return 0;

    // Check 2D memo
    if (memo[i][j] !== -1) return memo[i][j];

    // Recursive logic...
    let answer;
    if (text1[i] === text2[j]) {
        answer = 1 + dfs(i + 1, j + 1);
    } else {
        answer = Math.max(dfs(i + 1, j), dfs(i, j + 1));
    }

    // Save to 2D memo
    memo[i][j] = answer;
    return answer;
}
```

Variation B: String Keys for Complex States **The Problem:** You have 3 or 4 variables defining your state, or negative numbers are involved, making standard arrays impossible or very messy to use for a cache. **The Tweak:** Stop using Arrays for your `memo`. Use a standard dictionary/Hash

Map, and combine your state variables into a single unique string key (e.g., `"index,amountLeft,canRob"`).

Snippet for String Key Memo:

```
const memo = new Map();

function dfs(index, capacityLeft) {
    // Create a unique key for this specific combination of variables
    const key = index + "," + capacityLeft;

    if (memo.has(key)) {
        return memo.get(key);
    }

    // ... compute answer ...

    memo.set(key, answer);
    return answer;
}
```

Converting Top Down to Bottom Up / Tabulation Dynamic Programming

Here is a complete guide to understanding Tabulation (Bottom-Up Dynamic Programming) and exactly how to translate your Top-Down recursive thoughts into blazing-fast iterative tables.

---

1. Problem it solves and Why we need it

The Problem: Top-Down DP (Memoization) is incredibly intuitive because you just write a recursive function and add a notepad cache to it. However, it relies heavily on the computer's "Call Stack". If you have a problem that requires 10,000 recursive steps, your program will crash with a "Stack Overflow" error before it even gets to use the notepad. Furthermore, spinning up thousands of function calls carries a hidden performance cost in memory and time.

Why we need it: Bottom-Up DP (Tabulation) completely eliminates recursion. It solves the "Stack Overflow" problem and removes the function call overhead. Instead of starting at the massive final goal and digging down to find the base cases, it starts by solving the tiny base cases first, and uses `for` loops to build those answers up into the final goal.

---

2. What is it? (The Translation Guide)

Tabulation is the exact same logic as Memoization, just written in reverse using an Array (or Matrix) and loops instead of functions.

Here is the ultimate cheat sheet on how to translate your Top-Down thoughts into Bottom-Up code:

1. **State Variables = Array Dimensions:** Look at the arguments of your recursive `dfs()` function.

- 1 argument (e.g., `dfs(index)`) -> You need a 1D Array (`dp[i]`).
- 2 arguments (e.g., `dfs(index, weight)`) -> You need a 2D Matrix (`dp[i][j]`).

2. **Indices / Array Size:** Look at the maximum possible value of those arguments. If your `index` goes from 0 to N, your array must be size `N + 1` so it has enough room to store the answer for N.
3. **Initial Values = Base Cases:** Look at the `if` statements at the very top of your `dfs()` function.

- Top-Down: `if (index === 0) return 0;`
- Bottom-Up: `dp[0] = 0;`

4. **The Recurrence Relation (The Loop):** The main body of your `dfs()` function becomes the inside of your `for` loop. Every recursive call `dfs(...)` is literally just replaced with an array lookup `dp[...]`.

```
TABULATION WORKFLOW (Language Agnostic):

[SETUP]
1. Create a DP table (Array/Matrix) of the correct size.
2. Fill the starting indices with the Base Case answers.

[MAIN LOOP]
FOR i from BaseCaseIndex to FinalGoal:
   |
   |-- Calculate current answer by looking BACKWARDS at the table
   |   Example: dp[i] = dp[i - 1] + dp[i - 2]
   |
[END]
Return the last slot in the table (e.g., dp[N])
```

---

3. How does it work? (Walkthrough)

Let's convert the classic "Climbing Stairs" problem. You can climb 1 or 2 steps at a time. How many ways can you reach step N?

Scenario A: The 1D Conversion (Climbing Stairs) Goal: Reach step 4.

Top-Down Thought Process:

- State: `currentStep` (1 argument = 1D Array needed).
- Base Cases: `dfs(0) = 1` (1 way to stand on the ground), `dfs(1) = 1` (1 way to reach step 1).
- Logic: `dfs(i) = dfs(i - 1) + dfs(i - 2)`.

Bottom-Up Execution: Create an array `dp` of size 5 (indices 0 through 4). Pre-fill base cases: `dp[0] = 1`, `dp[1] = 1`.

```
Step 1: Loop i = 2
dp[2] = dp[1] + dp[0]  -> 1 + 1 = 2
Array: [1, 1, 2, empty, empty]

Step 2: Loop i = 3
dp[3] = dp[2] + dp[1]  -> 2 + 1 = 3
Array: [1, 1, 2, 3, empty]

Step 3: Loop i = 4
dp[4] = dp[3] + dp[2]  -> 3 + 2 = 5
Array: [1, 1, 2, 3, 5]
```

Done! We return `dp[4]`, which is 5. We built it from the ground up without a single recursive call.

Scenario B: The 2D Conversion (Longest Common Subsequence) Goal: Find the longest shared sequence between "ABC" and "AC".

Top-Down Thought Process:

- State: `index1` (for string A), `index2` (for string B). (2 arguments = 2D Matrix needed).
- Base Cases: If either string is empty (index is 0), the shared length is 0.
- Logic: If letters match, add 1 and move both indices diagonally back. If they don't match, take the max of moving index1 back OR moving index2 back.

Bottom-Up Execution: We build a grid. Rows are letters of "ABC", columns are letters of "AC". We add an extra "Empty" row/col at index 0 for our base cases (filled with 0s).

```
Grid Setup (Empty Strings = 0):
      Empty   A   C
Empty   0     0   0
  A     0     ?   ?
  B     0     ?   ?
  C     0     ?   ?
```

We loop through the grid left-to-right, top-to-bottom.

- **Row A, Col A:** Letters match! Look diagonally up-left (0) and add 1. `dp[1][1] = 1`.
- **Row A, Col C:** No match. Look Left (0) and Up (0). Max is 0. `dp[1][2] = 0`.
- **Row B, Col A:** No match. Look Left (0) and Up (1). Max is 1. `dp[2][1] = 1`.
- **Row C, Col C:** Letters match! Look diagonally up-left (which is Row B, Col A = 1) and add 1. `dp[3][2] = 2`.

Final Answer is at the very bottom right: `dp[3][2]` is 2.

---

4. Code Sample (JavaScript)

Here is the exact Bottom-Up translation for the "House Robber" problem.

```
/**
 * House Robber: Bottom-Up Dynamic Programming (Tabulation)
 * @param {number[]} nums - Money at each house
 * @return {number} - Maximum profit
 */
function robBottomUp(nums) {
    // Edge case checks
    if (nums.length === 0) return 0;
    if (nums.length === 1) return nums[0];

    // 1. Array Dimensions & Size:
    // We only need 1 state variable (the house index).
    // Array needs to be the same length as the houses.
    const dp = new Array(nums.length).fill(0);

    // 2. Initial Values (Base Cases):
    // If there is only 1 house (index 0), max profit is robbing it.
    dp[0] = nums[0];

    // If there are 2 houses (index 1), max profit is picking the richer one.
    dp[1] = Math.max(nums[0], nums[1]);

    // 3. The Recurrence Relation (Main Loop):
    // Start at index 2, since 0 and 1 are already solved.
    for (let i = 2; i < nums.length; i++) {
        // Just like recursion, we have two choices:
        // Choice 1: Rob this house (nums[i]) AND add the best profit from 2 houses ago (dp
        const robChoice = nums[i] + dp[i - 2];
```

```
        // Choice 2: Skip this house, and just keep the best profit from the previous house
        const skipChoice = dp[i - 1];

        // Record the best choice for this specific subproblem
        dp[i] = Math.max(robChoice, skipChoice);
    }

    // 4. Return the final goal
    // The last slot in the array holds the answer for the entire problem
    return dp[nums.length - 1];
}

// --- Usage ---
const houses = [2, 7, 9, 3, 1];
console.log(robBottomUp(houses));
// Output: 12 (Rob house 0 for 2, house 2 for 9, house 4 for 1)
```

---

5. LeetCode questions it applies to

- **Min Cost Climbing Stairs (LeetCode 746):** Create a 1D `dp` array. Base cases: `dp[0] = 0` and `dp[1] = 0` (you can start at either step for free). The loop is `dp[i] = cost[i] + Math.min(dp[i-1], dp[i-2])`.
- **Coin Change (LeetCode 322):** Create a 1D `dp` array of size `amount + 1`. Fill it with `Infinity`. Base case: `dp[0] = 0` (takes 0 coins to make 0 change). Loop through amounts 1 to N, checking every coin: `dp[amount] = Math.min(dp[amount], 1 + dp[amount - coin])`.
- **Unique Paths (LeetCode 62):** A classic 2D problem. Create an `M x N` grid. Base cases: Fill the entire top row and left column with 1s (only one way to move in a straight line). Nested loops fill the rest: `dp[row][col] = dp[row-1][col] + dp[row][col-1]`.
- **Longest Common Subsequence (LeetCode 1143):** Create an `(N+1) x (M+1)` 2D matrix filled with 0s. The nested loops compare characters and populate the grid exactly as shown in Walkthrough Scenario B.

---

6. Time and Space Complexity

The complexities for Tabulation are generally identical to Memoization, but without the hidden overhead of the Call Stack.

```
Time Complexity Derivation (House Robber):
-------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Initialize DP Array    | N slots              | O(N)
```

```
2. Main Loop             | Loops N times        | O(N)
3. Work inside loop      | Math.max lookup      | O(1)
-------------------------------------------------------
Total Time Complexity: O(N)
We iterate through the array exactly once.

Space Complexity Derivation:
-------------------------------------------------------
Task                     | Memory Used          | Big O
-------------------------|----------------------|--------
1. DP Array              | Stores N items       | O(N)
-------------------------------------------------------
Total Space Complexity: O(N)
*Note: Because there is no recursion, the Call Stack takes
O(1) space, making this strictly better than Top-Down in
real-world memory usage.
```

---

7. Variations and Common Tweaks

Variation A: Space Optimization (The Two-Variable Trick) The Problem: Look closely at the Fibonacci or House Robber loop: `dp[i] = Math.max(nums[i] + dp[i-2], dp[i-1])`. To calculate step `i`, we ONLY need the answers from step `i-1` and step `i-2`. We never look at step `i-3` or `i-100`. So why are we storing the entire `O(N)` array in memory? **The Tweak:** We completely delete the array. We use just two basic variables to hold the "previous" and "two steps back" answers. As we loop, we calculate the new answer, shift the variables forward, and discard the old data. This reduces Space Complexity from **O(N)** down to pure **O(1)**!

Snippet for O(1) Space House Robber:

```javascript
function robOptimized(nums) {
    if (nums.length === 0) return 0;

    let twoStepsBack = 0; // Represents dp[i-2]
    let oneStepBack = 0;  // Represents dp[i-1]

    for (let i = 0; i < nums.length; i++) {
        // Calculate current answer
        let currentChoice = Math.max(nums[i] + twoStepsBack, oneStepBack);

        // Shift our "sliding window" of memory forward for the next loop
        twoStepsBack = oneStepBack;
        oneStepBack = currentChoice;
    }
```

```
    // oneStepBack holds the final answer
    return oneStepBack;
}
```

Backtracking

Here is a complete guide to understanding and implementing the powerful Backtracking algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you are trying to crack a 3-digit combination lock, or you are trying to place 8 queens on a chessboard so that none of them attack each other. You need a systematic way to try different combinations without getting lost, missing any, or getting stuck in an infinite loop.

Why the "Naive" Brute Force fails: Pure brute force generates *every single possible combination* of the board or lock, and only after generating them does it check if they are valid. This is incredibly wasteful. If you place your first two queens and they are already attacking each other, there is zero reason to place the other 6 queens.

Why we need Backtracking: Backtracking is "smart" brute force. It builds a solution step-by-step. The absolute moment it realizes a sequence of choices violates the rules, it stops, takes a step back (backtracks), and tries the next available option. This ability to abandon bad paths early is called **Pruning**, and it saves massive amounts of computation time.

---

2. What is it? (Pseudocode & Flow)

Backtracking is heavily reliant on Depth-First Search (DFS) and Recursion.

Its entire philosophy can be summarized in three steps: **Choose, Explore, Un-Choose.** You make a choice, you recursively dive deeper to see where that choice leads, and then you *undo* that choice so your array/state is clean and ready to try the next option.

```
BACKTRACKING WORKFLOW (Language Agnostic):

function backtrack(current_path, options):
   |
   |-- [BASE CASE / SUCCESS CHECK]
   |   Does 'current_path' meet the final goal?
   |   --> If YES: Save a copy of 'current_path' to our results!
```

```
|               Return (turn back).
|
|-- [PRUNING / FAILURE CHECK] (Optional but powerful)
|   Has 'current_path' broken a rule?
|   --> If YES: Return immediately! (Dead end).
|
|-- [THE CHOICE LOOP]
|   For every available option in 'options':
      |
      |-- 1. CHOOSE: Add the option to 'current_path'.
      |              Mark option as used (if necessary).
      |
      |-- 2. EXPLORE: Call backtrack(current_path, updated_options).
      |               (Dive deep into this timeline).
      |
      |-- 3. UN-CHOOSE: Remove the option from 'current_path'.
      |                 Mark option as unused.
      |                 (This resets our state so the next loop iteration is clean!)
```

---

3. How does it work? (Walkthrough)

Let's look at the classic "Permutations" problem. We have an array `[1, 2, 3]` and we want to find all possible orderings of these numbers.

Scenario: Permutations of [1, 2, 3] To prevent reusing numbers, we will use a `used` checklist.

Step 0: Setup

- Array: `[1, 2, 3]`
- Path: `[]`
- Used: `[false, false, false]`

Step 1: The First Choice (Level 0) The loop looks at 1, 2, and 3.

- **Choose 1:** Path becomes `[1]`. `used[0]` is true.
- **Explore:** We dive into Level 1.

Step 2: Building the path (Level 1) The loop looks at 1, 2, and 3.

- 1 is used! Skip it.
- **Choose 2:** Path becomes `[1, 2]`. `used[1]` is true.
- **Explore:** We dive into Level 2.

Step 3: Hitting the Base Case (Level 2) The loop looks at 1, 2, and 3.

- 1 is used. 2 is used.
- **Choose 3:** Path becomes `[1, 2, 3]`. `used[2]` is true.

- **Explore:** We dive into Level 3.
- **BASE CASE TRIGGERED:** Path length is 3! We save `[1, 2, 3]` to our final answers. Return!

Step 4: The UN-CHOOSE (The Backtrack) We just returned to Level 2. The code execution picks up exactly where it left off: the Un-Choose step!

- **Un-choose 3:** Path becomes `[1, 2]`. `used[2]` becomes false.
- Level 2 loop finishes. Return to Level 1.
- **Un-choose 2:** Path becomes `[1]`. `used[1]` becomes false.
- Level 1 loop moves to its next option: 3!
- **Choose 3:** Path becomes `[1, 3]`. `used[2]` is true.
- **Explore:** Dive into Level 2. 1 is used, 3 is used. It will choose 2, hitting the base case to save `[1, 3, 2]`.

```
Visualizing the Decision Tree (Left side only):

                    [ ]
                  /  |  \
            Choose 1 |   Choose 3...
                /    |
             [1]  Choose 2...
            /   \
    Choose 2     Choose 3
      /            \
   [1,2]          [1,3]
    |               |
 Choose 3        Choose 2
    |               |
 [1,2,3]         [1,3,2]
 (SAVE!)         (SAVE!)
   |               |
 BACKTRACK!      BACKTRACK!
```

Notice how `current_path` acts like a single, reusable piece of clay. We mold it, save a snapshot of it when it looks right, and then smash it back into shape to try molding something else.

---

4. Code Sample (JavaScript)

Here is the exact code for generating Permutations.

```
/**
 * Backtracking algorithm to find all permutations of an array
 * @param {number[]} nums - The input array (e.g., [1, 2, 3])
 * @return {number[][]} - Array of all permutations
 */
```

```
function permute(nums) {
    const results = [];
    const currentPath = [];

    // Tracks which numbers are currently in our path so we don't reuse them
    // Example: used[0] = true means nums[0] is in currentPath
    const used = new Array(nums.length).fill(false);

    function backtrack() {
        // [BASE CASE]
        // If our path is the same length as the input, we found a valid permutation!
        if (currentPath.length === nums.length) {
            // We MUST push a COPY of the array. If we push currentPath directly,
            // the Un-choose steps will erase the answers we saved!
            results.push([...currentPath]);
            return;
        }

        // [THE CHOICE LOOP]
        // Iterate through all possible options
        for (let i = 0; i < nums.length; i++) {

            // If the number is already in our path, skip it!
            if (used[i]) {
                continue;
            }

            // 1. CHOOSE
            currentPath.push(nums[i]);
            used[i] = true;

            // 2. EXPLORE
            // Dive deeper to pick the next number in the sequence
            backtrack();

            // 3. UN-CHOOSE (Backtrack)
            // We just returned from the deep dive. Remove the number we
            // added so the next loop iteration starts with a clean slate.
            currentPath.pop();
            used[i] = false;
        }
    }

    // Kick off the algorithm
    backtrack();
    return results;
```

```
}

// --- Usage ---
console.log(permute([1, 2, 3]));
// Output: [ [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1] ]
```

---

5. LeetCode questions it applies to

Backtracking is the universal key to the "Combinatorics" category on LeetCode.

- **Subsets (LeetCode 78):** Generate all possible subsets of an array. The Choice Loop changes: instead of picking *any* number, you only pick numbers that come *after* your current index to avoid duplicates. The Base Case changes: you save the path at *every single step*, not just at the end.
- **Combination Sum (LeetCode 39):** You are given a target number (e.g., 7) and an array of numbers. Find all combinations that add up to 7. You pass a `remainingSum` variable down the recursion. **The Pruning Step:** If `remainingSum < 0`, you instantly return (stop searching). If it hits 0, you save the path!
- **N-Queens (LeetCode 51):** The ultimate Backtracking problem. You place a queen on row 0. Before placing a queen on row 1, you check columns and diagonals to see if it's safe. If it is, you choose, explore, and un-choose. If it isn't safe, you skip that square.

---

6. Time and Space Complexity

Because Backtracking explores all possible combinations, it is fundamentally an exponential or factorial time algorithm.

```
Time Complexity Derivation (For Permutations):
-------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Pick 1st number        | N options            | N
2. Pick 2nd number        | N - 1 options        | N - 1
3. Pick 3rd number        | N - 2 options        | N - 2
4. Array Copy (Base Case)| Copy N elements      | O(N)
-------------------------------------------------------
Total Time Complexity: O(N * N!)
We generate N! (N factorial) complete paths, and for each
completed path, we take O(N) time to copy the array into
our results.

(Note: For 'Subsets', the time is O(N * 2^N) because every
```

```
element has 2 choices: be included, or not be included).

Space Complexity Derivation:
-------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Current Path Array     | Stores up to N items | O(N)
2. Used Checklist Array   | Stores N booleans    | O(N)
3. Recursion Call Stack   | Goes N levels deep   | O(N)
-------------------------------------------------------
Total Space Complexity: O(N)
*This ignores the final 'results' array which holds the
answers, as output space is often excluded from auxiliary
space complexity.
```

---

7. Variations and Common Tweaks

Variation A: Subsets / Combinations (Order doesn't matter) **The Problem:** In Permutations, `[1, 2]` and `[2, 1]` are different answers. In Combinations or Subsets, they are considered the exact same thing. We need to prevent the algorithm from looking backwards. **The Tweak:** We pass a `startIndex` into our backtrack function. The `for` loop no longer starts at `0`; it starts at `startIndex`. When we dive deeper, we pass `i + 1` as the new start index. This forces the algorithm to only look strictly forward!

Snippet for Combinations/Subsets:

```javascript
// Add startIndex to the parameters
function backtrack(startIndex) {
    // Save every state!
    results.push([...currentPath]);

    // TWEAK: Loop starts at startIndex, not 0!
    for (let i = startIndex; i < nums.length; i++) {
        currentPath.push(nums[i]);

        // TWEAK: Pass i + 1 to force the next level to only look forward
        backtrack(i + 1);

        currentPath.pop();
    }
}
backtrack(0); // Kick off at index 0
```

Variation B: Handling Duplicate Inputs (Permutations II / Subsets II) **The Problem:** The input array has duplicates, like `[1, 2, 2]`. Standard backtracking will treat the two 2s as different numbers and generate identical, duplicate answers in your final result. **The Tweak:** You must **Sort** the input array first so duplicates are sitting next to each other. Then, inside the `for` loop, you add a check: "If this number is the exact same as the previous number, AND the previous number was just un-chosen/skipped, skip this one too!"

Snippet for Duplicate Handling:

```
// MUST sort first!
nums.sort((a, b) => a - b);

function backtrack() {
    // ... base case ...

    for (let i = 0; i < nums.length; i++) {
        if (used[i]) continue;

        // THE TWEAK: Skip duplicates at the same tree level
        // (i > 0) prevents out of bounds errors
        // (nums[i] === nums[i-1]) checks if it's a duplicate
        // (!used[i-1]) proves the previous duplicate is currently UN-CHOSEN,
        // meaning we are at the same horizontal level of the tree.
        if (i > 0 && nums[i] === nums[i - 1] && !used[i - 1]) {
            continue;
        }

        // ... Choose, Explore, Un-choose ...
    }
}
```

Minimax Algorithm (Zero Sum)

Here is a complete guide to understanding and implementing the **Minimax Algorithm** (the brain behind Zero-Sum Games).

---

1. Problem it solves and Why we need it

The Problem: Imagine you are building an AI to play Tic-Tac-Toe, Chess, or Connect 4. How does the computer know which move to make? If it just picks a move that looks good *right now* (a greedy approach), the human player might set a trap and win on the next turn.

Why we need it: In a "Zero-Sum" game, one player's win is exactly equal

to the other player's loss (+1 for me means -1 for you). We need an algorithm that can look into the future, map out every possible consequence of a move, and safely assume the opponent is going to play perfectly to ruin our day.

Minimax simulates the entire game out to the very end. It allows the AI to say: "If I do A, the opponent will do B to hurt me, but then I can do C and guarantee a win." It guarantees the **best worst-case scenario**.

---

2. What is it? (Pseudocode & Flow)

Minimax is a **Backtracking (Recursive) Algorithm**.

It divides the game into two perspectives that alternate every single turn:

1. **The Maximizer (The AI):** Wants to get the highest possible score (+10).
2. **The Minimizer (The Human):** Wants to give the AI the lowest possible score (-10).

The algorithm dives all the way down to the end of the game (a win, loss, or draw). It scores that final board state, and then bubbles those scores back up to the top.

```
MINIMAX ALGORITHM WORKFLOW:

function minimax(gameState, depth, isMaximizingPlayer):
   |
   |-- [BASE CASE]
   |   Is the game over? (Someone won, or it's a draw, or we hit a depth limit)
   |   --> If YES: Return the static score of the board (+10, -10, or 0).
   |
   |-- [THE MAXIMIZER'S TURN]
   |   If isMaximizingPlayer == TRUE:
   |      bestScore = -Infinity
   |      For every possible move:
   |         1. Make the move.
   |         2. score = minimax(newState, depth + 1, FALSE) <-- Pass turn to Minimizer!
   |         3. Undo the move.
   |         4. bestScore = Maximum of (bestScore, score)
   |      Return bestScore
   |
   |-- [THE MINIMIZER'S TURN]
   |   If isMaximizingPlayer == FALSE:
   |      bestScore = +Infinity
   |      For every possible move:
   |         1. Make the move.
   |         2. score = minimax(newState, depth + 1, TRUE) <-- Pass turn to Maximizer!
```

```
|         3. Undo the move.
|         4. bestScore = Minimum of (bestScore, score)
|      Return bestScore
```

---

3. How does it work? (Walkthrough)

Let's trace a simple scenario. The AI (Maximizer) has two choices. After the AI chooses, the Human (Minimizer) has two choices.

Scenario: The Choice Tree

- AI wants positive numbers (Right side of the number line).
- Human wants negative numbers (Left side of the number line).

```
Level 0 (Maximizer - AI's Turn)
                      [ ? ]  <-- What move should AI make?
                     /     \
Move:               Left    Right
                    /         \

Level 1 (Minimizer - Human's Turn)
                [ ? ]       [ ? ]
               /     \     /     \
Move:         L       R   L       R

Level 2 (Terminal States - Game Over!)
Scores:     (+10)   (-5) (+8)   (+12)
```

Walkthrough: Bubbling Up the Scores

Step 1: Evaluate the Minimizer's choices (Level 1) We look at the bottom of the tree. It is the Human's turn. The human wants the *smallest* possible number.

- **Look at the Left branch:** The human can choose between +10 and -5. They will obviously choose **-5**. (So, if AI goes Left, AI gets -5).
- **Look at the Right branch:** The human can choose between +8 and +12. They will obviously choose **+8**. (So, if AI goes Right, AI gets +8).

```
The tree now looks like this:

Level 0 (Maximizer - AI)
                      [ ? ]
                     /     \
Level 1          [-5]       [+8]   <-- Minimizer's guaranteed choices
```

Step 2: Evaluate the Maximizer's choices (Level 0) Now we are back at the top. It is the AI's turn. The AI wants the *largest* possible number.

- The AI looks at going Left (guarantees a score of -5).
- The AI looks at going Right (guarantees a score of +8).
- The AI chooses **Right**! The final score bubbled to the top is +8.

The Magic: The AI didn't blindly go Left just because it saw a +10 down there. It knew the human would never let it have the +10 and would force it into the -5 trap. It chose the safest, most optimal path.

---

4. Code Sample (JavaScript)

```
/**
 * A generalized Minimax algorithm template.
 * @param {Object} state - The current state of the game board
 * @param {boolean} isMaximizing - True if it's the AI's turn, false if Human
 * @return {number} - The optimal score for this branch
 */
function minimax(state, isMaximizing) {
    // 1. Base Case: Check if the game is over
    if (isGameOver(state)) {
        return evaluateBoard(state); // Returns +10 for AI win, -10 for Human win, 0 for dr
    }

    // 2. Maximizer's Turn (The AI)
    if (isMaximizing) {
        let bestScore = -Infinity;
        const availableMoves = getAvailableMoves(state);

        for (let i = 0; i < availableMoves.length; i++) {
            // Choose
            makeMove(state, availableMoves[i], "AI");

            // Explore (Notice we flip isMaximizing to FALSE)
            let score = minimax(state, false);

            // Un-choose (Backtrack)
            undoMove(state, availableMoves[i]);

            // Maximize the score
            bestScore = Math.max(bestScore, score);
        }
        return bestScore;
    }

    // 3. Minimizer's Turn (The Human)
    else {
```

```
        let bestScore = Infinity;
        const availableMoves = getAvailableMoves(state);

        for (let i = 0; i < availableMoves.length; i++) {
            // Choose
            makeMove(state, availableMoves[i], "HUMAN");

            // Explore (Notice we flip isMaximizing to TRUE)
            let score = minimax(state, true);

            // Un-choose (Backtrack)
            undoMove(state, availableMoves[i]);

            // Minimize the score
            bestScore = Math.min(bestScore, score);
        }
        return bestScore;
    }
}

// Helper to actually get the best move (not just the score)
function findBestMove(initialState) {
    let bestScore = -Infinity;
    let bestMove = null;
    const moves = getAvailableMoves(initialState);

    for (let i = 0; i < moves.length; i++) {
        makeMove(initialState, moves[i], "AI");
        // AI just went, so next turn is Minimizer (false)
        let score = minimax(initialState, false);
        undoMove(initialState, moves[i]);

        if (score > bestScore) {
            bestScore = score;
            bestMove = moves[i];
        }
    }
    return bestMove;
}
```

---

5. LeetCode questions it applies to

- **Predict the Winner (LeetCode 486):** You have an array of numbers. Players take turns picking numbers from either end of the array. The goal

is to have the most points. This is pure Minimax. The state is the array indices (left and right), and players alternate trying to maximize their score minus the opponent's score.

- **Stone Game (LeetCode 877):** Almost identical to "Predict the Winner". (Though there is a math trick to solve it in O(1) time, solving it with Minimax is expected in an interview setting to prove you understand game theory).
- **Guess Number Higher or Lower II (LeetCode 375):** You are guessing a number. Every time you guess wrong, you pay the cost of your guess. You want to guarantee a win while spending the minimum amount of money. The Minimizer is the game forcing you to pay the highest cost; the Maximizer is you trying to minimize that worst-case cost.

---

6. Time and Space Complexity

Let **b** equal the Branching Factor (how many legal moves you can make on an average turn). Let **d** equal the Maximum Depth (how many turns until the game ends).

```
Time Complexity Derivation:
-----------------------------------------------------------
Task                      | Operations           | Big O
--------------------------|----------------------|--------
1. Make a move            | Try 'b' moves        | b
2. Opponent responds      | Tries 'b' moves      | b * b
3. Game continues         | Loops 'd' levels deep| b * b * b...
-----------------------------------------------------------
Total Time Complexity: O(b^d)
Minimax checks every single possible reality. If you have
3 choices per turn, and a game lasts 10 turns, that is
3^10 = 59,049 board states to check. (For Chess, this number
is larger than the atoms in the universe, which is why we
use the tweaks below!)

Space Complexity Derivation:
-----------------------------------------------------------
Task                      | Memory Used          | Big O
--------------------------|----------------------|--------
1. Recursion Call Stack   | Goes 'd' levels deep | O(d)
2. Board State Copies     | Handled in-place     | O(1)
-----------------------------------------------------------
Total Space Complexity: O(d)
Because it is a Depth-First Search, it only needs memory
for the current branch it is exploring all the way down.
```

---

7. Variations and Common Tweaks

Variation A: Alpha-Beta Pruning (The Must-Know Optimization)
The Problem: Minimax is painfully slow because it calculates branches that it doesn't need to. In our example above, if the AI already knows it can get a guaranteed `+8` by going Right, and it starts exploring the Left path and instantly sees a trap worth `-100`, it *shouldn't bother exploring the rest of the Left path.* It already knows Left is worse than Right. **The Tweak:** We pass two variables down the tree: `alpha` (the best score the Maximizer has found so far) and `beta` (the best score the Minimizer has found so far). If we are exploring a branch and realize `beta <= alpha`, it means the opponent will never let us go down this branch anyway. We immediately `break` out of the loop!

Snippet for Alpha-Beta Pruning:

```
// Function signature adds alpha and beta
function minimaxAlphaBeta(state, depth, alpha, beta, isMaximizing) {
    // ... base cases ...

    if (isMaximizing) {
        let maxEval = -Infinity;
        for (let move of availableMoves) {
            // ... make move ...
            let eval = minimaxAlphaBeta(state, depth + 1, alpha, beta, false);
            // ... undo move ...

            maxEval = Math.max(maxEval, eval);
            alpha = Math.max(alpha, eval); // Update Alpha!

            // THE PRUNING TWEAK:
            if (beta <= alpha) {
                break; // Stop evaluating! This branch is useless.
            }
        }
        return maxEval;
    }
    // ... minimization logic acts symmetrically, updating beta ...
}
// Kick off with: minimaxAlphaBeta(initialState, 0, -Infinity, Infinity, true)
```

Variation B: Depth-Limited Minimax (For Chess / Complex Games)
The Problem: For games like Chess, reaching the true end of the game (`depth = 100+`) takes billions of years. **The Tweak:** You enforce a hard depth limit. If `depth === 5`, you pretend the game is over. Since nobody actually won, you use a **Heuristic Evaluation Function** to guess who is currently winning. For

example, in Chess, you count the pieces on the board (Queen = 9, Rook = 5). If the AI has more pieces, it returns a positive number.

Snippet for Depth-Limited:

```
function minimax(state, depth, isMaximizing) {
    // TWEAK: Stop early!
    if (depth === MAX_DEPTH || isGameOver(state)) {
        // TWEAK: Use an educated guess instead of a hard Win/Loss
        return heuristicEvaluation(state);
    }
    // ... continue standard minimax ...
}
```

Fair Share Distribution Algorithm

Here is a complete guide to understanding and implementing the Fair Share Distribution algorithm.

---

1. Problem it solves and Why we need it

The Problem: Imagine you have a large pile of discrete items (like 10,000,000 pennies, or network requests) and you need to distribute them among a specific number of people or servers. Sometimes you want everyone to get the exact same amount. Other times, the rules dictate that the first person gets 1 item, the second gets 2, the third gets 3, and so on, looping around until you run out.

Why the "Naive" Approach fails: The most obvious way to distribute items is to write a loop: give 1 item to the first person, 1 to the second, 1 to the third, and keep looping over the people until your pile hits zero. If you have N items, this takes O(N) time. If N is 10,000,000,000, your program will have to run 10 billion loop iterations just to hand out the items. This is far too slow!

Why we need it: The Fair Share Distribution algorithm uses modulo arithmetic and basic division to distribute millions of items instantly. It skips the massive loops and calculates exactly what each person's final pile will look like in mathematically constant time, or at worst, proportional to the number of *people* (O(K)), rather than the number of *items* (O(N)).

---

2. What is it?

Fair Share Distribution is a math-based approach to allocation. It relies on two fundamental operations:

1. **Division (The Base Share):** If you divide the total items by the number of recipients, the whole number result tells you the "baseline" that every single person is guaranteed to receive.
2. **Modulo / Remainder (The Leftovers):** If the items don't divide perfectly evenly, the remainder tells you exactly how many leftover items need to be handed out one-by-one to the first few people in line.

---

3. How does it work? (Walkthrough)

Let's walk through the standard uniform distribution, and then a more complex progressive distribution.

Scenario A: The Standard Uniform Distribution **Input:** 14 Candies, 4 People.

We want to give everyone an equal share. If there are leftovers, we hand them out one-by-one starting from Person 1.

Step 1: The Math

- Base Share = Math.floor(14 / 4) = 3
- Leftovers (Remainder) = 14 % 4 = 2

Step 2: The Allocation

- Everyone gets the Base Share of 3.
- The Leftovers (2) are given to the first 2 people.

```
People:        [ P1,  P2,  P3,  P4 ]
-------------------------------------
Base Share:    [  3,   3,   3,   3 ]
Leftovers:     [ +1,  +1,   0,   0 ]
-------------------------------------
Final Share:   [  4,   4,   3,   3 ]
```

We did this instantly without looping 14 times!

Scenario B: The Progressive Distribution (The LeetCode Classic) This is a famous interview scenario. You have a pool of items. You give 1 to the first person, 2 to the second, 3 to the third… When you reach the last person, you loop back to the first person and give them the next number in the sequence. You stop when you run out of items (the last person gets whatever is left).

Input: 10 Candies, 3 People.

Let's trace what the naive loop does first:

```
Round 1:
P1 needs 1. Give 1. (Remaining: 9)
P2 needs 2. Give 2. (Remaining: 7)
P3 needs 3. Give 3. (Remaining: 4)

Round 2:
P1 needs 4. Give 4. (Remaining: 0)
```

Final Arrays: P1 gets (1+4)=5, P2 gets 2, P3 gets 3. `[5, 2, 3]`.

How to do this algorithmically without looping N times: We use a while loop, but we jump by the *request size*, not by 1.

- Setup: `ans = [0, 0, 0]`, `candies = 10`, `giveAmount = 1`, `index = 0`
- **Step 1:** Give `giveAmount` (1) to `index 0`.
- `ans = [1, 0, 0]`. Candies = 9. `giveAmount` becomes 2. `index` becomes 1.
- **Step 2:** Give `giveAmount` (2) to `index 1`.
- `ans = [1, 2, 0]`. Candies = 7. `giveAmount` becomes 3. `index` becomes 2.
- **Step 3:** Give `giveAmount` (3) to `index 2`.
- `ans = [1, 2, 3]`. Candies = 4. `giveAmount` becomes 4. `index` becomes 0 (wrapped around using modulo!).
- **Step 4:** Give `giveAmount` (4) to `index 0`.
- `ans = [5, 2, 3]`. Candies = 0. `giveAmount` becomes 5. `index` becomes 1.
- Stop! Candies are 0.

---

4. Code Sample (JavaScript)

Here is the code for the Progressive Fair Share Distribution (Scenario B).

```javascript
/**
 * Distributes candies progressively (1, 2, 3...) to num_people.
 * @param {number} candies - Total number of candies to distribute
 * @param {number} num_people - Number of people to distribute to
 * @return {number[]} - Array of final candy counts per person
 */
function distributeCandies(candies, num_people) {
    // Initialize an array of size num_people filled with 0s
    const result = new Array(num_people).fill(0);
```

```
    let currentGiveAmount = 1;
    let index = 0;

    // Keep distributing until we have no candies left
    while (candies > 0) {
        // The actual amount we give is either the amount they are supposed to get,
        // OR whatever candies we have left (if we don't have enough to fulfill the request,
        let amountToGive = 0;
        if (candies >= currentGiveAmount) {
            amountToGive = currentGiveAmount;
        } else {
            amountToGive = candies;
        }

        // Add the candies to the current person
        result[index] = result[index] + amountToGive;

        // Subtract from our total stash
        candies = candies - amountToGive;

        // Increment the amount for the NEXT person
        currentGiveAmount++;

        // Move to the next person.
        // Using modulo (%) makes it loop back to 0 when it hits num_people!
        index = (index + 1) % num_people;
    }

    return result;
}

// --- Usage ---
console.log(distributeCandies(10, 3));
// Output: [ 5, 2, 3 ]
```

---

5. LeetCode questions it applies to

- **Distribute Candies to People (LeetCode 1103):** This is the exact algorithm implemented in Section 4. It asks you to distribute items in an increasing sequence (1, 2, 3, 4…) wrapping around the array until you run out.
- **Fair Distribution of Cookies (LeetCode 2305):** A massive twist on the concept. You are given bags of cookies of different sizes. You must distribute them to `K` children such that the "unfairness" (the maximum

total cookies given to any single child) is minimized. This cannot be solved with math; it requires Backtracking (see Variations).

- **Find the Winner of the Circular Game (LeetCode 1823):** Also known as the Josephus Problem. You use modulo arithmetic to figure out exactly who gets "eliminated" in a circle without having to simulate looping around the circle step-by-step.

---

6. Time and Space Complexity

Let C be the total number of Candies (Items). Let K be the number of People (Buckets).

```
Time Complexity Derivation (For Progressive Distribution):
-----------------------------------------------------------
Task                     | Operations           | Big O
-------------------------|----------------------|--------
1. Initialize Array      | K slots              | O(K)
2. While Loop            | Subtracts increasing | O(sqrt(C))
                         | amounts (1+2+3...=C) |
-----------------------------------------------------------
Total Time Complexity: O(K + sqrt(C))
Because we subtract 1, then 2, then 3 from the total, the
number of loops we run is roughly the square root of 2*C.
This is exponentially faster than a naive O(C) loop!

Space Complexity Derivation:
-----------------------------------------------------------
Task                     | Memory Used          | Big O
-------------------------|----------------------|--------
1. Result Array          | K slots              | O(K)
-----------------------------------------------------------
Total Space Complexity: O(K)
We only store the final answers for the K people.
```

---

7. Variations and Common Tweaks

Variation A: The O(1) Math Baseline Distribution **The Problem:** You just want to distribute N items perfectly evenly to K people, with leftovers given to the first few people (Scenario A). You want to do this instantly without any while loops. **The Tweak:** You use simple division and the modulo operator to populate the array directly.

Snippet for Baseline Distribution:

```
function distributeEvenly(items, people) {
    const result = new Array(people);

    // Find the guaranteed baseline amount
    const baseAmount = Math.floor(items / people);

    // Find how many items are leftover
    const leftovers = items % people;

    for (let i = 0; i < people; i++) {
        if (i < leftovers) {
            // The first few people get the base + 1 leftover
            result[i] = baseAmount + 1;
        } else {
            // Everyone else gets just the base
            result[i] = baseAmount;
        }
    }
    return result;
}
```

Variation B: Optimal Fair Share (Minimizing the Maximum) The Problem: You have specific, un-splittable bags of items (e.g., a bag of 8 candies, a bag of 15 candies). You want to distribute the bags to K people to make it as fair as possible. "Fair" means the person with the *most* candies has as *few* as possible. **The Tweak:** You cannot use math because the bags are rigid. You must use **Backtracking** to try giving every bag to every person, keeping track of the best possible maximum.

Snippet for Optimal Fair Share (Backtracking):

```
let minUnfairness = Infinity;

// bags = array of cookie bag sizes, k = number of children
function distributeCookies(bags, k) {
    const children = new Array(k).fill(0);

    function backtrack(bagIndex) {
        // Base case: We handed out all bags!
        if (bagIndex === bags.length) {
            // Find the child who got the MOST cookies in this scenario
            let maxCookiesForOneChild = 0;
            for (let i = 0; i < k; i++) {
                if (children[i] > maxCookiesForOneChild) {
                    maxCookiesForOneChild = children[i];
                }
```

```
            }
            // Update our global minimum unfairness
            if (maxCookiesForOneChild < minUnfairness) {
                minUnfairness = maxCookiesForOneChild;
            }
            return;
        }

        // Try giving the current bag to every single child
        for (let i = 0; i < k; i++) {
            // Pruning optimization: If this child already has more cookies than our
            // best unfairness score, giving them more is pointless! Turn back.
            if (children[i] + bags[bagIndex] >= minUnfairness) {
                continue;
            }

            // Choose
            children[i] = children[i] + bags[bagIndex];

            // Explore
            backtrack(bagIndex + 1);

            // Un-choose (Backtrack)
            children[i] = children[i] - bags[bagIndex];

            // Optimization: If a child has 0 cookies, all remaining children
            // are identical empty buckets. No need to try the others!
            if (children[i] === 0) break;
        }
    }

    backtrack(0);
    return minUnfairness;
}
```